Khulna University of Engineering & Technology
Department of Computer Science and Engineering

Computer Graphics Laboratory: Course Project Report

ThermalSense

A graphics pipeline, end to end, in a military outpost

bunkers, soldiers, UAVs, terrain, vegetation, a solar day and a simulated thermal sensor



The camp at 09:30, rendered by the program (`--capture`, 1920×1080, EPIC preset, RTX 4070). Every building, vehicle and person is built from the unit primitives of Section 6.

Student	Md. Shakhoyat Rahman Shujon
Roll	2107104
Language / API	C++17, OpenGL 3.3 core, GLSL 330
Scene	5,736 drawn parts, 31 composite objects, 26 controllable units

Report date: 9 October 2026. Every figure is a capture made by the program, a plot of the code's own formulas, or a snapshot of the formula-driven workbook `report/workbooks/cg_workbook.xlsx`.

Contents

1 Introduction and objectives **3**

2 The scene in pictures **3**

3 Tools, hardware and setup **5**

4 Architecture and scene layout **5**

5 Controls **6**

Part A. Class concepts along the graphics pipeline 7

6 Primitives and geometry **7**

7 Modelling transformations **9**

8 Hierarchical modelling and the scene graph **10**

9 3D viewing transformation **11**

10 Projection: perspective, orthographic, isometric **13**

11 Projective transformation: frustum, NDC, depth **14**

12 Clipping **15**

13 Hidden surface removal **18**

14 Rasterisation and anti-aliasing **19**

15 Illumination model **21**

16 Shading: flat, Gouraud, Phong **24**

17 Texture mapping **25**

18 Ray tracing concepts in the project **26**

Part B. Advanced work 28

19 Object decomposition catalogue **28**

20 Colour interpolation, with critique **34**

21 Day and night system **36**

22 Curved objects and tessellation **37**

23 Dynamic and controllable objects **38**

24 Reflection: approximated and traced **39**

25 Thermal sensor and image-processing pipeline **40**

26 Game-mode AI **44**

27 Line follower and MicroMouse logic **45**

28 Hard problems and trade-offs **46**

Back matter 48

29 Performance evaluation **48**

30 Limitations and future work **49**

31 Conclusion **50**

A Workbook sheets **52**

1 Introduction and objectives

ThermalSense is a real-time 3D scene of a forward military outpost. It started as a lab exercise: build objects only from unit primitives with a scale, rotation and translation table. It grew into a full renderer and a small game. This report follows the graphics pipeline in the order taught in class [1, 2] (Part A) and then shows the advanced work built on it (Part B). Every claim is backed by a capture, a debug view, an equation, a plot or a workbook sheet.

Objectives.

O1. Model a complete scene from a small set of primitives with hierarchical transforms (Sections 6–8, 19).
O2. Implement and visualise each pipeline stage: viewing, projection, clipping, hidden-surface removal, rasterisation, illumination, shading, texturing (Sections 9–17).
O3. Show ray-tracing concepts inside the same scene (Sections 18, 24).
O4. Drive lighting from a physical day/night model (Section 21).
O5. Build a simulated thermal sensor and an image-processing chain on its output (Section 25).
O6. Make the scene interactive: controllable units, AI, path planning (Sections 23, 26, 27).
O7. Measure performance and state the trade-offs honestly (Sections 28, 29, 30).

What is simulated. The "thermal" image is not real sensing. Each part carries a temperature value; a sensor model blurs, adds noise and fixed-pattern error, and maps heat to a false-colour palette (Section 25). The audit in `docs/audit.md` lists every feature as present, partial or absent; this report describes only present or partial features, and says which.

2 The scene in pictures

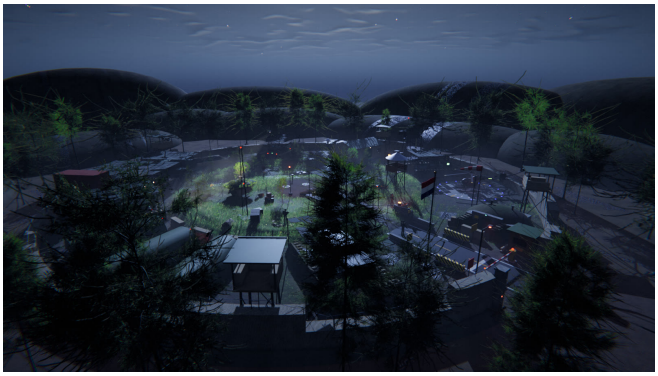
Figure 2.1 shows the same camp across the day and the two ways to use it. All images are 1920×1080 captures from the program at the EPIC preset; the clock is set by the capture script, nothing is painted over.



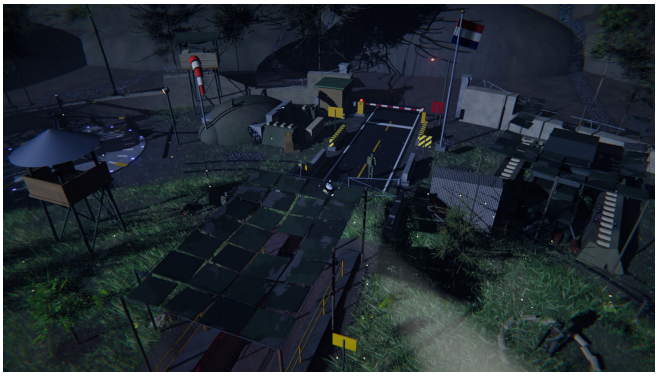
(a) Noon: short shadows, analytic sky with clouds.



(b) Sunset: the key light turns orange (Section 21).



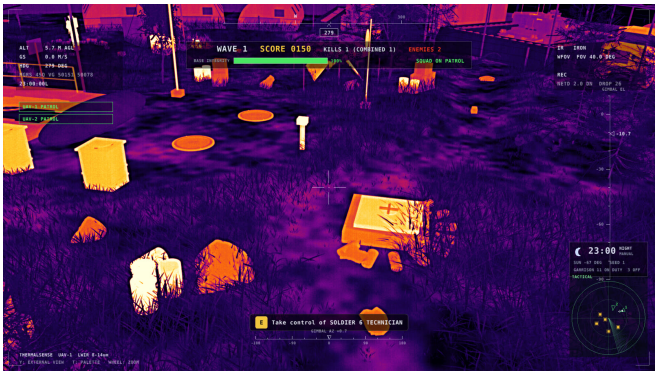
(c) Night: moon key light, lamps, beacon.



(d) Night: UAV searchlights (spot lights) with volumetric cones.



(e) Observation: a soldier in third person, game HUD.



(f) Game: a UAV gimbal camera in thermal, tracking an intruder.

Figure 2.1: The camp from noon to night. Notice that only the light changes between (a), (b) and (c): geometry, materials and camera are the same scene; the sky, sun colour, ambient level and fog colour all follow the sun's elevation. (e) and (f) are the two uses of the scene: inspecting it, and playing the night defence.

Observation and game. The scene is always live. By day there are no enemies, the UAVs stay docked on their pads and the soldiers work (Figure 2.1e); this is the observation setting used for most teaching views. At night intruder rovers attack in waves and the UAVs patrol, detect, lock and fire (Figure 2.1f, Section 26). `Shift+F2` ("day ops") allows waves by day. A separate key that switches enemies on and off at any hour, as the brief suggested for `G`, is not implemented (`G` explodes objects into primitives, Section 19).

3 Tools, hardware and setup

Language, build	C++17; GNU Make (`Makefile`), CMake, and a zig/clang cross-build of `ThermalSense.exe` (`tools/build_windows.sh`)
Graphics API	OpenGL 3.3 core profile, GLSL 330 (shaders embedded in `src/rendering/Shader*.cpp`); GLAD loader, GLFW window
Own libraries	`math/Math3D` (vectors, column-major 4×4 matrices, `lookAt`, `perspective`, `ortho`); no GLM
Third-party	GLFW, GLAD, cgltf, stb_image, meshoptimizer (vegetation assets only); ffmpeg for video
Assets	primitives are procedural; the EPIC vegetation uses CC0 Poly Haven models (`assets/LICENSES.md`)
GPU	NVIDIA GeForce RTX 4070. Captures, plots and the benchmark run headless under WSL2 (Mesa d3d12 driver on the same GPU); the interactive game runs natively on Windows
CPU, memory	Intel Core i9-13900K (32 threads), 16 GB visible to WSL2
Resolution, preset	1920×1080, EPIC preset (MSAA, TAA, GTAO, PCSS shadows, SSR, volumetric fog, bloom, AgX tone mapping), seed 1
Evidence tools	`--capture` (named shots), `--benchmark`, `--export-docs` (every part and matrix as JSON), `--record` (2-minute director cut), F9 session recording with `--replay`

Table 3.1: Tools and setup. Every figure in this report can be regenerated with `tools/v10/capture_all.sh`, `tools/v10/plots.py` and `tools/v10/build_cg_workbook.py`.

Time scale. In automatic mode one in-game day lasts 290.6 s (4.8 real minutes): 10 s per in-game hour by day, 13.3 s per hour by night, and 1.5 times slower while the sun is within 6° of the horizon (`DayCycleConfig`). The clock can also be set or scrubbed by hand.

Determinism. The simulation runs at a fixed 60 Hz tick and every random stream derives from the seed. A recorded session replays bit for bit: 60 s recorded with random input and variable frame times, written to text and replayed, reproduce all 60 per-second world hashes.

4 Architecture and scene layout

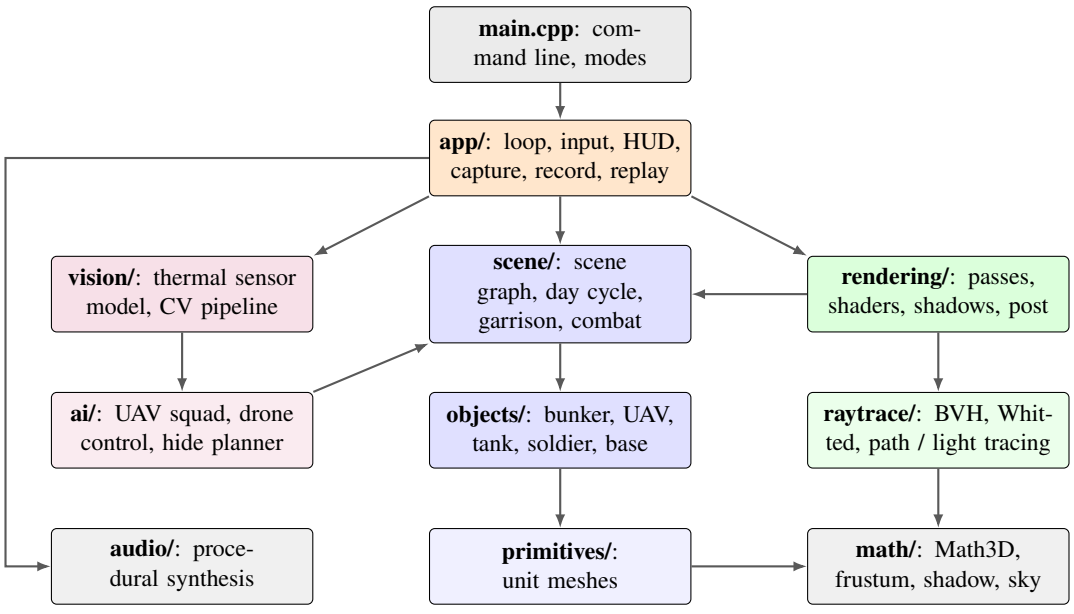


Figure 4.1: Module architecture (directories under `src/`). Notice the split: `scene/` and `objects/` are pure CPU code that runs without a GPU (the `--export-docs` path uses it); everything that touches OpenGL lives in `rendering/` and `app/`.



Figure 4.2: Top-down layout: an orthographic top view rendered by the program (`Ctrl+P`). Labels are placed from the world coordinates in `objects/BaseLayout.h`; the perimeter wall has radius 34 m. Notice that an orthographic top view keeps distances true, so this picture doubles as a scaled site plan; $+y$ points at the viewer.

Frame loop. Each frame: fixed 60 Hz simulation ticks (scene, AI, combat, UAV cameras), render with interpolation between the last two ticks, the UAV images' vision on a worker thread, HUD, present. Offline modes (`--capture`, `--record`) run the same code with a fixed step.

5 Controls

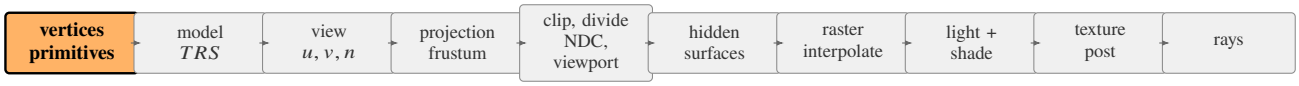
The full table (123+ bindings, contexts, and what each key did before v9) is exported by the program with `--export-keys`; it has no conflicts. Table 5.1 lists the keys used in this report. The brief suggested `N`/`D` for night/day, `G` for game mode, `H` for help and `F9` for a benchmark; those keys were already taken (v1 renderer, movement, explode, local axes, session clip), so the existing bindings stay and the equivalent functions are listed instead.

Cameras and units		**Teaching views**	
`C`	orbit, free, chase, UAV camera, walk	`F`	visible, primitive diagnostic, Z-buffer, thermal
`Home` / `End`	free fly from here / back	`X`	solid, hidden-line, x-ray wireframe
`Tab` / `Shift+Tab`	next / previous unit (soldiers, UAVs, vehicles)	`O`	shading: per object, flat, Gouraud, Phong
`V`	first / third person; UAV chase, FPV, gimbal	`Ctrl+L`	light terms: all, ambient, diffuse, specular
`F4`	list of every moving object's point of view	`Ctrl+P`	perspective, ortho top / front / side, isometric
`K`	cinematic director (orbit / fixed shots)	`G`	explode objects into their primitives
`E`	interact: doors, take control, ride along	`H`	local axes of the hierarchy
`W A S D`	move / fly; `PgUp` / `PgDn` up / down	`Z`	depth test on / off
`Mouse`	look; `LMB` fire; `RMB` aim	`I`	indexed / non-indexed drawing
`Esc` / `P`	release, pause menu / pause	`M`	MicroMouse flood field and guide line
Time of day		`;`	raster, hybrid RT, Whitted, path, light tracing
`F2`	automatic clock on / off	`'`	ray debugger (click a pixel)
`,` / `.`	scrub the clock back / forward (held)	`` ` ``	Frame Explorer: 16 pipeline stages
`/`	jump to the next phase (dawn, day, dusk, night)	`B`	PBR / classic Blinn-Phong
`Esc` →TIME	dawn, noon, dusk, midnight presets	`T` / `Y`	thermal palette / gimbal view
`Shift+F2`	day ops (enemy waves by day)	`F3`	HUD: game, debug (fps, ms, draws, triangles), off
Recording		`F5` – `F10`	quality preset, TAA, SSR, flare, DOF
`F9`	session clip in / out (replayable)	`Shift+F9`	auto-exposure
`F12`	screenshot	`L` / `R`	foliage / rain

Table 5.1: Controls used in this report (`src/scene/KeyMap.cpp`). Camera translation uses `W A S D`; time of day uses `F2`, `,` `.` `/` and the pause menu, so no key serves two purposes.

Part A. Class concepts along the graphics pipeline

6 Primitives and geometry

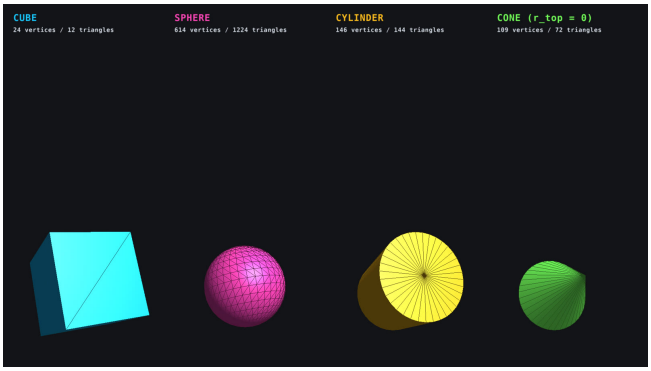


Concept. Every solid object in the scene is a hierarchy of instances of a few unit meshes centred at the origin. A mesh is a vertex array (position + normal) and an index array; one mesh is uploaded once and drawn many times with different model matrices.

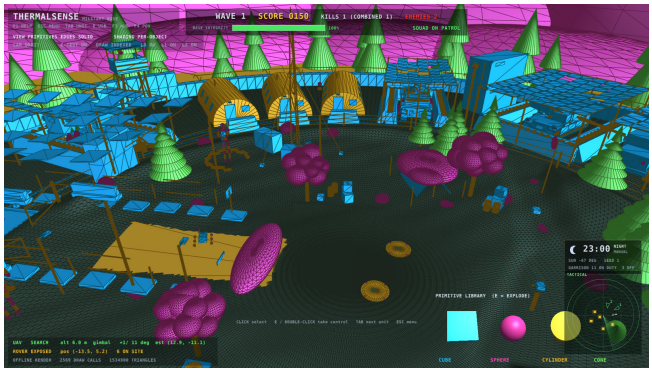
$$\text{sphere: } \mathbf{p}(\theta,\phi) = r\,(\cos\phi\cos\theta,\ \sin\phi,\ \cos\phi\sin\theta) \tag{6.1}$$

$$\text{cylinder, frustum, cone: } \mathbf{p}(\theta,y) = \big(r(y)\cos\theta,\ y,\ r(y)\sin\theta\big),\quad r(y) = r_b + (r_t - r_b)\big(y + \tfrac{1}{2}\big)$$

with $\theta \in [0, 2\pi)$ sampled at s slices and $\phi \in [-\frac{\pi}{2}, \frac{\pi}{2}]$ at k stacks; a cone is a frustum with $r_t = 0$. Cuboids are scaled unit cubes.



(a) The unit primitives, as the program draws them.



(b) G: the camp exploded into primitives, coloured by type.

Figure 6.1: Primitives in the program. Notice in (b) that the whole camp (bunker, vehicles, towers, soldiers) is made of the same few meshes: the colour shows which primitive each part is.

Primitive	Generator call	Vertices	Triangles	Instances
Cube	`createCube()`	24	12	2,478
Sphere	`createSphere(36, 18)`	614	1,224	826
Cylinder	`createCylinder(36)`	146	144	2,133
Cylinder (frustum)	`createCylinder(36, 0.03, 0.5, false)`	72	72	3
Terrain disc	`createTerrainDisc(130, 384, 36)`	49,921	97,440	1
Cone	`createCylinder(36, 0.5, 0, true)`	109	72	295
Primitive instances in the scene				5,736

Table 6.1: The primitive library (`ts --export-docs`). Instances are counted over the whole scene.

Counts: closed-form formulas, exported counts and instances

Primitive	slices s	stacks / rings k	V formula	V	T formula	T	V export	T export	instances (export)
Cube	-	-	6 faces x 4	24	6 faces x 2	12	24	12	2478
Sphere	36	18	(k-1)s + 2	614	2s(k-1)	1224	614	1224	826
Cylinder	36	-	4s + 2	146	4s	144	146	144	2133
Cylinder (frustum)	36	-	2s	72	2s	72	72	72	3
Cone	36	-	3s + 1	109	2s	72	109	72	295
Terrain disc	384	130	1 + k s	49,921	s(2k-1)	99,456	49,921	97,440	1
Grass blade	-	3	2k + 1	7	2k - 1	5	7	5	-
Scene total (library)									5736

Parametric equations

Primitive	Point p(parameters)	Parameter ranges
Cube	p = 0.5 n + 0.5 (a u + b v), face frame (n, u, v) with u x v = n	6 faces, a, b in {-1, +1}; 2 CCW triangles (0,1,2), (0,2,3) per face
Sphere	p = 0.5 (sin(th) cos(ph), cos(th), sin(th) sin(ph)), normal = p / 0.5	th = pi k / stacks, k = 1..stacks-1; ph = 2 pi j / slices; 2 poles; south half mirrored (y → -y)
Cylinder / frustum / cone	p = (r(z) cos(ph), r(z) sin(ph), z), r(-0.5) = r_bottom, r(+0.5) = r_top	ph = 2 pi j / slices; cylinder r = 0.5 / 0.5, frustum 0.5 / 0.03, cone 0.5 / 0 (apex); caps = fans
Terrain disc	p = (r cos(ph), h(x, z), r sin(ph)), r = 36 k / rings	k = 0..130, ph = 2 pi j / 384; h = terrainHeight(x, z); normal by central differences
Grass blade	p = (+-0.625 w(t), t - 0.5, 0), t in {0, 0.38, 0.72}; tip (0, 0.5, 0)	bent and scaled per instance in the vertex shader (wind, lean); one-sided strip

input = value copied from the code, white = formula

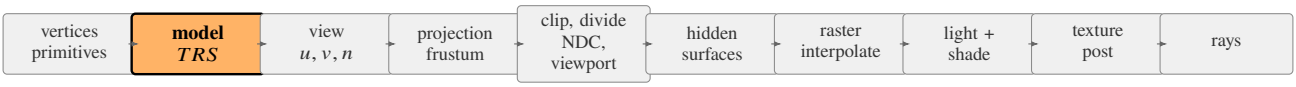
Figure 6.2: Workbook sheet `Primitives`: vertex and triangle counts as formulas of the slice and stack parameters, checked against the counts the program exports. Notice that indexing saves the vertices shared by neighbouring triangles: the sphere draws 1224 triangles from 614 vertices instead of 3672. (full sheet: Appendix A.1)

Where in the project. `src/primitives/Cube.cpp, Sphere.cpp (createSphere(36,18)), Cylinder.cpp`

`(createCylinder(slices, rTop, rBottom, caps)), Terrain.cpp`

Critique. The brief lists eleven primitives. This code has seven: cube (and cuboid), sphere, cylinder, frustum, cone, the terrain disc, and an instanced 5-triangle grass blade (`rendering/Grass.cpp`). There is no torus, wedge, hemisphere or helix generator; where the scene needs a mound (the bunker's earth berm) it uses a flattened sphere ($5.4 \times 1.0 \times 2.0$ m, `objects/Bunker.cpp`). A fixed tessellation is also wasteful: all 826 sphere instances use 1224 triangles each (about 1.0 million triangles) even when a sphere covers a few pixels (Section 22).

7 Modelling transformations



Concept. Each part's local matrix places a unit primitive: scale first, then rotate, then translate. In homogeneous coordinates all three are 4×4 matrices, so one product gives the whole placement.

$$M = T(\mathbf{t})\, R_y(\psi)\, R_x(\theta)\, R_z(\varphi)\, S(\mathbf{s}) \tag{7.1}$$

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_y = \begin{bmatrix} \cos\psi & 0 & \sin\psi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\psi & 0 & \cos\psi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad S = \mathrm{diag}(s_x, s_y, s_z, 1)$$

$$\text{rotation about a pivot } \mathbf{p}: \qquad M_{\text{pivot}} = T(\mathbf{p})\, R\, T(-\mathbf{p}) \tag{7.2}$$

Order matters: $TRS \neq SRT$. Scaling after a rotation would shear a rotated box; translating before a rotation would swing the part around the parent's origin. The turret, gun barrel, rotor and door hinges use Eq. (7.2): each sits in a group node whose origin is the pivot.

Listing 1: The local matrix of every scene node (`src/scene/Transform.h`).

```
// Model matrix M = T * R * S with R = Ry * Rx * Rz: the primitive is scaled first,
// then rotated (roll, pitch, yaw), then translated.
Mat4 matrix() const { return translate(translation) * rotateEulerYXZ(rotationDeg) * ts::scale(scale); }
```



Figure 7.1: Local versus world coordinates (H: local axes of the hierarchy), shown on a soldier's joints: each red/green/blue triad is one local frame. Notice that the axis triads are rotated with their parts: each part's geometry is defined in its own frame, and only the product of matrices up the hierarchy places it in the world.

Inputs exported by the program (part UAV-14, parent UAV-00)

Part	UAV-14 Arm Front-right (Cube, createCube)			
	x	y	z	w
S (scale)	0.0250	0.0180	0.2200	
R_deg (pitch x, yaw y, roll z)	0	-45	0	
T (m)	-0.1061	0	0.1061	
local vertex p (cube corner)	0.5000	0.5000	0.5000	1

Scale and the three rotations

$$S=\begin{bmatrix}0.0250 & 0 & 0 & 0\\ 0 & 0.0180 & 0 & 0\\ 0 & 0 & 0.2200 & 0\\ 0 & 0 & 0 & 1\end{bmatrix},\quad R_x=\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix},\quad R_y=\begin{bmatrix}0.7071 & 0 & -0.7071 & 0\\ 0 & 1 & 0 & 0\\ 0.7071 & 0 & 0.7071 & 0\\ 0 & 0 & 0 & 1\end{bmatrix},\quad R_z=\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}$$

Translation, local matrix, parent and world matrices

$$T=\begin{bmatrix}1 & 0 & 0 & -0.1061\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0.1061\\ 0 & 0 & 0 & 1\end{bmatrix},\quad M=\begin{bmatrix}0.0177 & 0 & -0.1556 & -0.1061\\ 0 & 0.0180 & 0 & 0\\ 0.0177 & 0 & 0.1556 & 0.1061\\ 0 & 0 & 0 & 1\end{bmatrix}$$

$$W_{\text{par}}=\begin{bmatrix}-1 & 0 & -8.700\times10^{-8} & 0\\ 0 & 1 & 0 & 7.5000\\ 8.700\times10^{-8} & 0 & -1 & 12.5000\\ 0 & 0 & 0 & 1\end{bmatrix},\quad W=\begin{bmatrix}-0.0177 & 0 & 0.1556 & 0.1061\\ 0 & 0.0180 & 0 & 7.5000\\ -0.0177 & 0 & -0.1556 & 12.3939\\ 0 & 0 & 0 & 1\end{bmatrix}$$

The vertex through the pipeline

stage	x	y	z	w
local p	0.5000	0.5000	0.5000	1
world = World p	0.1750	7.5090	12.3073	1
eye = V world	-6.3735	1.1973	-21.5719	1
clip = P eye	-7.6882	2.5676	21.4755	21.5719
NDC = clip / w	-0.3564	0.1190	0.9955	1
window (x px, y px top-left, depth, -)	617.8569	475.7275	0.9978	-

Checks

check	value
max |M - export|	1.800×10^{-8}
max |World - export|	2.092×10^{-7}
inside clip volume |x|,|y|,|z| ≤ w	yes
eye-space distance -z_eye (m)	21.5719

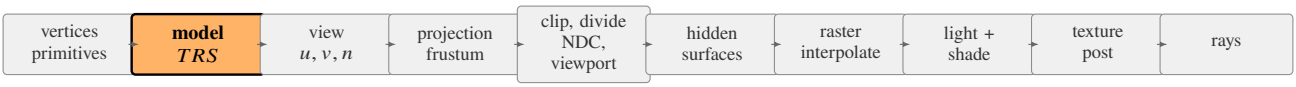
input = value copied from the code, white = formula

Figure 7.2: Workbook sheet `Transform_Trace`: one real vertex of one real UAV part traced through S, R, T, the parent matrix, view, projection, perspective divide, NDC and viewport, each matrix as a live 4×4 grid. Notice the check column: the world matrix computed in the sheet matches the one the program exports. (full sheet: Appendix A.12)

Where in the project. `src/scene/Transform.h (matrix), src/math/Math3D.cpp (translate, rotateEulerYXZ, scale), src/objects/Builder.cpp`

Critique. Euler angles in a fixed order are easy to tabulate (the brief's T/R/S table) but suffer gimbal lock at $\theta = \pm90°$; the camera rig and the interpolation between simulation ticks therefore use quaternions (`RigPose.rot`). Reflection and shear are not used by any part.

8 Hierarchical modelling and the scene graph



Concept. Articulated objects are trees. Moving a node moves everything below it, because each world matrix is the parent's world matrix times the local one. Animation then only changes a few local angles.

$$W_{\text{node}} = W_{\text{parent}}\, M_{\text{node}} = M_{\text{root}}\, M_1\, M_2 \cdots M_{\text{node}} \tag{8.1}$$

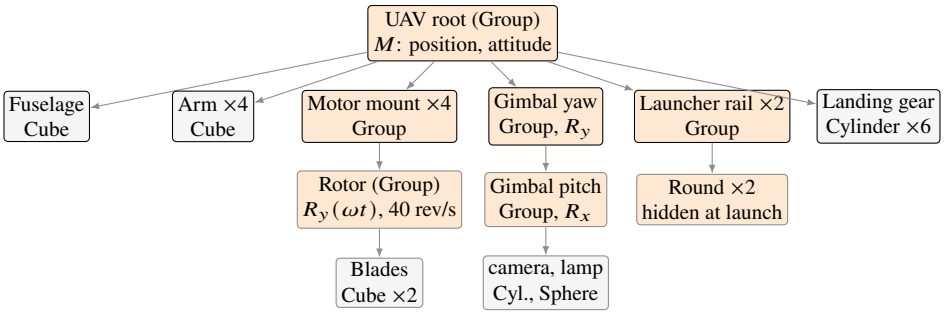
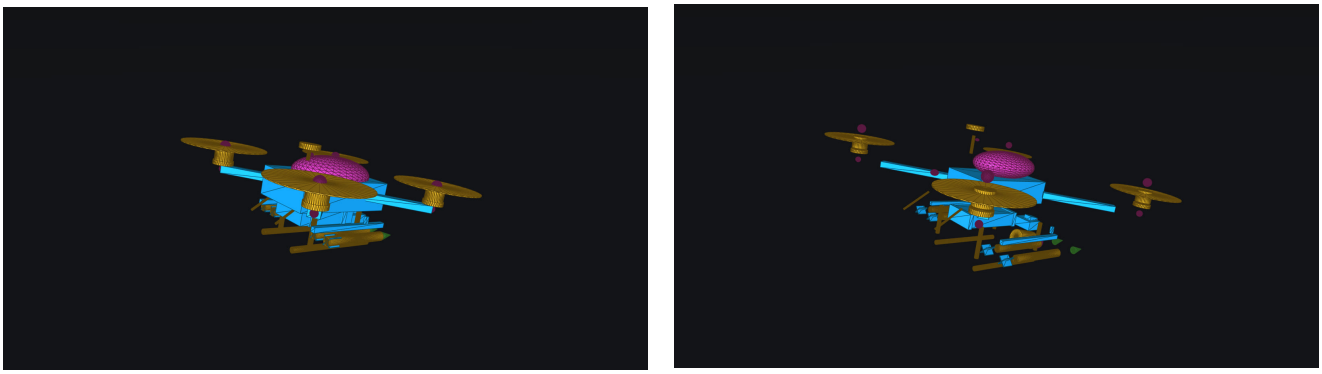


Figure 8.1: Scene graph of the UAV (192 nodes for the two UAVs in `--export-docs`; simplified). Orange nodes are groups (a transform, no mesh); grey nodes draw a primitive. Notice that the camera and the searchlight hang under two gimbal groups: the visual-servo controller only writes two angles, and Eq. (8.1) carries the lamp, its light direction and the camera frame along.



(a) UAV assembled.

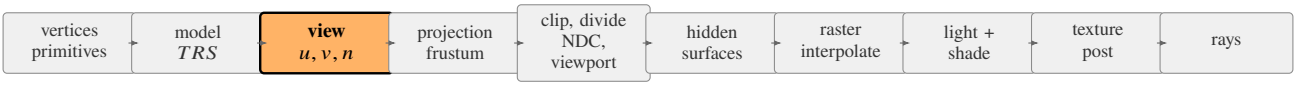
(b) UAV exploded along its hierarchy (G).

Figure 8.2: The same tree drawn assembled and exploded. Notice that the explode offset is applied per group, so sub-assemblies (motor mount with rotor, gimbal with camera) move out together.

Where in the project. `src/scene/SceneNode.h (world = parent.world * local.matrix()), src/scene/Scene.cpp updateWorld(), src/objects/Uav.cpp`

Critique. The world matrices are recomputed for all 5,736 parts every tick on the CPU, even for static buildings. This costs little here (the simulation step is a few milliseconds), but a larger scene would cache static subtrees or move the product to the GPU.

9 3D viewing transformation



Concept. The view matrix moves the world into the camera's frame. It is built from the eye **e**, the point looked at **a** and an up hint **up**, by an orthonormal basis and a translation.

$$\mathbf{n} = \frac{\mathbf{e}-\mathbf{a}}{\|\mathbf{e}-\mathbf{a}\|}, \quad \mathbf{u} = \frac{\mathbf{up}\times\mathbf{n}}{\|\mathbf{up}\times\mathbf{n}\|}, \quad \mathbf{v} = \mathbf{n}\times\mathbf{u}, \qquad V = \begin{bmatrix} u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ v_x & v_y & v_z & -\mathbf{v}\cdot\mathbf{e} \\ n_x & n_y & n_z & -\mathbf{n}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.1}$$

The code writes the same matrix with $\mathbf{f} = -\mathbf{n}$ (forward), $\mathbf{s} = \mathbf{u}$ (side) and $\mathbf{u} = \mathbf{v}$ (`math/Math3D.cpp lookAt`); the camera looks down its $-z$ axis.



(a) Soldier, first person.



(b) Third person (spring arm).



(c) UAV-mounted gimbal camera.

Figure 9.1: Three cameras on the same scene; each is just a different (**e**, **a**, **up**). Notice in (c) that the UAV camera's eye and axes come from the gimbal nodes of Figure 8.1: the view matrix is the inverse of the camera node's world matrix.

Inputs and the u, v, n basis

Input / derived	x	y	z
target (m)	0	0.8000	0
up	0	1	0
yaw (deg)	32		
pitch (deg)	27		
distance (m)	34		
fov y (deg)	50		
width (px)	1920		
height (px)	1080		
near (m)	0.0500		
far (m)	600		
near, comparison (m)	0.5000		
aspect = width / height	1.7778		
eye = target + d (cos p sin y, sin p, cos p cos y)	16.0535	16.2357	25.6910
eye - target	16.0535	15.4357	25.6910
n = normalize(eye - target)	0.4722	0.4540	0.7556
up x n	0.7556	0	-0.4722
u = normalize(up x n)	0.8480	0	-0.5299
v = n x u	-0.2406	0.8910	-0.3850
code: s = normalize(f x up), f = -n; |s - u|	0		
u . v, u . n, v . n (orthogonal: 0)	0	0	0

View and perspective matrices

$$V = \begin{bmatrix} 0.8480 & 0 & -0.5299 & 0 \\ -0.2406 & 0.8910 & -0.3850 & -0.7128 \\ 0.4722 & 0.4540 & 0.7556 & -34.3632 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad P = \begin{bmatrix} 1.2063 & 0 & 0 & 0 \\ 0 & 2.1445 & 0 & 0 \\ 0 & 0 & -1.0002 & -0.1000 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Depth mapping (24-bit), near 0.05 m and 0.5 m

z_eye (m)	z_ndc (near 0.05)	depth d (near 0.05)	metres per depth step (near 0.05)	z_ndc (near 0.5)	depth d (near 0.5)	metres per depth step (near 0.5)
-0.0500	-1	0	2.980×10^{-9}	clipped	clipped	clipped
-0.1000	8.334×10^{-5}	0.5000	1.192×10^{-8}	clipped	clipped	clipped
-0.5000	0.8002	0.9001	2.980×10^{-7}	-1	0	2.978×10^{-8}
-1	0.9002	0.9501	1.192×10^{-6}	8.340×10^{-4}	0.5004	1.191×10^{-7}
-2	0.9502	0.9751	4.768×10^{-6}	0.5013	0.7506	4.764×10^{-7}
-5	0.9802	0.9901	2.980×10^{-5}	0.8015	0.9008	2.978×10^{-6}
-10	0.9902	0.9951	1.192×10^{-4}	0.9016	0.9508	1.191×10^{-5}
-34	0.9972	0.9986	0.0014	0.9722	0.9861	1.377×10^{-4}
-50	0.9982	0.9991	0.0030	0.9817	0.9908	2.978×10^{-4}
-100	0.9992	0.9996	0.0119	0.9917	0.9958	0.0012
-200	0.9997	0.9998	0.0477	0.9967	0.9983	0.0048
-400	0.9999	1.0000	0.1907	0.9992	0.9996	0.0191
-600	1	1	0.4291	1	1	0.0429

input = value copied from the code, white = formula

Figure 9.2: Workbook sheet `View_Projection`: **u**, **v**, **n** from cross products for the default orbit camera, the view matrix, the perspective matrix, and the depth table used in Section 11. (full sheet: Appendix A.13)

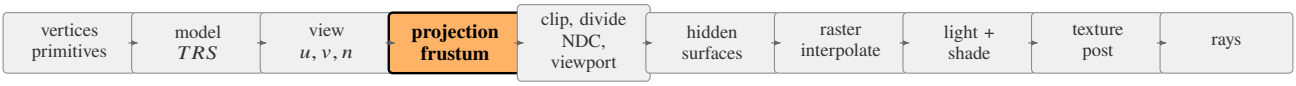
Camera system. Orbit (default), free fly (Home), chase, UAV camera, walk, first and third person on any soldier

(Tab, V), UAV chase / FPV / gimbal, any moving object's point of view (F4) and the cinematic director (K). Changes of camera blend over 0.6 s on a critically damped spring (`CameraRig::kBlendTime`).

Where in the project. `src/math/Math3D.cpp lookAt, src/scene/Camera.cpp compute(), src/scene/CameraRig.cpp, src/scene/UavView.cpp`

Critique. When **up** ∥ **n** (looking straight down) the cross product vanishes. The top view of Section 10 avoids this by using **up** = −**z**; the free camera clamps its pitch to ±85°.

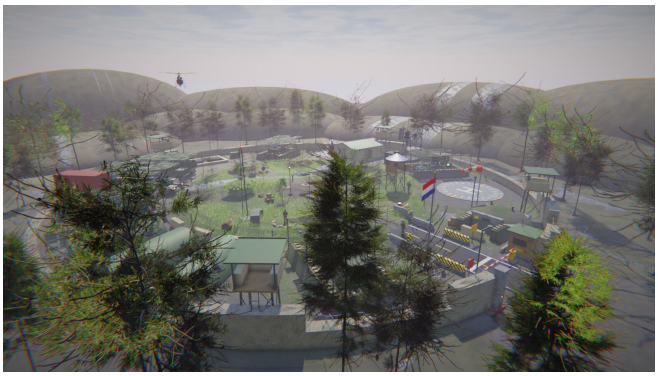
10 Projection: perspective, orthographic, isometric



Concept. A projection maps the view volume to the canonical cube $[-1,1]^3$. Perspective shrinks distant objects (projectors meet at the eye); parallel projections keep sizes (projectors are parallel). Multiview drawings use orthographic views along the principal axes; an isometric view looks along $(1,1,1)$ so the three axes are foreshortened equally.

$$P_{\text{persp}} = \begin{bmatrix} \frac{f}{a} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{-(F+N)}{F-N} & \frac{-2FN}{F-N} \\ 0 & 0 & -1 & 0 \end{bmatrix}, \; f = \cot\tfrac{\text{fov}}{2}; \qquad P_{\text{ortho}} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{F-N} & -\frac{F+N}{F-N} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{10.1}$$

Isometric view direction: elevation $\arctan(1/\sqrt{2}) = 35.26°$, azimuth 45°; each axis is foreshortened by $\sqrt{2/3} = 0.816$.



(a) Perspective (fov 56°).



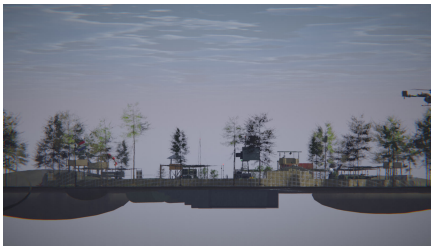
(b) Isometric (axonometric).



(c) Orthographic top (plan).



(d) Orthographic front (from $+z$).



(e) Orthographic side (from $+x$).

Figure 10.1: The same scene and time under five projections (Ctrl+P). Notice in (a) that the near huts look larger than identical far ones and parallel roads converge; in (b)–(e) equal objects stay equal anywhere in the frame. In (c) the plan shows true distances (it is the site plan of Figure 4.2); in (d) and (e) everything is flattened onto a vertical plane, so depth is lost and only heights can be read.

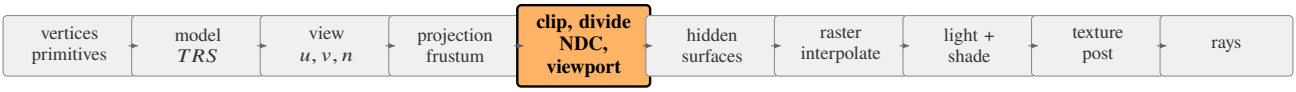
Method	Visual result	Cost	When to use
Perspective	depth cue, convergence, realistic	same matrix cost; non-linear depth (Section 11)	play views, video
Orthographic multiview	true lengths along two axes, no depth cue	same; linear depth	plans, elevations, shadow maps
Isometric	three faces visible, equal foreshortening	same; linear depth	layout overviews, games

Table 10.1: Differences between the projections.

Where in the project. `src/math/Math3D.cpp perspective / ortho; src/app/AppGame.cpp composeView (teaching projections); the sun's shadow cascades use ortho`

Critique. The screen-space effects (SSAO/GTAO, SSR, fog, TAA) reconstruct positions assuming a perspective projection. In the orthographic views they still run, so fog and reflections in (d) and (e) are not physically meaningful; the views are for geometry, not for lighting.

11 Projective transformation: frustum, NDC, depth



Concept. After the projection matrix a vertex is in clip space (x_c, y_c, z_c, w_c) with $w_c = -z_{\text{eye}}$. Dividing by w_c (the perspective divide) gives normalised device coordinates; the viewport maps them to pixels. Depth becomes a hyperbolic function of distance, which concentrates precision near the camera.

$$(x_n, y_n, z_n) = \frac{(x_c, y_c, z_c)}{w_c}, \quad x_s = \tfrac{W}{2}(x_n{+}1),\ y_s = \tfrac{H}{2}(y_n{+}1), \quad z_w = \tfrac{1}{2}(z_n{+}1) = \tfrac{1}{2}\left(\tfrac{F{+}N}{F{-}N} - \tfrac{2FN}{(F{-}N)\,d}{+}1\right),\ d = -z_{\text{eye}} \tag{11.1}$$

$$\Delta d_{\min} \approx \frac{d^2\,(F-N)}{F\,N}\,2^{-24} \quad \text{(smallest distinguishable depth step with a 24-bit buffer)} \tag{11.2}$$

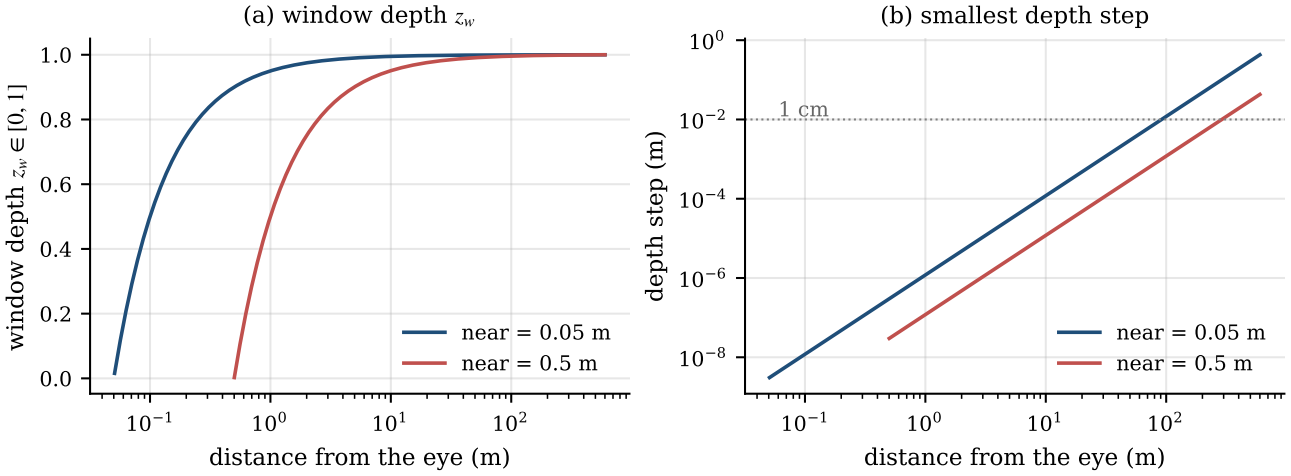
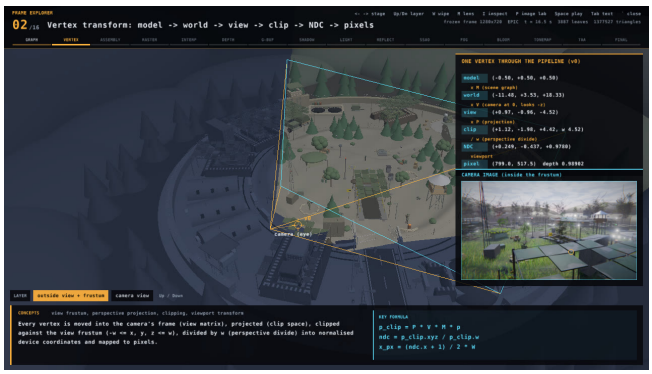


Figure 11.1: Depth mapping of the program's projection ($F = 600$ m, plotted from Eq. (11.1)). Notice in (a) that with $N = 0.05$ m half of the depth range is used in the first 0.1 m, and in (b) that at 100 m two surfaces closer than about 1 m cannot be ordered. Raising N to 0.5 m buys a factor of ten everywhere; this is the z-fighting trade-off of Section 28.

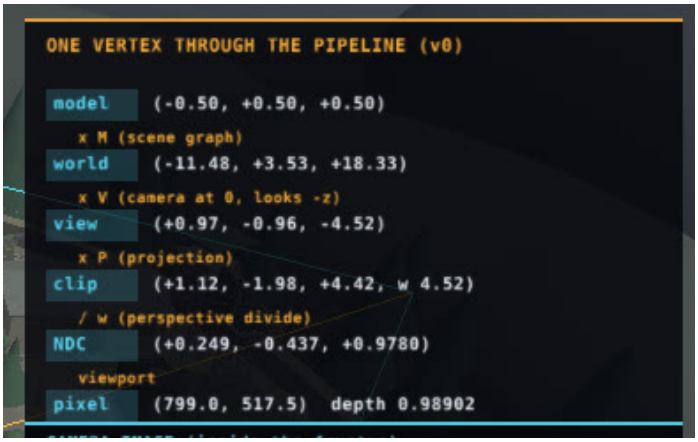


(a) F: linearised Z-buffer view.



(b) Frame Explorer: vertex stage (clip space).

Zoomed panels of the Frame Explorer frame:



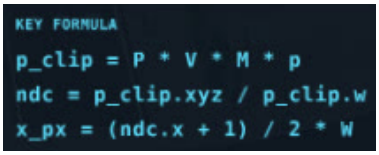
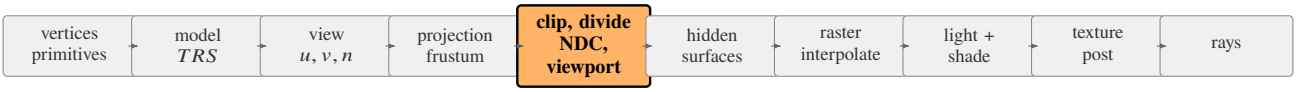


Figure 11.2: Depth in the program. Notice that (a) shows depth linearised (d recovered from z_w by inverting Eq. (11.1)); the raw z_w would be almost white everywhere beyond a few metres.

Where in the project. `src/math/Math3D.cpp perspective; near plane 0.05 m and far 600 m in src/app/AppGame.cpp composeView; Z-buffer view shader in src/rendering/ShaderSources.cpp`

Critique. The near plane of 0.05 m exists so a first-person rifle and a soldier's own hands are not clipped. It costs depth precision far away (Figure 11.1b). Reversed-Z with a floating-point depth buffer would remove the problem but needs `glClipControl` (OpenGL 4.5), which the 3.3 baseline does not have.

12 Clipping



Concept. Primitives outside the view volume must be removed and primitives crossing it cut, before rasterisation. The GPU clips every triangle against the six planes $-w_c \le x_c, y_c, z_c \le w_c$. The program adds two CPU steps: whole parts are culled against the frustum before they are submitted, and grass is culled per cell.

$$\text{plane } i: \ \mathbf{a}_i \cdot \mathbf{p} + d_i \ge 0 \text{ inside}; \qquad \text{sphere } (\mathbf{c}, r) \text{ rejected if } \exists i: \ \mathbf{a}_i \cdot \mathbf{c} + d_i < -r \tag{12.1}$$

The six planes come from the rows of PV (Gribb–Hartmann): left = $\text{row}_4 + \text{row}_1$, right = $\text{row}_4 - \text{row}_1$, and so on.

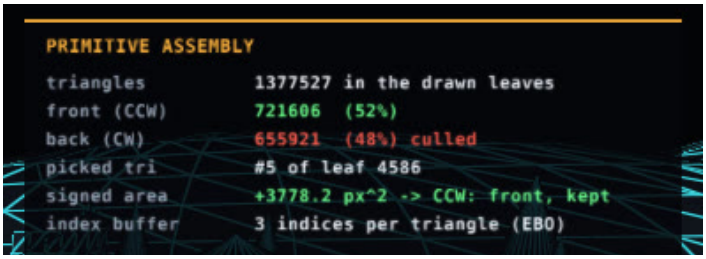


(a) Near/far clipping in an orthographic front view.



(b) Frame Explorer: primitive assembly and the frustum.

Zoomed panels of the Frame Explorer frame:



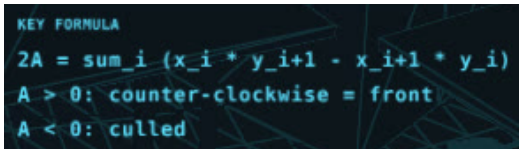


Figure 12.1: Clipping in the program. In (a) the view volume is an 80 m slab around the camp: notice that the backdrop hills and the far terrain are cut flat at the near and far planes, and the cut faces show the inside of the terrain mesh, exactly as clipping produces (no capping). (b) shows the assembled triangles of the frame.

1. Cohen–Sutherland outcodes

line	x0	y0	x1	y1	T0	B0	R0	L0	code0	T1	B1	R1	L1	code1	bits 0	bits 1	AND of codes	decision
L1	2	2	8	6	0	0	0	0	0	0	0	0	0	0	0000	0000	0	accept (both inside)
L2	-2	-1	-1	9	0	1	0	1	5	1	0	0	1	9	0101	1001	1	reject (same side)
L3	-3	4	6	12	0	0	0	1	1	1	0	0	0	8	0001	1000	0	clip (subdivide)
L4	-2	3	12	5	0	0	0	1	1	0	0	1	0	2	0001	0010	0	clip (subdivide)

2. Liang–Barsky parameters

line	dx	dy	p1	p2	p3	p4	q1	q2	q3	q4	t enter = max(0, q/p, p<0)	t exit = min(1, q/p, p>0)	parallel outside	visible	x0'	y0'	x1'	y1'
L1	6	4	-6	6	-4	4	2	8	2	6	0	1	no	yes	2	2	8	6
L2	1	10	-1	1	-10	10	-2	12	-1	9	2	0.9000	no	no	-	-	-	-
L3	9	8	-9	9	-8	8	-3	13	4	4	0.3333	0.5000	no	yes	0	6.6667	1.5000	8
L4	14	2	-14	14	-2	2	-2	12	3	5	0.1429	0.8571	no	yes	0	3.2857	10	4.7143

3. Sutherland–Hodgman, stage 1 (left edge)

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	-2	3	7	-2	0	1	0	1.8889	I, P
2-3	7	-2	12	10	1	1	-	-	P
3-1	12	10	-2	3	1	0	0	4	I
out	(0, 1.89) (7, -2) (12, 10) (0, 4)								
4 vertices									

Stage 2 (right edge)

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	0	1.8889	7	-2	1	1	-	-	P
2-3	7	-2	12	10	1	0	10	5.2000	I
3-4	12	10	0	4	0	1	10	9	I, P
4-1	0	4	0	1.8889	1	1	-	-	P
out	(7, -2) (10, 5.2) (10, 9) (0, 4) (0, 1.89)								
5 vertices									

input = value copied from the code, white = formula

Figure 12.2: Workbook sheet `Clipping`: Cohen–Sutherland outcodes, Liang–Barsky parameters and Sutherland–Hodgman steps. These are worked textbook examples computed by formulas, not code in the program. (full sheet: Appendix A.14)

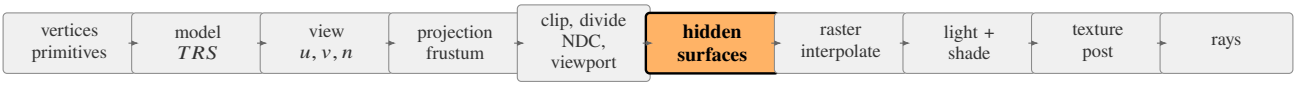
Method	What it does	Cost	Used in the program
Cohen–Sutherland	4-bit outcodes, trivial accept/reject, then intersect	cheap for mostly inside/outside lines	no (workbook)
Liang–Barsky	parametric t range, at most one division per edge	fewer intersections	no (workbook)
Sutherland–Hodgman	polygon against one plane at a time	output grows per plane	by the GPU (in clip space)
Frustum culling	whole object against 6 planes (Eq. 12.1)	6 dot products per part	yes, `math/Frustum`

Table 12.1: Clipping methods compared.

Where in the project. `src/math/Frustum.h,cpp (plane extraction, sphere test); grass cell culling in src/rendering/Grass.cpp`

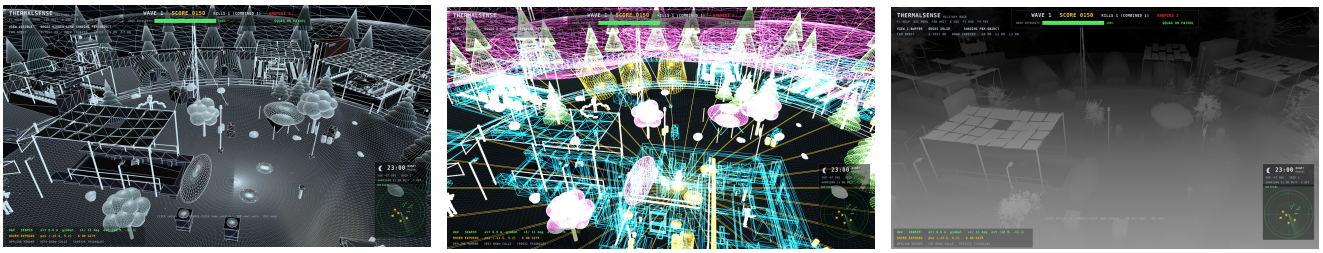
Critique. There is no interactive user-defined clip window (the brief's item); clipping is shown through the orthographic slab and the workbook. Frustum culling with bounding spheres is conservative: a long thin part (a wall) has a large sphere and is drawn even when only its sphere touches the frustum.

13 Hidden surface removal



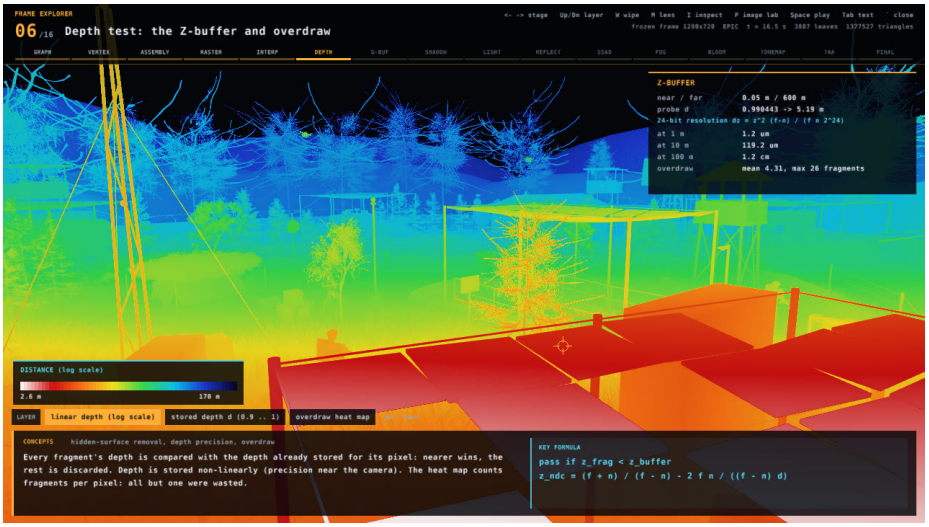
Concept. Two tests decide what is visible. Back-face culling drops triangles facing away from the eye before rasterisation (about half of a closed mesh). The Z-buffer keeps, per pixel, the nearest depth so far and rejects fragments behind it, in any drawing order.

$$\text{back face if } (\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0) \cdot (\mathbf{p}_0 - \mathbf{e}) \geq 0; \qquad \text{write } (c, z) \text{ iff } z < Z[x, y] \tag{13.1}$$

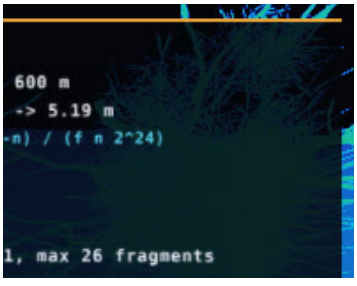


(a) Hidden-line wireframe (X). **(b)** X-ray wireframe: no depth test. **(c)** The Z-buffer itself.

Figure 13.1: Hidden-surface removal made visible. Notice that (a) and (b) draw the same edges: in (a) the depth buffer of the solid pass hides edges behind surfaces, in (b) everything shows through, including the underground bunker.



Zoomed panels of the Frame Explorer frame:



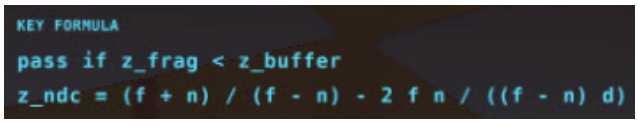


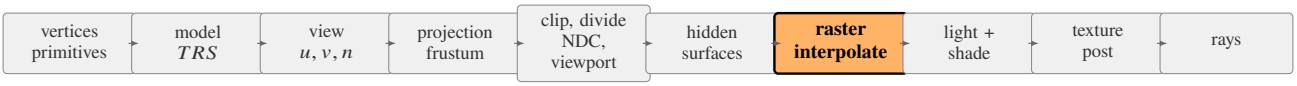
Figure 13.2: Frame Explorer depth stage. Notice that depth is resolved per fragment, so intersecting parts (a barrel through a wall) are cut exactly along their intersection line; a painter's sort by object could not do this.

Where in the project. `glEnable(GL_CULL_FACE) / GL_DEPTH_TEST in src/rendering/RendererFrame.cpp;` Z `depth test toggle; depth pre-pass at EPIC`

Critique. Culling is disabled for the terrain (an open surface must cast shadows from both sides) and for foliage cards and grass (thin, seen from both sides), so these pay for both faces. A depth pre-pass fills the Z-buffer first,

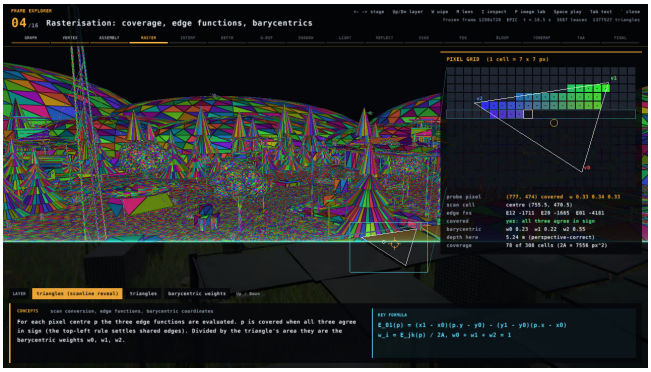
so the expensive lighting shader runs about once per pixel instead of once per overlapping fragment; translucent effects (rotor blur discs, beams) need sorting (Section 28).

14 Rasterisation and anti-aliasing



Concept. Rasterisation finds the pixels whose centres a triangle covers (edge functions) and interpolates the vertex attributes with barycentric weights. Because the triangle was projected, attributes must be interpolated perspective-correctly: interpolate a/w and $1/w$, then divide.

$$E_{ij}(\mathbf{x}) = (\mathbf{x} - \mathbf{p}_i) \times (\mathbf{p}_j - \mathbf{p}_i), \quad \lambda_k = \frac{E_k}{E_0 + E_1 + E_2}, \qquad a = \frac{\sum_k \lambda_k a_k / w_k}{\sum_k \lambda_k / w_k} \tag{14.1}$$



(a) Frame Explorer: the raster scan of one triangle.



(b) Affine (left) vs perspective-correct (right).

Zoomed panels of the Frame Explorer frame:

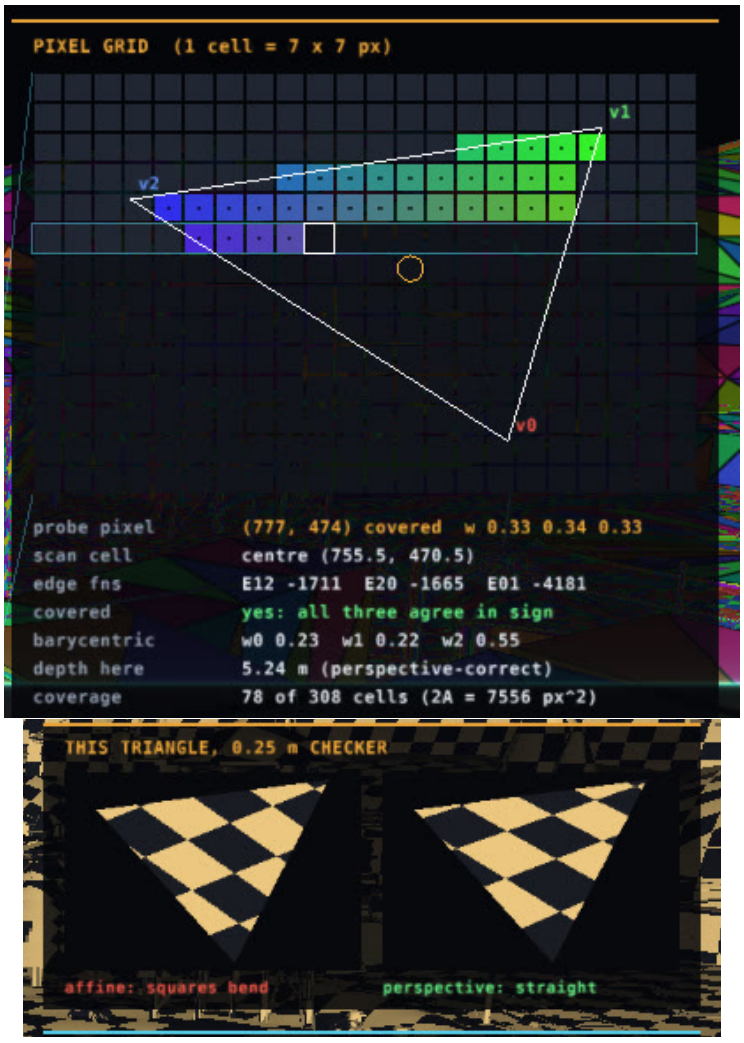
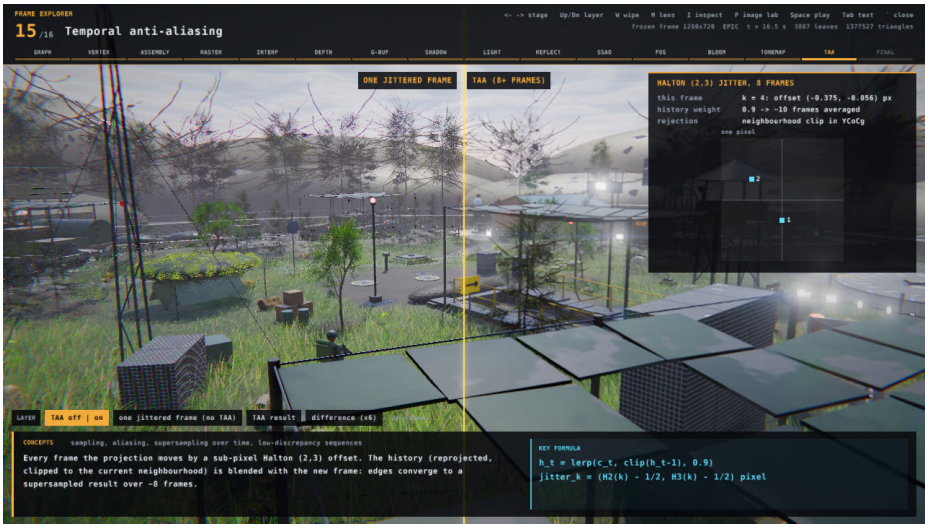
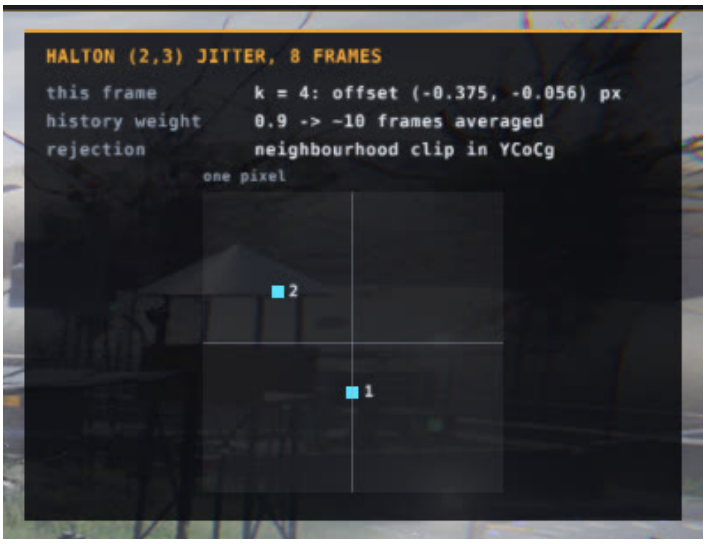


Figure 14.1: Rasterisation in the Frame Explorer. Notice in (b) that the affine checker bends along the diagonal of each quad, because a is interpolated linearly in screen space; the perspective-correct one stays straight.

Anti-aliasing. Edges sampled once per pixel form stairs. The program uses MSAA (several coverage samples, one shading per pixel): 2× at LOW, 4× at HIGH, 8× at ULTRA, and 4× plus TAA at EPIC. TAA jitters the projection by a Halton(2,3) sub-pixel offset every frame and blends the history, clipped to the variance box of the current 3×3 neighbourhood in YCoCg (`ShadersPostFx.cpp kTaaFrag`).



Zoomed panels of the Frame Explorer frame:



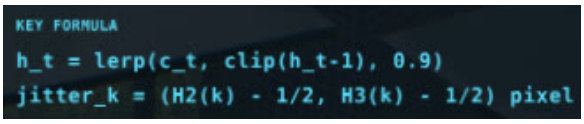
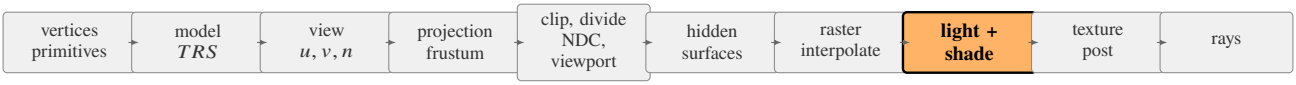


Figure 14.2: Frame Explorer TAA stage. Notice the thin wires and grass blades: without temporal accumulation they break into dotted lines; with it they are continuous, at the cost of slight blur in motion.

Where in the project. `Rasterisation is done by the GPU; the explorer re-draws it on the CPU for teaching (src/rendering/FrameExplorer.cpp, math/ExplorerMath.cpp). MSAA/TAA in src/rendering/RendererFrame.cpp, ShadersPost.cpp`

Critique. TAA trades aliasing for ghosting: a fast UAV leaves a faint trail for a few frames. The v9 vegetation work measured that, with a static camera, pixels without vegetation still change by about 2.4/255 per frame from the jitter alone, and vegetation pixels by 2.6–6.6/255 (target 1.5; not met, `docs/v9_notes/v9-notes.md`).

15 Illumination model



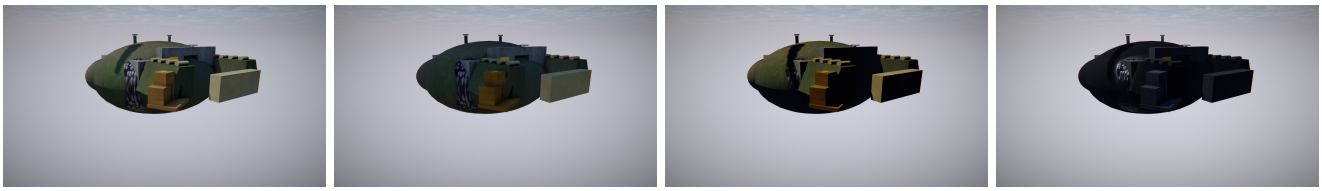
Concept. The local illumination model sums an ambient term, and for every light a diffuse (Lambert) and a specular (Blinn–Phong) term, scaled by the light's attenuation and, for a spot light, its cone factor.

$$I = k_a I_a + \sum_l f_{\text{att},l}\, c_l\, I_l \Big[k_d\, \max(0, \mathbf{N}\cdot\mathbf{L}_l) + k_s\, \max(0, \mathbf{N}\cdot\mathbf{H}_l)^n \Big], \qquad \mathbf{H} = \frac{\mathbf{L}+\mathbf{V}}{\|\mathbf{L}+\mathbf{V}\|} \tag{15.1}$$

$$f_{\text{att}} = \frac{1}{1 + k_l d + k_q d^2}, \qquad c_{\text{spot}} = \left(\text{clamp}\!\left(\frac{\cos\gamma - \cos\theta_o}{\cos\theta_i - \cos\theta_o}, 0, 1 \right) \right)^2 \tag{15.2}$$

Light	Type	Parameters (exported by the program)	When
Sun	directional	direction and colour from the solar model (Section 21), intensity up to 3.2	sun above -1°
Moon	directional	intensity 0.95, colour (0.55, 0.62, 0.80)	night
Beacon	point	$k_l = 0.35$, $k_q = 0.45$, pulsing 0.6–2.0	night
UAV searchlight	spot	$\theta_i = 9^\circ$, $\theta_o = 14^\circ$, $k_l = 0.05$, $k_q = 0.03$	night, UAV airborne
Practical lamps	point	up to 16 at once (floodlights, lamps, fire barrel)	dusk to dawn
Ambient	hemisphere / SH	night constant; by day the irradiance of the drawn sky	always

Table 15.1: Light sources (`--export-docs`, `scene/DayCycle`).



(a) All terms. **(b)** Ambient only. **(c)** Diffuse only. **(d)** Specular only.

Figure 15.1: The terms of Eq. (15.1) one at a time on the ammunition bunker at 10:30 (`Ctrl+L`). Notice: ambient (b) has no direction, so the shape reads only through colour; diffuse (c) depends on $\mathbf{N} \cdot \mathbf{L}$ and darkens faces turned from the sun; specular (d) is small and sits on edges and rims, and moves with the camera because $\mathbf{H}$ depends on $\mathbf{V}$.

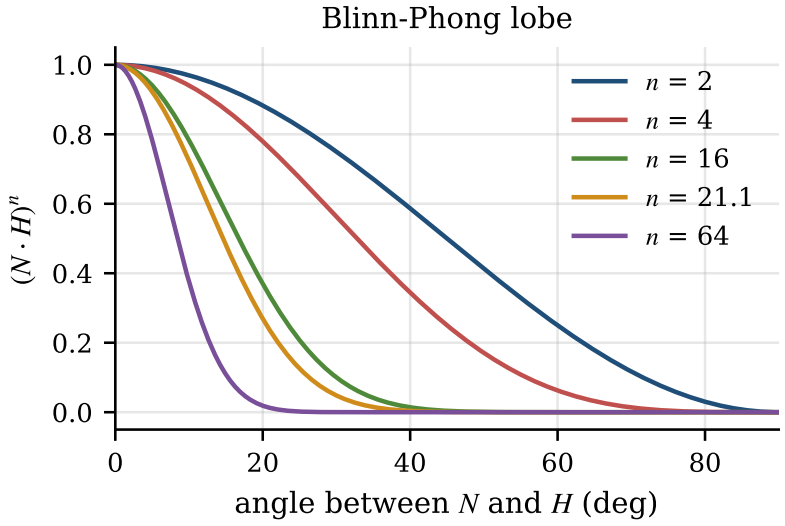


Figure 15.2: The specular lobe $(\mathbf{N} \cdot \mathbf{H})^n$ for shininess values that occur in the scene (2 to 64). Notice that $n = 64$ gives a highlight eight times narrower than $n = 2$.

Geometry: sun, moon and searchlight at one roof point

Quantity	Noon	Sunset	Night (moon)	UAV spot (beam axis)	UAV spot (roof point)
hour (clock)	12	17.5000	23	23	23
H = 15 (hour - 12) (deg)	0	82.5000	165	165	165
sin e = sin phi sin d + cos phi cos d cos H	0.8878	0.0889	-0.9187	-0.9187	-0.9187
sun elevation e (deg)	62.6000	5.0981	-66.7312	-66.7312	-66.7312
daylight w = smoothstep(-8, 6, e)	1	0.9881	0	0	0
surface point P.x (m)	0	0	0	0	0
surface point P.y (m)	-0.4500	-0.4500	-0.4500	-0.4500	-0.4500
surface point P.z (m)	-1.3000	-1.3000	-1.3000	5.8195	-1.3000
to light (unnormalised) x	0	-0.9883	0.5598	0	0
to light (unnormalised) y	0.8878	0.0889	0.3868	7.8060	7.8060
to light (unnormalised) z	-0.4602	-0.1244	-0.7328	6.5505	13.6700
distance d = |to light| (m)	1	1	1.0000	10.1903	15.7417
L.x (unit)	0	-0.9883	0.5598	0	0
L.y (unit)	0.8878	0.0889	0.3868	0.7660	0.4959
L.z (unit)	-0.4602	-0.1244	-0.7328	0.6428	0.8684
angle to spot axis (deg)	-	-	-	0.4138	20.2741
spot cone c^2, c = clamp((cos a - cos o)/(cos i - cos o))	-	-	-	1	0
attenuation 1 / (1 + kl d + kq d^2)	-	-	-	0.2162	0.1084
air mass m (Kasten-Young)	1.1257	10.1423	-	-	-
light intensity (scalar)	3.2000	1.8600	0.9500	1.0811	0
N . L (clamped at 0)	0.8878	0.0889	0.3868	0.7660	0.4959
L + V (H before normalising) x	0.4514	-0.5368	1.0112	0.5261	0.4514
L + V y	1.3570	0.5581	0.8560	1.3129	0.9651
L + V z	0.2988	0.6346	0.0262	1.2941	1.6274
N . H	0.9288	0.5574	0.6460	0.6848	0.4962
(N . H)^n (0 when N . L = 0)	0.8627	0.3107	0.4173	0.4690	0.2462

Per-channel ambient, diffuse and specular terms

Term (channel)	Noon	Sunset	Night (moon)	UAV spot (beam axis)	UAV spot (roof point)
Il = light colour x intensity R	3.1760	1.0747	0.5225	1.0811	0
Il = light colour x intensity G	3.1560	0.6804	0.5890	1.0271	0
Il = light colour x intensity B	3.1166	0.2727	0.7600	0.8865	0
ambient Ia R	0.3400	0.3330	0.0600	0.0600	0.0600
ambient Ia G	0.3600	0.3527	0.0650	0.0650	0.0650
ambient Ia B	0.4000	0.3927	0.0850	0.0850	0.0850
ambient kd Ia R	0.1020	0.0999	0.0180	0.0180	0.0180
ambient kd Ia G	0.1080	0.1058	0.0195	0.0195	0.0195
ambient kd Ia B	0.1120	0.1100	0.0238	0.0238	0.0238
diffuse kd Il (N.L) R	0.8459	0.0286	0.0606	0.2484	0
diffuse kd Il (N.L) G	0.8406	0.0181	0.0683	0.2360	0
diffuse kd Il (N.L) B	0.7748	0.0068	0.0823	0.1901	0
specular ks Il (N.H)^n R	0.2082	0.0254	0.0166	0.0385	0
specular ks Il (N.H)^n G	0.2069	0.0161	0.0187	0.0366	0
specular ks Il (N.H)^n B	0.2043	0.0064	0.0241	0.0316	0
total I R	1.1561	0.1539	0.0952	0.3050	0.0180
total I G	1.1555	0.1400	0.1065	0.2921	0.0195
total I B	1.0911	0.1232	0.1302	0.2455	0.0238
luminance Y of total (Rec. 709)	1.1510	0.1418	0.1058	0.2915	0.0195

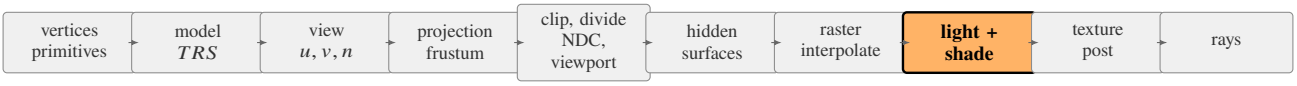
input = value copied from the code, white = formula

Figure 15.3: Workbook sheet `Illumination`: the three terms for one roof point at noon, sunset, under the moon and under a UAV searchlight. (full sheet: Appendix A.15)

Where in the project. `kLighting in src/rendering/ShaderSources.cpp (compose(), brdf()); lights in src/scene/Scene.cpp, src/scene/DayCycle.cpp`

Critique. At HIGH and above the program uses a physically based GGX/Cook–Torrance BRDF by default; `B` switches to classic Blinn–Phong. The term split (`Ctrl+L`) applies to the scene-object and terrain shaders; grass and foliage use their own shaders and always show all terms.

16 Shading: flat, Gouraud, Phong



Concept. Shading decides where Eq. (15.1) is evaluated. Flat: once per triangle with the face normal. Gouraud: at the vertices, colours interpolated. Phong: normals interpolated and the model evaluated per fragment.

$$\text{Gouraud: } I(\mathbf{x}) = \textstyle\sum_k \lambda_k \, I(\mathbf{N}_k), \qquad \text{Phong: } I(\mathbf{x}) = I\!\left(\frac{\sum_k \lambda_k \mathbf{N}_k}{\left\|\sum_k \lambda_k \mathbf{N}_k\right\|}\right) \tag{16.1}$$



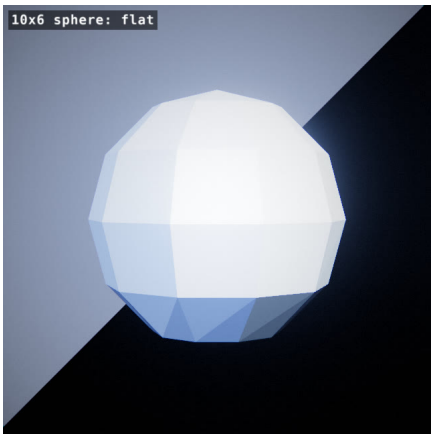
(a) UAV, flat.



(b) UAV, Gouraud.



(c) UAV, Phong.



(d) Low-polygon sphere, flat.

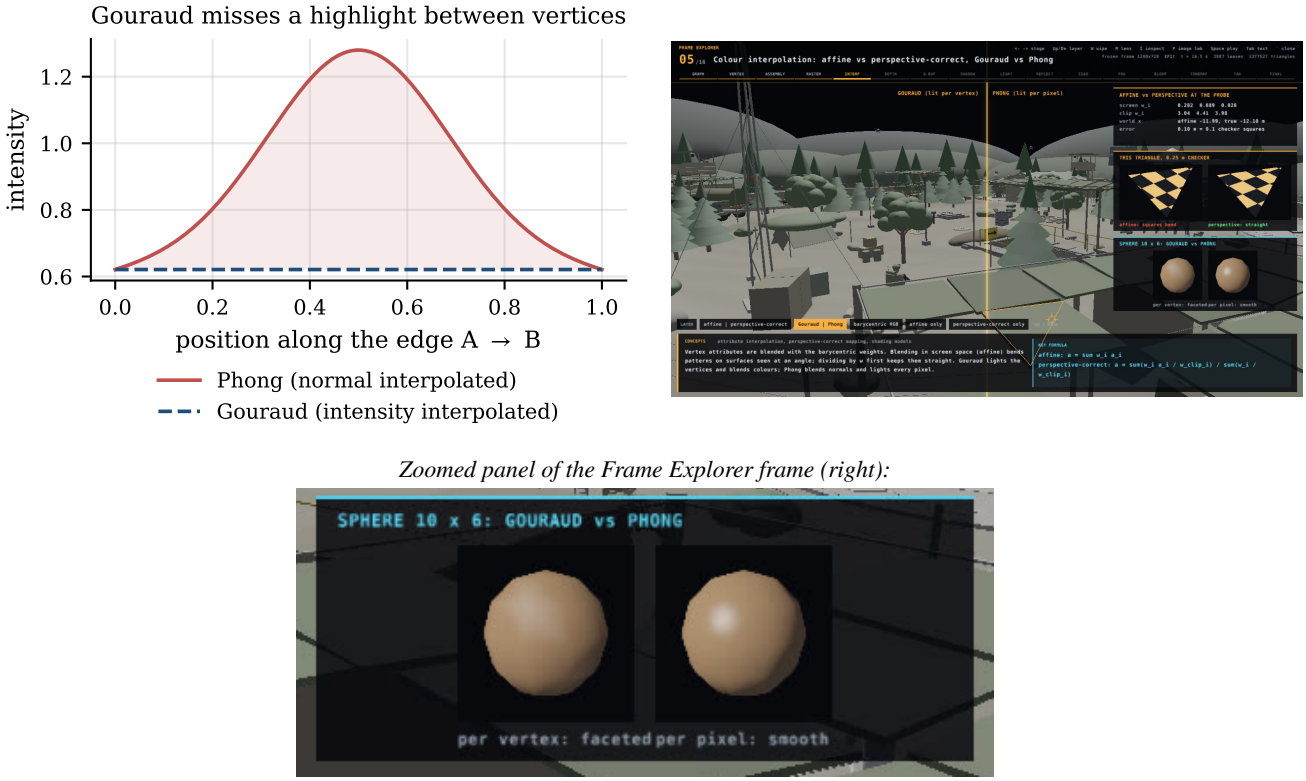


(e) Sphere, Gouraud.



(f) Sphere, Phong.

Figure 16.1: The same UAV and a low-polygon sphere under flat, Gouraud and Phong shading (0, lab figure). Notice the facets in flat shading, the Mach bands (bright and dark lines the eye adds at the slope changes of a piecewise-linear intensity) along Gouraud's triangle edges, and the round, sharp highlight that only Phong keeps.



Zoomed panel of the Frame Explorer frame (right):

Figure 16.2: Left: intensity along one edge whose highlight peak falls between the vertices. Gouraud interpolates the two dim vertex values and misses the peak entirely; Phong recovers it. Right: the same comparison in the Frame Explorer.

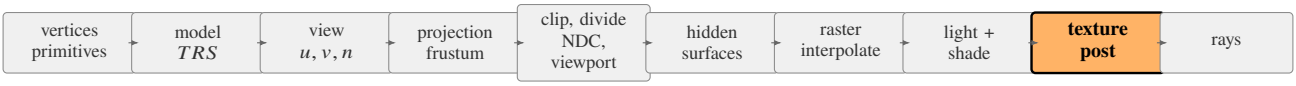
Method	Visual result	Cost	When to use
Flat	faceted, constant per face	1 evaluation per triangle	hard-edged boxes, CAD views
Gouraud	smooth, but highlights smeared or lost, Mach bands	1 per vertex	dense meshes, diffuse-only
Phong	smooth, correct highlights	1 per fragment	everything shiny; the default here

Table 16.1: Differences between the shading methods.

Where in the project. `ShadingOverride in src/rendering/Renderer.h; flat: a geometry shader computes the face normal and lights the centroid once (kFlatGeom); Gouraud: lighting in the vertex shader (kGouraudVert); Phong: per fragment (src/rendering/ShaderSources.cpp)`

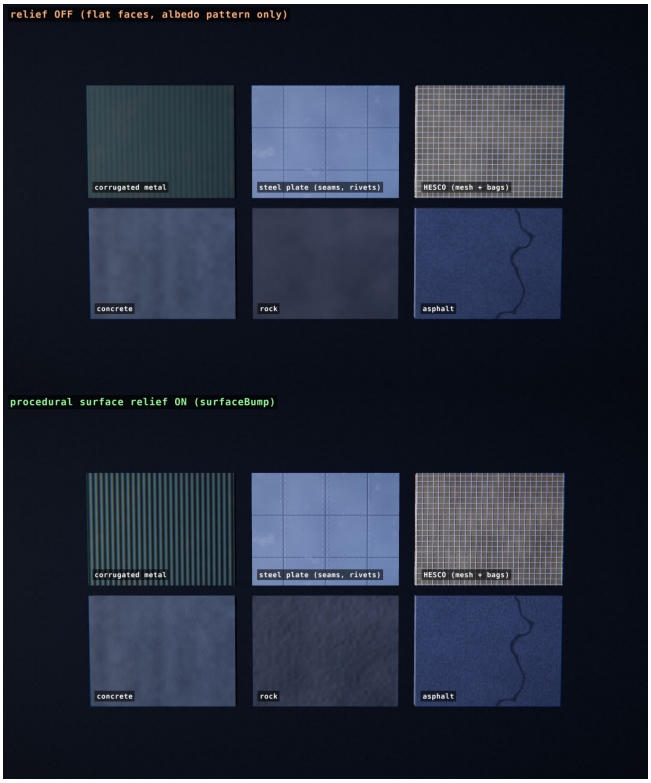
Critique. The terrain uses a hybrid: per-vertex irradiance re-aimed per fragment by a detail normal (`terrainShade`), because per-fragment lighting of 97 440 terrain triangles under every lamp would cost more than it shows.

17 Texture mapping

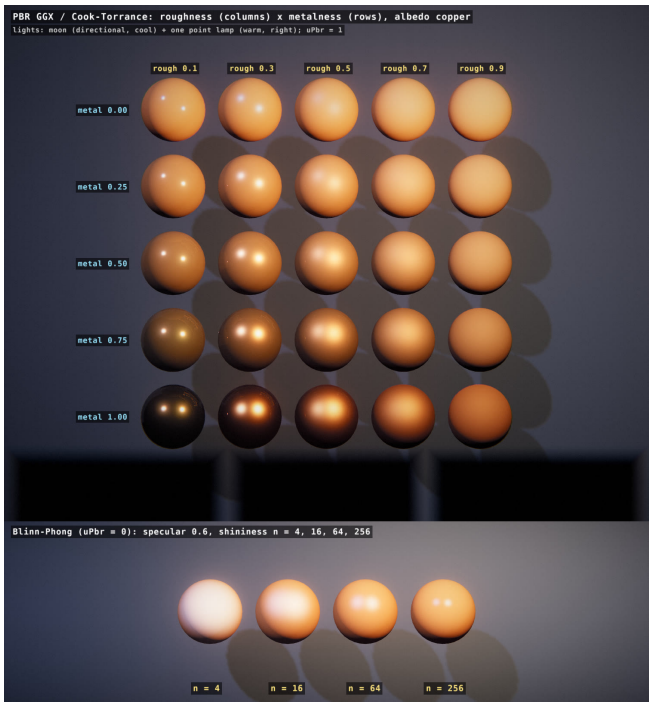


Concept. A texture adds detail without geometry. A surface point gets coordinates (u, v) (stored per vertex, or computed from position), the texture is sampled there with filtering and mip-maps, and the result modulates a material property. A procedural texture computes the value with a function instead of an image.

$$\text{bump: } \mathbf{N}' = \text{normalize}\big(\mathbf{N} - \nabla h\big), \qquad \text{mip level: } \lambda = \log_2 \max\left(\left\|\frac{\partial(u,v)}{\partial x}\right\|, \left\|\frac{\partial(u,v)}{\partial y}\right\|\right) \tag{17.1}$$



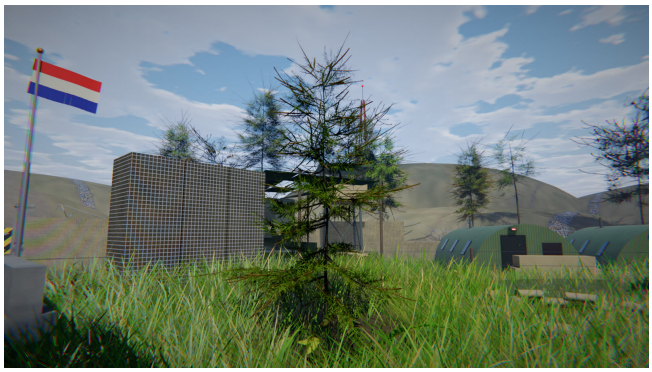
(a) Procedural relief (bump) off / on.



(b) Procedural materials on unit primitives.



(c) Image textures: CC0 grass atlas on instanced blades.



(d) Image textures: glTF tree with alpha-tested leaf cards.

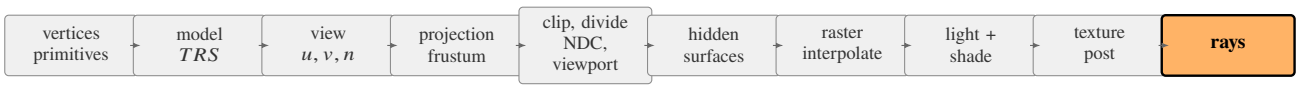
Figure 17.1: Textures in the program. Notice in (a) that the geometry is identical; only the normal is perturbed by Eq. (17.1), so the silhouette stays straight while the lighting shows relief. (c) and (d) use image textures with mip-maps; the leaves are cut out by alpha-to-coverage.

Where coordinates come from. Primitives carry no UV set. Their surfaces (concrete, camouflage, sandbags, wood, metal, terrain soil and grass) are procedural: noise and cellular patterns evaluated from the world position in the fragment shader (`surfaceAlbedo`, `surfaceBump`, `terrainShade`). The v9 vegetation assets bring real UV-mapped image textures (glTF, `stb_image`).

Where in the project. `surfaceAlbedo / surfaceBump / terrainShade in src/rendering/ShaderSources.cpp; image textures in src/rendering/Foliage.cpp, Grass.cpp`

Critique. Procedural world-space patterns need no UV unwrapping and never stretch, but they slide over a moving part unless evaluated in object space, and they cannot be mip-mapped; aliasing of fine noise is left to MSAA and TAA.

18 Ray tracing concepts in the project



Concept. A ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is intersected with the scene. Rays answer three questions here: what is under the cursor (picking), is a point lit (shadow rays), and what does a mirror or glass show (reflection and refraction rays, Whitted [3]). The program includes a CPU/GPU ray tracer over the same scene with a BVH.

$$\text{sphere: } t^2(\mathbf{d}\cdot\mathbf{d}) + 2t\,\mathbf{d}\cdot(\mathbf{o}-\mathbf{c}) + \|\mathbf{o}-\mathbf{c}\|^2 - r^2 = 0; \quad \text{plane: } t = \frac{(\mathbf{p}_0-\mathbf{o})\cdot\mathbf{n}}{\mathbf{d}\cdot\mathbf{n}} \tag{18.1}$$

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\mathbf{n}, \qquad \mathbf{t} = \eta\,\mathbf{d} + \big(\eta\cos\theta_i - \sqrt{1-\eta^2(1-\cos^2\theta_i)}\big)\mathbf{n},\ \eta = \tfrac{n_1}{n_2} \tag{18.2}$$



Figure 18.1: The same view in the five render modes ([;]): raster, hybrid (raster plus ray-traced reflections), Whitted, path tracing and light tracing. Notice the shiny surfaces: the raster image can only reflect what is on screen, Whitted traces exact mirror reflections and shadow rays, and the path tracer adds soft indirect light and soft shadows.



Figure 18.2: The ray debugger ([']), click a pixel) draws the ray tree of one pixel: primary ray, shadow rays to each light, reflection and refraction children.

1. Ray–sphere

1. Ray-sphere	x	y	z
ray origin o	0	2	-10
direction (unnormalised)	0.1000	-0.0500	1
d = normalize	0.0994	-0.0497	0.9938
sphere centre c	0	1	0
radius r	1.5000		
oc = o - c	0	1	-10
a = d . d	1		
b = 2 d . oc	-19.9755		
c' = oc . oc - r^2	98.7500		
discriminant b^2 - 4 a c'	4.0222		
t1 = (-b - sqrt(disc)) / 2a (near)	8.9850		
t2 = (-b + sqrt(disc)) / 2a (far)	10.9905		
hit p = o + t1 d	0.8929	1.5535	-1.0706
normal n = (p - c) / r	0.5953	0.3690	-0.7138
d . n	-0.6685		
reflection r = d - 2 (d . n) n	0.8953	0.4437	0.0395
check |r| (= 1)	1.0000		

2. Ray–plane (ground)

2. Ray-plane (ground y = 0)	x	y	z
ray origin o	0	5	0
direction (unnormalised)	1	-1	0.5000
d = normalize	0.6667	-0.6667	0.3333
plane normal n	0	1	0
point on plane p0	0	0	0
n . d	-0.6667		
t = n . (p0 - o) / (n . d)	7.5000		
hit p = o + t d	5	0	2.5000
reflection r = d - 2 (d . n) n	0.6667	0.6667	0.3333

3. Snell refraction (air to glass)

3. Snell refraction (air → glass)	x	y	z
incident d (unnormalised)	0.5000	-1	0
d = normalize	0.4472	-0.8944	0
normal n (facing against d)	0	1	0
n1, n2	1	1.5000	
eta = n1 / n2	0.6667		
cos i = -d . n	0.8944		
k = 1 - eta^2 (1 - cos^2 i)	0.9111		
t = eta d + (eta cos i - sqrt k) n	0.2981	-0.9545	0
incidence angle (deg)	26.5651		
refraction angle (deg)	17.3461		
Snell check n1 sin i - n2 sin t	0		
Fresnel R (exact, fresnelDielectric)	0.0409		
Schlick R0 = ((n1 - n2) / (n1 + n2))^2	0.0400		

4. Total internal reflection

incidence angle (deg)	k = 1 - eta^2 sin^2 i	TIR?	Fresnel R
critical angle asin(1/1.5)	41.8103		
20	0.7368	no	0.0417
30	0.4375	no	0.0552
41	0.0316	no	0.3798
42	-0.0074	yes	1
60	-0.6875	yes	1

5. Whitted recursion

bounce k	kr^k	cumulative	share of the infinite sum
0	1	1	0.5000
1	0.5000	1.5000	0.7500
2	0.2500	1.7500	0.8750
3	0.1250	1.8750	0.9375
4	0.0625	1.9375	0.9688
5	0.0312	1.9688	0.9844
6	0	1.9688	0.9844

input = value copied from the code, white = formula

Figure 18.3: Workbook sheet `Ray_Tracing`: ray–sphere and ray–plane intersections, reflection, Snell refraction and the contribution of each recursion level. (full sheet: Appendix A.18)

Uses in the game. Mouse picking casts a ray through the clicked pixel (middle click jumps into that object); the interaction prompt tests a segment against wall boxes for line of sight; the UAV searchlight's beam length is a ray marched to the terrain and refined by bisection; a third-person camera arm is a sphere cast against the walls.

Where in the project. `src/raytrace/ (Intersect, Bvh, RayTracer), src/rendering/RayTracerGL.cpp, src/scene/Interact.cpp (segmentClear), src/scene/Scene.cpp (beam length)`

Critique. The ray-traced modes are teaching modes: at 1080p the path tracer runs at reduced internal resolution (0.5) and accumulates over frames, so it is not used in play. The raster renderer approximates reflections with screen-space reflections (Section 24).

Part B. Advanced work

19 Object decomposition catalogue

Builds on: primitives (6), modelling transforms (7), hierarchy (8).

Every object is listed with its assembled view, its exploded view where the program draws one (G), and a parts table generated from `--export-docs`: part kinds grouped by name and primitive, their count, the local scale S of the first part of each kind, its base colour, and its material (specular weight k_s and Blinn–Phong shininess n). The workbook sheet `Decomposition` lists every single part of the UAV and a soldier with T, R, S and parent

(Figure 19.11). Table 19.1 gives the totals for all objects.

Object (per unit)	×n	Parts	Groups	Anim.	Cube	Sphere	Cylinder	Frustum	Cone
Terrain	1	1	1	0	–	–	–	–	–
Backdrop	1	120	47	3	0	24	10	0	86
Environment	1	652	89	2	46	201	237	0	168
Command Bunker	1	427	18	18	309	19	99	0	0
Autonomous UAV	2	79	17	13	34	13	27	1	4
Base	1	1253	33	0	558	41	654	0	0
Barracks hall	1	277	5	1	225	2	50	0	0
UAV pads	1	18	3	0	4	8	6	0	0
Target Rover	6	26	2	8	6	12	8	0	0
Main Battle Tank	3	255	29	118	170	5	80	0	0
Scout Helicopter	1	91	19	17	40	21	28	1	1
AA Gatling Gun	1	78	5	26	19	5	54	0	0
Air-defence radar	1	108	4	4	62	3	42	0	1
SAM launcher	1	112	9	10	74	5	32	0	1
MLRS rocket truck	1	136	22	28	45	9	76	0	6
Ammunition bunker	1	43	2	0	19	10	14	0	0
Camp camouflage	1	658	21	0	277	48	309	0	24
Boom barrier	1	20	2	2	17	2	1	0	0
Windsock	1	15	7	6	1	0	14	0	0
Flag	1	47	14	13	40	2	5	0	0
Soldier	8	75	20	15	16	39	20	0	0
Sky Clock	1	1	1	0	0	1	0	0	0
Scene total (all units)		5736							

Table 19.1: Every composite object, per unit: parts, group nodes, animated nodes and primitives used, with the number of units n of each design (`--export-docs`).



Part	Primitive	n	S (cm)	RGB	k_s / n
Round fin	Cube	16	0.2×3.0×3.0		0.50 / 64
Round nose	Cone	4	2.2×2.2×4.0		0.90 / 128
Round body	Cylinder	4	2.2×2.2×17.0		0.50 / 64
Landing strut	Cylinder	4	1.2×1.2×9.0		0.60 / 64
Motor cooling ring	Cylinder	4	5.0×5.0×1.0		0.60 / 64
Rotor blur disc	Cylinder	4	20.5×20.5×0.2		0.20 / 16
Prop blade	Cube	4	20.0×0.4×1.8		0.30 / 32
Prop hub	Sphere	4	2.5×2.5×2.5		0.90 / 128
Motor	Cylinder	4	4.5×4.5×3.5		0.90 / 128
24 more part kinds		31			

79 parts: Cone 4, Cube 34, Cylinder 27, Frustum 1, Sphere 13

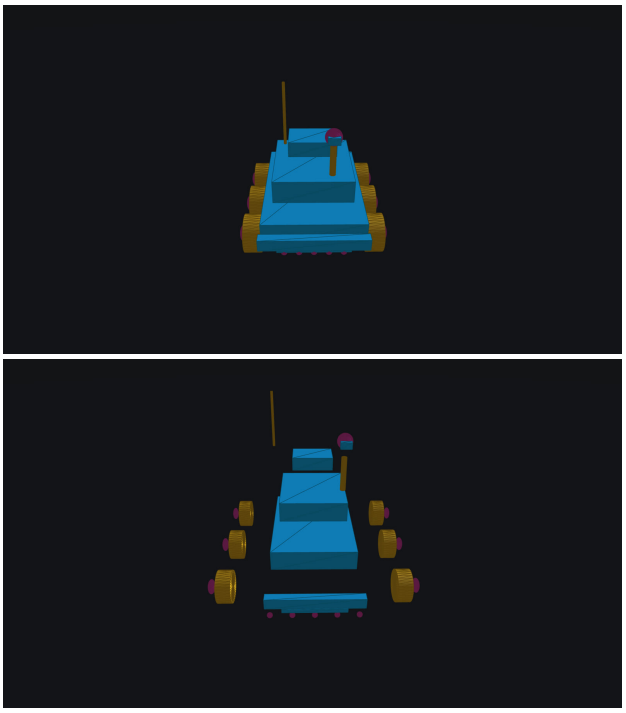
Figure 19.1: UAV. Notice how small parts dominate the count: 16 fins on four rounds are a fifth of the parts but invisible from the camp.



Part	Primitive	n	*S* (cm)	RGB	k_s / n
Deltoid	Sphere	2	15.5×17.0×15.0		0.06 / 2
Elbow pad	Sphere	2	10.0×11.0×5.5		0.10 / 3
Forearm muscle	Sphere	2	12.0×18.0×12.0		0.06 / 2
Hand	Sphere	2	9.0×12.0×8.0		0.10 / 3
Forearm	Cylinder	2	11.0×11.0×26.0		0.06 / 2
Elbow joint	Sphere	2	13.0×13.0×13.0		0.06 / 2
Upper arm	Cylinder	2	13.0×13.0×30.0		0.06 / 2
Shoulder joint	Sphere	2	13.0×13.0×13.0		0.06 / 2
Trapezius	Sphere	2	20.0×8.5×15.0		0.06 / 2
39 more part kinds		57			

75 parts: Cube 16, Cylinder 20, Sphere 39

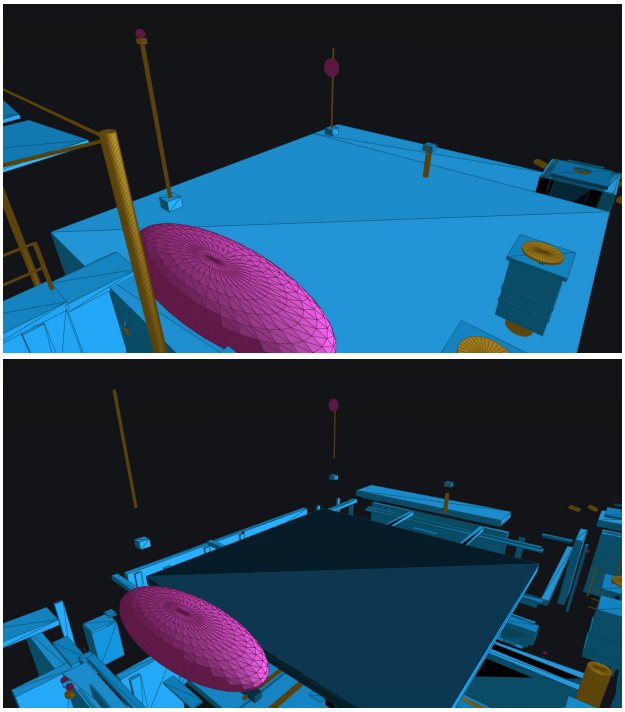
Figure 19.2: Soldier (rigged; all eight share it). Notice the joints in the exploded view: limbs are separate group nodes, so the gait only writes joint angles.



Part	Primitive	n	*S* (cm)	RGB	k_s / n
IR sensor	Sphere	5	2.5×2.5×2.5		0.50 / 64
Hub R	Sphere	3	7.0×7.0×3.0		0.70 / 64
Hub L	Sphere	3	7.0×7.0×3.0		0.70 / 64
Wheel R	Cylinder	3	16.0×16.0×7.0		0.05 / 8
Wheel L	Cylinder	3	16.0×16.0×7.0		0.05 / 8
Whip antenna	Cylinder	1	1.2×1.2×30.0		0.70 / 64
Scanner window	Cube	1	5.0×3.0×2.0		0.90 / 96
Scanner head	Sphere	1	7.0×7.0×7.0		0.90 / 96
Sensor mast	Cylinder	1	2.5×2.5×16.0		0.70 / 64
5 more part kinds		5			

26 parts: Cube 6, Cylinder 8, Sphere 12

Figure 19.3: Target rover (intruder). Notice that 8 of its nodes are animated (wheels, sensor mast); the scene has 6 rovers of this design, 26 parts each.



Part	Primitive	n	S (cm)	RGB	k_s / n
Camo net panel	Cube	27	116.4×3.5×106.4		0.08 / 2
Guard rail	Cylinder	12	4.0×4.0×230.0		0.24 / 9
Ceiling beam	Cube	11	340.0×20.0×24.0		0.09 / 2
Ladder rung	Cylinder	10	3.0×3.0×50.0		0.51 / 28
Fixture housing	Cube	10	130.0×5.0×30.0		0.51 / 28
Sandbag	Cylinder	9	24.0×18.0×50.0		0.06 / 2
Rack LED row	Cube	9	3.0×6.0×60.0		0.25 / 49
Guard rail post	Cylinder	8	6.0×6.0×100.0		0.24 / 9
Counterfort rib	Cube	8	18.0×39.3×30.0		0.09 / 2
162 more part kinds		323			

427 parts: Cube 309, Cylinder 99, Sphere 19

Figure 19.4: Command bunker (underground). Notice that the bunker is the fourth-largest object (427 parts, after Base, Camp camouflage, Environment): its rooms below ground are modelled and walkable.



Part	Primitive	n	S (cm)	RGB	k_s / n
Spoke	Cube	18	68.4×7.0×53.0		0.47 / 25
Hub	Cylinder	18	38.0×38.0×51.0		0.40 / 16
Tyre	Cylinder	18	76.0×76.0×45.0		0.10 / 3
Smoke grenade tube	Cylinder	8	9.0×9.0×22.0		0.47 / 25
Side skirt	Cube	8	7.0×50.0×120.0		0.14 / 7
Cupola vision block	Cube	6	16.0×10.0×10.0		0.33 / 194
Return roller R	Cylinder	3	18.0×18.0×20.0		0.10 / 3
Return roller L	Cylinder	3	18.0×18.0×20.0		0.10 / 3
Driver periscope	Cube	3	18.0×8.0×6.0		0.33 / 194
167 more part kinds		170			

† scale 9e-05 at export: invisible until fired

255 parts: Cube 170, Cylinder 80, Sphere 5

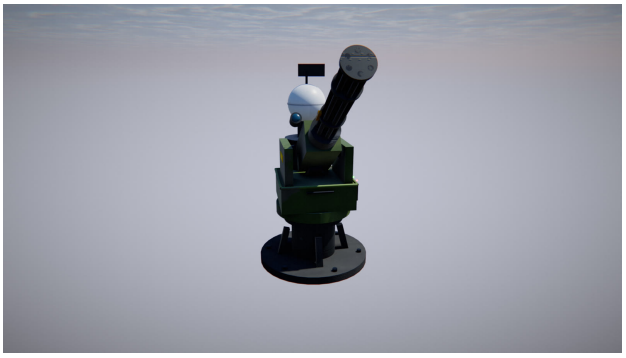
Figure 19.5: Main battle tank. Notice that 118 of its nodes are animated (road wheels, track, turret, gun): driving only changes local matrices.



Part	Primitive	n	S (cm)	RGB	k_s / n
Pitch link	Cylinder	5	3.0×3.0×48.7		0.80 / 96
Blade erosion strip	Cube	5	3.5×3.6×345.0		0.53 / 49
Blade tip cap	Cube	5	23.0×3.5×30.0		0.35 / 32
Blade grip	Cube	5	12.0×8.0×35.0		0.80 / 96
VHF antenna	Cube	1	2.0×20.0×20.0		0.50 / 48
Belly antenna	Cube	1	2.0×18.0×25.0		0.50 / 48
Drive shaft cover	Cylinder	1	7.0×7.0×330.0		0.40 / 48
FLIR window	Sphere	1	13.0×13.0×5.0		1.00 / 160
FLIR ball	Sphere	1	24.0×24.0×24.0		0.50 / 48
66 more part kinds		66			

91 parts: Cone 1, Cube 40, Cylinder 28, Frustum 1, Sphere 21

Figure 19.6: Scout helicopter. Notice the one frustum (a cylinder with two radii): it is the searchlight's visible cone.



Part	Primitive	n	S (cm)	RGB	k_s / n
Shell casing	Cylinder	20	hidden†		0.51 / 28
Anchor bolt	Cylinder	6	9.0×9.0×6.0		0.49 / 25
Gatling barrel	Cylinder	6	7.0×7.0×200.0		0.49 / 25
Mount body edge	Cylinder	4	5.0×5.0×140.0		0.32 / 21
Pedestal brace	Cube	4	12.0×60.0×12.0		0.43 / 16
Housing vent	Cube	3	42.0×2.5×4.5		0.49 / 25
Grab handle	Cylinder	1	4.0×4.0×50.0		0.49 / 25
Danger stencil L	Cube	1	1.0×12.0×45.0		0.13 / 5
Danger stencil R	Cube	1	1.0×12.0×45.0		0.13 / 5
32 more part kinds		32			

† scale 1e-04 at export: invisible until fired

78 parts: Cube 19, Cylinder 54, Sphere 5

Figure 19.7: AA Gatling gun.



Part	Primitive	n	S (cm)	RGB	k_s / n
Slew gear tooth	Cube	12	10.0×12.0×10.0		0.41 / 21
Tyre tread	Cylinder	8	103.0×103.0×8.0		0.09 / 2
Canister stencil	Cube	4	0.6×12.0×90.0		0.13 / 5
Canister rib	Cube	4	47.0×47.0×7.0		0.41 / 21
Rear hatch	Cube	4	46.0×46.0×6.0		0.41 / 21
Canister cap	Cube	4	40.0×40.0×4.0		0.26 / 9
Muzzle collar	Cube	4	50.0×50.0×6.0		0.41 / 21
Hazard band	Cube	4	46.0×46.0×18.0		0.17 / 12
Canister tube	Cube	4	44.0×44.0×340.0		0.16 / 4
38 more part kinds		64			

112 parts: Cone 1, Cube 74, Cylinder 32, Sphere 5

Figure 19.8: SAM launcher.



Part	Primitive	n	S (cm)	RGB	k_s / n
Tube cap	Cylinder	12	31.0×31.0×4.0		0.26 / 9
Muzzle ring	Cylinder	12	34.0×34.0×8.0		0.41 / 21
Launch tube	Cylinder	12	30.0×30.0×360.0		0.16 / 4
Tyre tread	Cylinder	12	118.4×118.4×9.0		0.09 / 2
Round flame	Sphere	6	28.6×28.6×132.0		0.16 / 9
Round fins B	Cube	6	2.0×66.0×26.4		0.41 / 21
Round fins A	Cube	6	66.0×2.0×26.4		0.41 / 21
Round nose	Cone	6	22.0×22.0×61.6		0.26 / 16
Round body	Cylinder	6	22.0×22.0×220.0		0.29 / 16
32 more part kinds		58			

136 parts: Cone 6, Cube 45, Cylinder 76, Sphere 9

Figure 19.9: MLRS rocket truck.

Air-defence radar

Part	Primitive	n	S (cm)	RGB	k_s / n
Antenna rib	Cube	14	66.0×8.0×8.0		0.70 / 64
Mast brace	Cylinder	12	5.0×5.0×169.4		0.70 / 64
Mast ring bar	Cylinder	12	7.0×7.0×90.0		0.70 / 64
IFF dipole	Cube	6	4.0×20.0×4.0		0.70 / 64
Mast leg	Cylinder	4	14.0×14.0×408.0		0.70 / 64
Shelter roof edge	Cylinder	2	5.0×5.0×300.0		0.37 / 21
Bearing seal	Cylinder	1	104.0×104.0×4.0		0.50 / 48
Door placard	Cube	1	1.2×26.0×36.0		0.29 / 16
Shelter status LED	Sphere	1	5.0×5.0×5.0		0.29 / 111
55 more part kinds		55			

108 parts: Cone 1, Cube 62, Cylinder 42, Sphere 3

Barracks hall

Part	Primitive	n	S (cm)	RGB	k_s / n
Opening frame	Cube	35	26.0×5.0×82.0		0.15 / 8
Ladder rung	Cylinder	12	2.5×2.5×34.0		0.40 / 16
Bunk rail	Cube	12	200.0×6.0×4.0		0.40 / 16
Bunk post	Cube	12	5.0×192.0×5.0		0.40 / 16
Gable	Cube	10	22.0×20.0×64.0		0.11 / 3
North wall	Cube	10	60.0×282.0×22.0		0.11 / 3
Dumbbell	Cylinder	8	17.0×17.0×30.0		0.49 / 21
West wall	Cube	7	22.0×282.0×258.0		0.11 / 3
East wall	Cube	7	22.0×282.0×98.0		0.11 / 3
68 more part kinds		164			

277 parts: Cube 225, Cylinder 50, Sphere 2

Ammunition bunker

Part	Primitive	n	S (cm)	RGB	k_s / n
Wing sandbag	Cylinder	8	28.0×17.0×50.0		0.06 / 2
Turf clump	Sphere	5	100.0×40.0×90.0		0.13 / 5
Crate	Cube	3	55.0×45.0×55.0		0.12 / 4
Fuel drum	Cylinder	2	40.0×40.0×60.0		0.34 / 5
Vent cap	Cylinder	2	34.0×34.0×6.0		0.41 / 21
Vent pipe	Cylinder	2	20.0×20.0×100.0		0.41 / 21
Wing wall slope	Cube	2	40.0×140.0×240.0		0.06 / 2
Wing wall core	Cube	2	60.0×130.0×240.0		0.06 / 2
Headwall block	Cube	2	130.0×190.0×50.0		0.09 / 2
14 more part kinds		15			

43 parts: Cube 19, Cylinder 14, Sphere 10

Flag

Part	Primitive	n	S (cm)	RGB	k_s / n
Flag cloth	Cube	36	20.0×50.0×2.0		0.07 / 2
Cleat	Cube	2	8.0×5.0×5.0		0.50 / 14
Halyard	Cylinder	2	1.5×1.5×740.0		0.37 / 4
Flag spot stand	Cube	1	20.0×12.0×20.0		0.09 / 2
Flag spot lens	Sphere	1	30.0×30.0×5.0		0.31 / 147
Flag spot housing	Cylinder	1	34.0×34.0×30.0		0.41 / 21
Flagpole finial	Sphere	1	24.0×24.0×24.0		0.51 / 49
Flagpole upper	Cylinder	1	11.0×11.0×450.0		0.50 / 14
Flagpole lower	Cylinder	1	16.0×16.0×450.0		0.50 / 14
1 more part kinds		1			

47 parts: Cube 40, Cylinder 5, Sphere 2

Windsock

Part	Primitive	n	S (cm)	RGB	k_s / n
Mouth hoop	Cylinder	6	2.5×2.5×36.0		0.50 / 14
Sock cone	Cylinder	5	28.0×28.0×55.0		0.07 / 2
Windsock pole	Cylinder	2	10.0×10.0×310.0		0.50 / 14
Hoop spoke	Cylinder	1	2.5×2.5×31.0		0.50 / 14
Windsock base	Cube	1	50.0×8.0×50.0		0.09 / 2

15 parts: Cube 1, Cylinder 14

Boom barrier (gate)

Part	Primitive	n	S (cm)	RGB	k_s / n
Boom stripe (white)	Cube	5	50.0×12.0×10.0		0.17 / 12
Boom stripe (red)	Cube	5	50.0×12.0×10.0		0.19 / 16
Warning lamp	Sphere	1	22.0×22.0×22.0		0.31 / 147
Boom tip lamp	Sphere	1	12.0×12.0×12.0		0.31 / 147
Pivot bolt	Cylinder	1	12.0×12.0×60.0		0.41 / 21
Counterweight	Cube	1	30.0×34.0×26.0		0.09 / 2
Counterweight arm	Cube	1	90.0×8.0×8.0		0.41 / 21
Boom cradle	Cube	1	36.0×8.0×20.0		0.41 / 21
Boom receiver	Cube	1	30.0×95.0×30.0		0.17 / 12
3 more part kinds		3			

20 parts: Cube 17, Cylinder 1, Sphere 2

Figure 19.10: Parts tables of the radar, barracks hall, ammunition bunker, flag, windsock and gate barrier. Their assembled views appear in Figures 2.1 and 15.1; the whole camp exploded into primitives is Figure 6.1b.

UAV-1 parts (first 26 rows of block 3)

Part	Primitive	Parent (group)	Sx	Sy	Sz	Rx (deg)	Ry (deg)	Rz (deg)	Tx (m)	Ty (m)	Tz (m)
Fuselage	Cube	Autonomous UAV (UAV-00)	0.160	0.070	0.260	0	0	0	0	0	0
Canopy	Sphere	Autonomous UAV (UAV-00)	0.150	0.080	0.220	0	0	0	0	0.040	0.010
Battery	Cube	Autonomous UAV (UAV-00)	0.100	0.040	0.160	0	0	0	0	-0.055	-0.020
GPS mast	Cylinder	Autonomous UAV (UAV-00)	0.008	0.008	0.050	90	0	0	0	0.070	-0.070
GPS puck	Cylinder	Autonomous UAV (UAV-00)	0.050	0.050	0.012	90	0	0	0	0.098	-0.070
Arm Front-left	Cube	Autonomous UAV (UAV-00)	0.025	0.018	0.220	0	45	0	0.106	0	0.106
Motor	Cylinder	Motor mount Front-left (UAV-07)	0.045	0.045	0.035	90	0	0	0	0.020	0
Prop hub	Sphere	Rotor (UAV-09)	0.025	0.025	0.025	0	0	0	0	0	0
Prop blade	Cube	Rotor (UAV-09)	0.200	0.004	0.018	0	0	0	0	0	0
Rotor blur disc	Cylinder	Motor mount Front-left (UAV-07)	0.205	0.205	0.002	90	0	0	0	0.045	0
Nav LED	Sphere	Motor mount Front-left (UAV-07)	0.018	0.018	0.018	0	0	0	0	-0.012	0
Motor cooling ring	Cylinder	Motor mount Front-left (UAV-07)	0.050	0.050	0.010	90	0	0	0	0.004	0
Arm Front-right	Cube	Autonomous UAV (UAV-00)	0.025	0.018	0.220	0	-45	0	-0.106	0	0.106
Motor	Cylinder	Motor mount Front-right (UAV-15)	0.045	0.045	0.035	90	0	0	0	0.020	0
Prop hub	Sphere	Rotor (UAV-17)	0.025	0.025	0.025	0	0	0	0	0	0
Prop blade	Cube	Rotor (UAV-17)	0.200	0.004	0.018	0	0	0	0	0	0
Rotor blur disc	Cylinder	Motor mount Front-right (UAV-15)	0.205	0.205	0.002	90	0	0	0	0.045	0
Nav LED	Sphere	Motor mount Front-right (UAV-15)	0.018	0.018	0.018	0	0	0	0	-0.012	0
Motor cooling ring	Cylinder	Motor mount Front-right (UAV-15)	0.050	0.050	0.010	90	0	0	0	0.004	0
Arm Rear-left	Cube	Autonomous UAV (UAV-00)	0.025	0.018	0.220	0	135	0	0.106	0	-0.106
Motor	Cylinder	Motor mount Rear-left (UAV-23)	0.045	0.045	0.035	90	0	0	0	0.020	0
Prop hub	Sphere	Rotor (UAV-25)	0.025	0.025	0.025	0	0	0	0	0	0
Prop blade	Cube	Rotor (UAV-25)	0.200	0.004	0.018	0	0	0	0	0	0
Rotor blur disc	Cylinder	Motor mount Rear-left (UAV-23)	0.205	0.205	0.002	90	0	0	0	0.045	0
Motor cooling ring	Cylinder	Motor mount Rear-left (UAV-23)	0.050	0.050	0.010	90	0	0	0	0.004	0
Status LED	Sphere	Motor mount Rear-left (UAV-23)	0.016	0.016	0.016	0	0	0	0	-0.012	0

input = value copied from the code, white = formula

Figure 19.11: Workbook sheet `Decomposition`: every UAV and soldier part with primitive, dimensions, *T*/*R*/*S*, parent, colour and material, and `COUNTIFS` totals per primitive that match Table 19.1. The body shows the first 26 UAV parts with their transforms. (full sheet: Appendix A.2)

Where in the project. `src/objects/*.cpp (one builder per object), src/app/DocExport.cpp (export), tools/v10/gen_decomp.py (tables)`

Critique. Static scenery (base walls, camouflage nets, environment props) is listed only in Table 19.1; its 1,253 + 652 + 658 parts are repetitive kit pieces. Exploded views exist for the UAV, rover, bunker, soldier and the whole scene; the other vehicles are shown assembled.

20 Colour interpolation, with critique

Builds on: rasterisation (14), shading (16), illumination (15).

Colour is interpolated in four places: across a triangle (barycentric), between shading models (Gouraud vs Phong), between colour spaces (sRGB vs linear), and over time (sky and fog presets over the day). Each has a naive version

that fails visibly.

$$c_{\text{lin}} = \begin{cases} c_s/12.92 & c_s \le 0.04045 \\ \left(\frac{c_s+0.055}{1.055}\right)^{2.4} & \text{else} \end{cases}, \qquad \text{mixLog}(a,b,t) = \exp\big((1-t)\ln a + t \ln b\big) \tag{20.1}$$

A. The worked triangle

vertex	x (px)	y (px)	clip w	tex u	N.x	N.y	N.z	I vertex
V0	100	100	1	0	-0.6012	-0.4008	0.6913	0.4148
V1	400	150	4	1	0.6007	-0.3004	0.7409	0.4445
V2	220	380	10	0.5000	0	0.6000	0.8000	0.4801
light	L = V = (0, 0, 1), H = (0, 0, 1)							
kd, ks, n	0.6000	0.6000	40					
2 x area (edge fn)	78,000							

B. Barycentric weights

sample	px	py	w0	w1	w2	sum	inside
centroid	240	210	0.3333	0.3333	0.3333	1	yes
near V1	360	160	0.0949	0.8410	0.0641	1	yes
edge V0-V2 mid	160	240	0.5000	0	0.5000	1	yes
centre-left	200	180	0.5205	0.2359	0.2436	1	yes
outside	420	330	-0.4744	0.7949	0.6795	1	no

D. Texture u: affine vs perspective-correct

sample	u affine	u persp.	error
centroid	0.5000	0.2222	0.2778
near V1	0.8731	0.6852	0.1879
edge V0-V2 mid	0.2500	0.0455	0.2045
centre-left	0.3577	0.1178	0.2399

C. Gouraud vs Phong at the same points

sample	I Gouraud	N.x	N.y	N.z	|N|	I Phong	Phong - Gouraud
centroid	0.4465	-1.667×10^{-4}	-0.0337	0.7441	0.7448	1.1752	0.7288
near V1	0.4440	0.4482	-0.2522	0.7400	0.9011	0.4929	0.0489
edge V0-V2 mid	0.4474	-0.3006	0.0996	0.7457	0.8101	0.5740	0.1266
centre-left	0.4377	-0.1712	-0.1333	0.7295	0.7611	0.6851	0.2474

E. sRGB vs linear blending of colours A and B

t	naive R	naive G	naive B	linear R	linear G	linear B	Y naive	Y linear
0	255	64	0	255	64	0	0.2493	0.2493
0.2500	191.2500	80	63.7500	224.6104	85.7120	136.9602	0.1721	0.2436
0.5000	127.5000	96	127.5000	187.5160	102.2531	187.5160	0.1446	0.2379
0.7500	63.7500	112	191.2500	136.9602	116.0203	224.6104	0.1644	0.2323
1	0	128	255	0	128	255	0.2266	0.2266

F. Sky zenith colour over the day

hour	e formula (deg)	e export (deg)	diff	daylight w	phase (no hysteresis)	zenith R	zenith G	zenith B	Y zenith	phase export
0	-71.8000	-71.8000	0	0	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
4	-29.3751	-29.3751	-6.790×10^{-6}	0	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
5.5000	-8.6859	-8.6859	1.244×10^{-5}	0	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
6	-1.7809	-1.7810	5.904×10^{-6}	0.4167	DAWN	0.0617	0.0752	0.1058	0.0746	DAWN
6.5000	5.0981	5.0981	5.641×10^{-6}	0.9881	DAY	0.1885	0.2126	0.2329	0.2089	DAY
7	11.9319	11.9319	-5.069×10^{-6}	1	DAY	0.2206	0.2653	0.3436	0.2615	DAY
9	38.2196	38.2196	3.337×10^{-5}	1	DAY	0.2852	0.4034	0.6962	0.3994	DAY
12	62.6000	62.6000	0	1	DAY	0.3089	0.4463	0.7643	0.4400	DAY
15	38.2196	38.2196	3.337×10^{-5}	1	DAY	0.2852	0.4034	0.6962	0.3994	DAY
17	11.9319	11.9319	-5.069×10^{-6}	1	DAY	0.2206	0.2653	0.3436	0.2615	DAY
17.5000	5.0981	5.0981	5.641×10^{-6}	0.9881	DAY	0.1885	0.2126	0.2329	0.2089	DAY
18	-1.7809	-1.7810	5.904×10^{-6}	0.4167	DUSK	0.0617	0.0752	0.1058	0.0746	DUSK
18.5000	-8.6859	-8.6859	1.244×10^{-5}	0	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
20	-29.3751	-29.3751	-6.790×10^{-6}	0	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT

input = value copied from the code, white = formula

Figure 20.1: Workbook sheet `Color_Interp`: barycentric weights at sample points of one triangle, Gouraud vs Phong values at the same points, affine vs perspective-correct texture coordinates with w, sRGB vs linear blends of two colours, and the sky colours tabulated over the day. (full sheet: Appendix A.17)

Interpolation	Naive version and its failure	What the program does	Evidence
Across a triangle	affine in screen space: textures bend	perspective-correct (a/w, $1/w$)	Fig. 14.1b
Lighting	Gouraud: highlight between vertices lost, Mach bands	Phong per fragment	Figs. 16.1, 16.2
Colour space	blending sRGB values: mid-tones too dark, e.g. 50 % red/green gives a muddy (0.5, 0.5, 0)	material colours converted to linear (`toLinear`), lit and blended in linear HDR, encoded once at the end	Fig. 20.1
Sky over the day	linear mix of night and day presets: the night tint is swamped by the day value until very late	presets blended in log space (Eq. 20.1): equal steps are equal ratios, which the eye sees as even	Figs. 2.1, 21.1

Table 20.1: Colour interpolation: naive method, failure, the program's method.

Where in the project. `toLinear() and the output encoding in src/rendering/ShaderSources.cpp; mixLog() in src/scene/DayCycle.cpp; Frame Explorer interpolation stage`

Critique. Log-space blending (`mixLog`, base 2 in the code) is undefined for a zero channel: every preset must stay strictly positive, and a preset with a near-zero channel makes the blend linger near black for most of the transition. Linear-space blending is right for light, but the result is still stored in 8 bits per channel; the only defence against banding in dark gradients is the film grain added in the final pass (`ShadersPostFx.cpp`).

21 Day and night system

Builds on: illumination (15), colour interpolation (20).

The sun follows a solar-position model for the camp's latitude (Khulna, $\varphi = 22.8°$, early October, declination $\delta = -4.6°$). It rises in the east, crosses the southern sky and sets in the west. Every lighting parameter is a function of its elevation e.

$$\sin e = \sin\varphi \sin\delta + \cos\varphi \cos\delta \cos H, \quad H = 15°(\text{hour} - 12) \tag{21.1}$$
$$T_c(e) = \exp\big(-\tau_c\,(m(e) - 1)\big), \quad \tau = (0.06,\ 0.11,\ 0.21)$$

with the Kasten–Young air mass [4] $m(e) = 1/\big(\sin e + 0.50572\,(e + 6.07995°)^{-1.6364}\big)$. The sun's key light is $3.2\,s(e)\,T(e)$ with s a smoothstep from $-1°$ to $10°$; blue has the largest optical depth, so it is removed first near the horizon and the light turns orange.

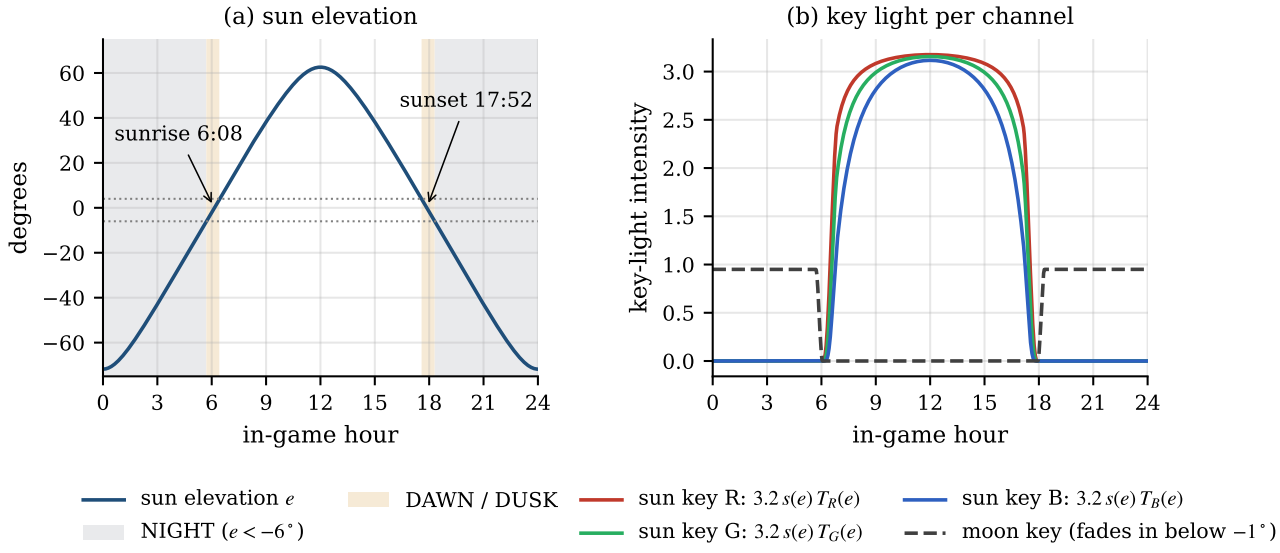


Figure 21.1: The code's solar model over 24 hours (plotted from Eq. (21.1)). Notice in (a) the asymmetric twilight bands where the phases switch (night below $-6°$, day above $4°$), and in (b) that red outlasts blue at both ends of the day: that is the sunset colour of Figure 2.1b, produced by the formula, not painted.



(a) Dawn: the intruders withdraw.



(b) Noon.



(c) Dusk: shift change.

Figure 21.2: Phases of the day. Notice how the fog colour follows the sky's horizon colour in each phase: both come from the same analytic sky (Preetham model [5]) evaluated for the current sun direction.

Derived quantity	Rule in `DayCycle::applyTo`
Key light	sun above -1° ($3.2\,s(e)\,T(e)$, direction quantised to 1° for the shadow cache); else the moon, fading in as $1 - s_{[-6,-1]}(e)$
Sky, ambient, fog	by day: a Preetham sky for the sun direction; its spherical-harmonic irradiance is the ambient light; fog takes the horizon colour; night presets blended in log space
Exposure	key value moves from the night to the day value as $s_{[-8,10]}(e)$
Practical lamps, searchlights	lamps on as $1 - s_{[-4,2]}(e)$; UAV searchlights only below -2°
Rules	intruder waves only at night (unless day ops); soldiers' day shift 06:24–17:36

Table 21.1: Everything that follows the sun's elevation.

Transitions. The clock runs automatically (one day = 290.6 s) or is scrubbed by hand ([,] [.]), jumps to the next phase ([/]) or to a preset in the pause menu; lighting follows continuously in every case.

Where in the project. `src/scene/DayCycle.h,cpp, src/math/SkyModel.cpp (Preetham sky, SH ambient)`

Critique. The moon is not an orbiting body: it is a fixed-direction night key light (intensity 0.95), so it does not rise as the sun sets as the brief asked; the sun arc is fully physical. The brief's [N]/[D] keys were not added because both are taken (Section 5).

22 Curved objects and tessellation

Builds on: primitives (6), shading (16).

Curved surfaces are drawn as polygons. The silhouette error of a sphere with s slices is the sagitta of one chord; the triangle count of `createSphere(s, k)` is $2s(k-1)$. Halving the error costs about twice the triangles.

$$\varepsilon = r\left(1 - \cos\frac{\pi}{s}\right) \approx \frac{r\pi^2}{2s^2}, \qquad N_\triangle = 2s(k-1) \tag{22.1}$$

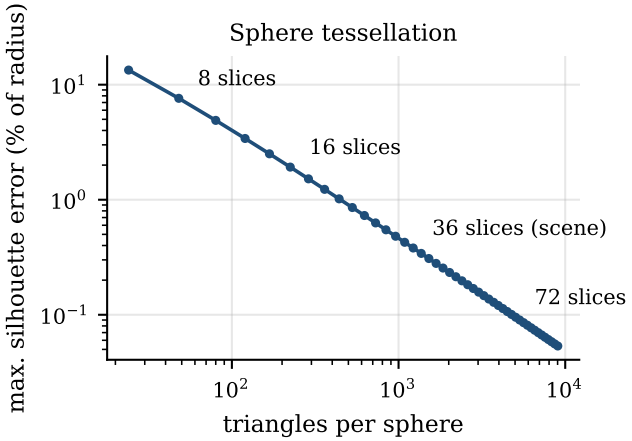




Figure 22.1: Left: silhouette error against triangle count (Eq. (22.1)); the scene uses 36 slices (0.38 % of the radius, 1224 triangles). Right: a low-polygon sphere under flat, Gouraud and Phong shading. Notice that smooth shading hides the facets inside the outline but cannot hide the polygonal silhouette: only more triangles fix the edge.

Curved objects in the scene. Tank road wheels and tyres, barrels, masts, helmets and heads (spheres and cylinders); the bunker's earth berm (a flattened sphere); rotor blur discs (flat cylinders); the soldiers' walking paths and the patrol routes are curves too (centripetal Catmull–Rom splines, Section 27). The CC0 trees (v9) carry their own detail levels: three simplified meshes (meshoptimizer) and an octahedral impostor far away.

Where in the project. `src/primitives/Sphere.cpp, Cylinder.cpp; LOD chain in src/rendering/Foliage.cpp`

Critique. There is no torus or helix generator, and the primitives have no level of detail: a distant sphere costs the same 1224 triangles as a close one. Choosing s from projected size (e.g. $s \propto \sqrt{\text{pixels}}$) would cut the sphere triangles by a large factor in the overview shots.

23 Dynamic and controllable objects

Builds on: hierarchy (8), viewing (9).

26 units can be controlled (soldiers, two UAVs, tanks, helicopter, AA gun, radar, launchers, blast doors, gate). Motion is integrated at a fixed 60 Hz tick; between ticks the drawn pose is interpolated, so motion is smooth at any frame rate. Almost every follow behaviour (character speed, cameras, doors) uses one tool: a critically damped spring, which reaches its target as fast as possible without overshoot and has an exact step for any Δt.

$$\ddot{x} = -\omega^2(x - x^*) - 2\omega\dot{x} \;\Rightarrow\; x(t) = x^* + \big((x_0 - x^*) + (\dot{x}_0 + \omega(x_0 - x^*))\,t\big)e^{-\omega t} \tag{23.1}$$

Object	Motion model (measured or set in the code)
Soldier (player)	speed follows the input on a spring, $\omega = 17\,\text{s}^{-1}$ accelerating, 30 braking; 90 % of walking speed in 0.23 s; coyote time 0.12 s; slope limit 40°
Soldier legs	gait phase advanced by distance / stride; two-link IK keeps stance feet on the ground (slip ≤ 0.22 cm)
Third-person camera	spring-arm follow, sphere cast against walls; shorter arm indoors
UAV	duty state machine (docked, take-off, patrol, return, landing); chase camera springs $\omega = 10$ (position), 5 (heading); bank 0.3° per °/s of turn
Hinged doors	hinge angle on a spring, $\omega = 4.2\,\text{s}^{-1}$ (hall): 95 % open in 1.13 s; the swinging leaf is a moving collision box
Eye adaptation	exposure time constant 0.35 s towards brighter, 1.1 s towards darker; +1.25 EV indoor bias

Table 23.1: Dynamic objects and their motion models (`docs/v9_notes/v9-notes.md`).



(a) Door closed, prompt. **(b)** Opening (spring). **(c)** Open.



(d) Just inside: eye not adapted. **(e)** Two seconds later: adapted. **(f)** Sprint: FOV widens by up to 6°.

Figure 23.1: Controllable dynamics in the program. Notice in (d)–(e) that the room is the same frame of geometry; only the exposure changed, following the slower bright-to-dark time constant.

Where in the project. `src/scene/CharacterMotor.cpp, Gait.cpp, CameraRig.cpp, UavView.cpp, Door.cpp, EyeAdapt.cpp, ai/SquadDuty.cpp; src/scene/TickInterp.cpp (60 Hz tick, interpolation)`

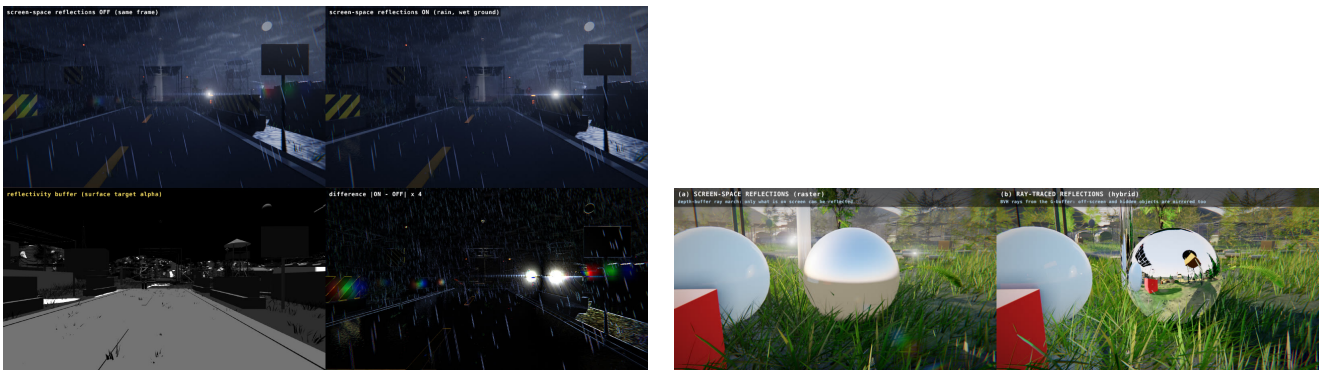
Critique. The fixed tick makes motion deterministic (a recorded session replays bit for bit) but adds up to one tick (16.7 ms) of input latency; render interpolation hides the stepping, not the latency.

24 Reflection: approximated and traced

Builds on: ray tracing (18), illumination (15).

The raster renderer approximates mirror reflection with screen-space reflections (SSR): for a reflective pixel it marches the reflected ray through the depth buffer and copies the colour where the ray passes behind a stored depth. It can only reflect what is on screen. The hybrid mode replaces that with real reflection rays against a BVH of the scene; Whitted, path and light tracing render the whole image with rays. The reflection weight is a Fresnel term.

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5 \quad \text{(Schlick)}, \qquad L_{\text{out}} = (1 - F)\, L_{\text{local}} + F\, L(\mathbf{r}) \tag{24.1}$$



(a) Reflections off / SSR on (wet ground, metal). **(b)** SSR vs ray-traced reflections.

Figure 24.1: Approximate versus traced reflection. Notice in (b) where SSR fails: objects outside the screen or hidden behind others are missing from the reflection, and the reflection fades towards the screen edge; the traced reflection has them.

Mode	ms / frame	ray pass ms	internal size
Raster	16.47	0.00	1280×720
Hybrid RT	28.73	16.33	1280×720
Whitted RT	31.08	25.53	960×540
Path Tracing	26.95	22.85	640×360
Light Tracing	47.86	40.78	640×360

Table 24.1: Cost of each render mode at 1280×720 on the RTX 4070 (`--capture raytrace_perf`). Hybrid traces at full size; Whitted, path and light tracing render at a reduced internal size and upscale.

Method	Visual result	Cost	When to use
SSR	correct for visible geometry; holes for off-screen or hidden objects	one ray march per pixel in the depth buffer	real-time play (EPIC)
Hybrid RT reflections	complete reflections	BVH traversal per reflective pixel	stills, demonstration
Whitted	exact mirrors, refraction, hard shadows; no indirect light	recursive rays per pixel	teaching, specular scenes
Path / light tracing	soft shadows, indirect light, caustics (light)	many paths per pixel, noise, denoising	reference images

Table 24.2: Differences between reflection methods.

Where in the project. `SSR in src/rendering/ShadersPost.cpp; src/raytrace/RayTracer.cpp (Whitted, path, light tracing), Bvh.cpp; src/rendering/RayTracerGL.cpp`

Critique. The hybrid mode traces only the reflection. The screen-space passes (SSAO/GTAO, fog, SSR) work on the primary image, so they cannot apply to what is seen in a traced reflection; Whitted, path and light tracing also run at reduced internal resolution (Table 24.1) and are for demonstration, not play.

25 Thermal sensor and image-processing pipeline

Builds on: rendering to texture, convolution, histograms; Section 15 (the heat render reuses the scene passes).

Simulated, not sensed. Each part has a heat value (engines and people hot, sun-heated walls warm, ground cool; the sun adds a solar term by day). Each UAV renders its view of these values into a 320×240 image; a sensor model turns the ideal image into what an uncooled microbolometer would output, and the computer-vision (CV) pipeline finds targets in it.

$$y = Q\big(g \odot (h_{\mathrm{PSF}} * x) + o + c + n\big), \quad h_{\mathrm{PSF}} = \tfrac{1}{16}\begin{bmatrix}1 & 2 & 1\end{bmatrix}^{\top}\begin{bmatrix}1 & 2 & 1\end{bmatrix}, \quad n \sim \mathcal{N}(0, 2^2) \tag{25.1}$$

(g, o: per-pixel gain and offset (fixed pattern, partly removed by the shutter NUC), c: column stripes, n: temporal noise in DN, Q: rounding to 8 bits.) Then (following [6]): Otsu threshold [7] (with a floor at heat 0.8), despeckle (a hot pixel needs 2 hot neighbours), connected components by two-pass union-find, blob features, a target filter, and a Kalman multi-target tracker that fuses both UAVs.

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}, \quad S_x = \begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}, \quad \|\nabla I\| = \sqrt{(S_x * I)^2 + (S_y * I)^2}, \quad s = c\, r^{\gamma} \tag{25.2}$$

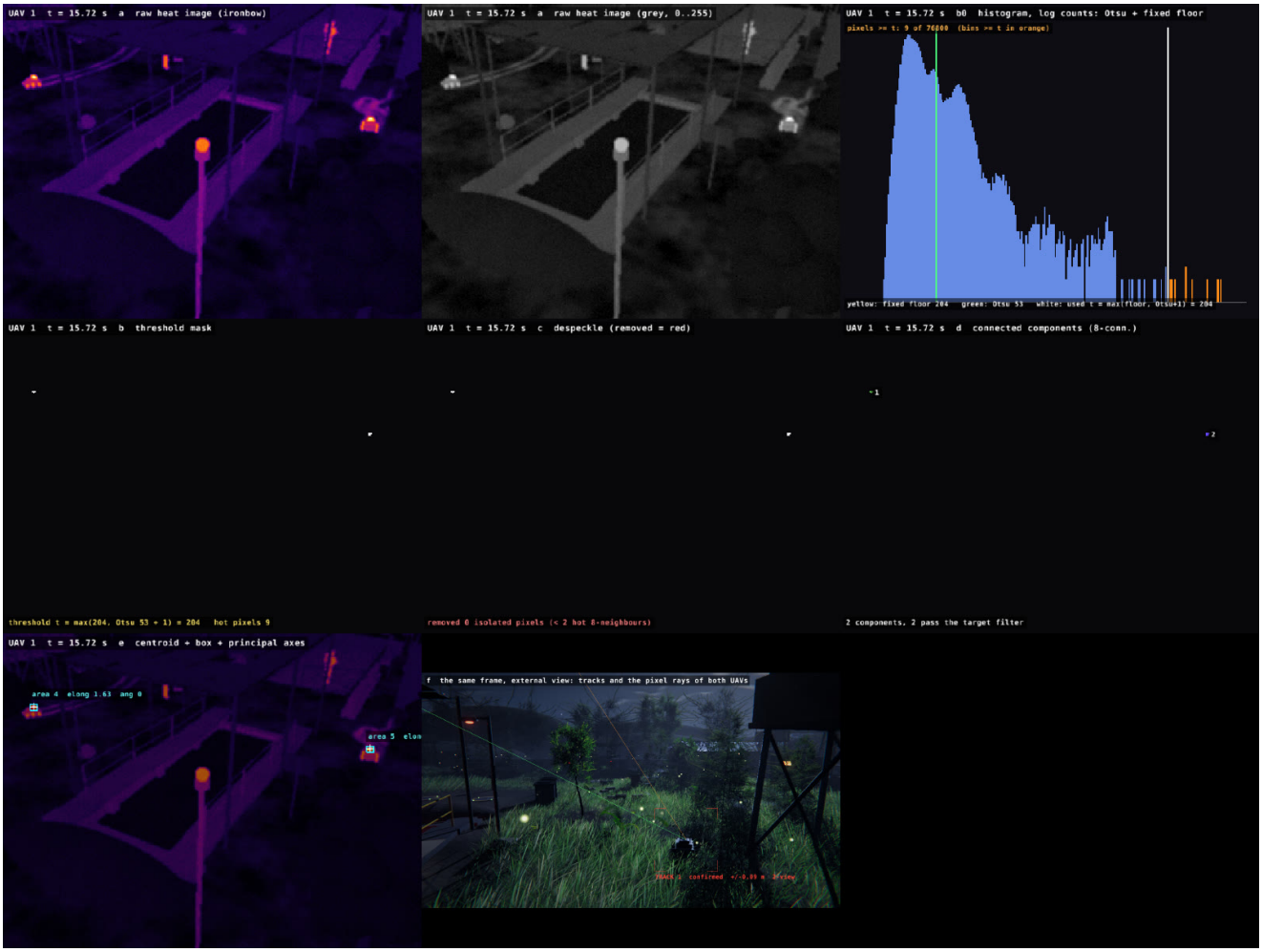
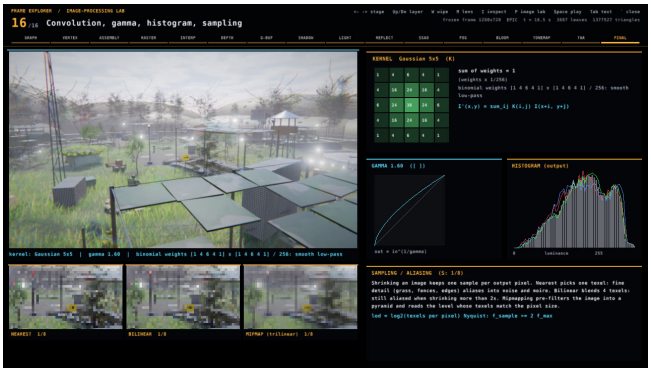
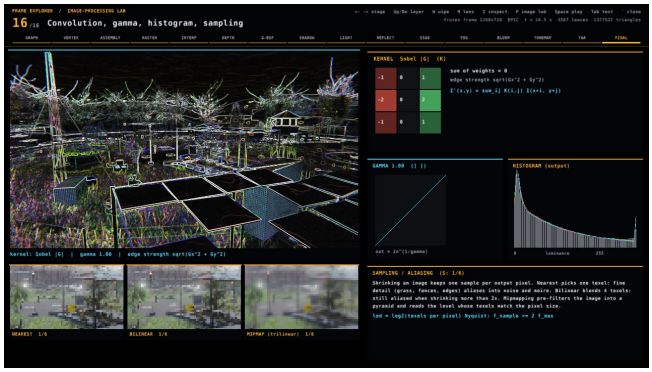


Figure 25.1: The CV pipeline on one real UAV frame (lab figure): raw sensor image, histogram and Otsu threshold, mask, despeckle, components, overlay. Notice that the sensor noise and the warm walls survive the blur but not the threshold-plus-despeckle; only compact hot blobs with a hot core become targets.



(a) Frame Explorer image lab: Gaussian and gamma.



(b) Sobel gradient magnitude.

Zoomed kernel panels of (a) and (b):

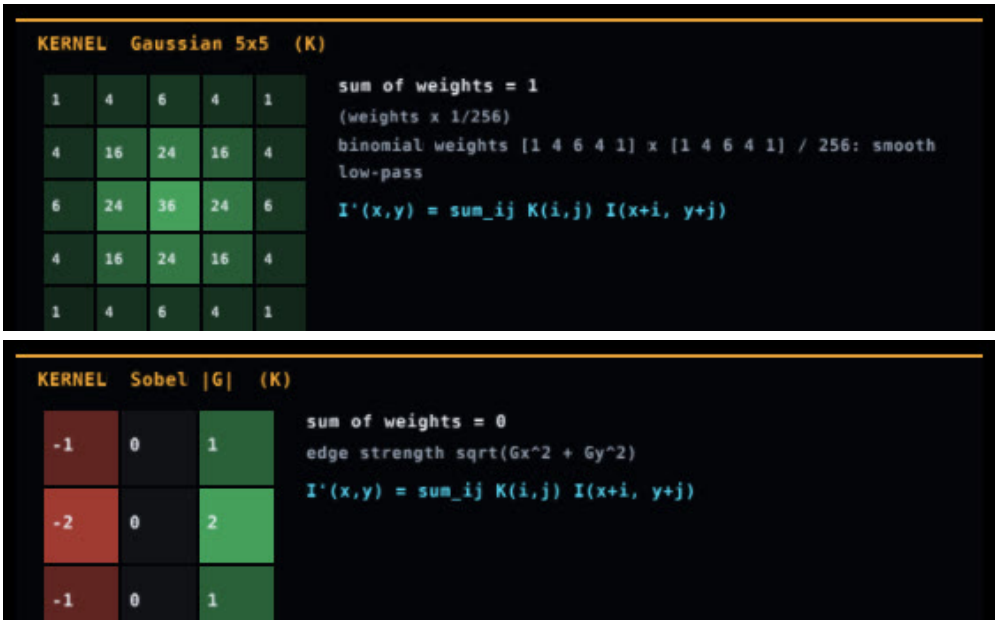
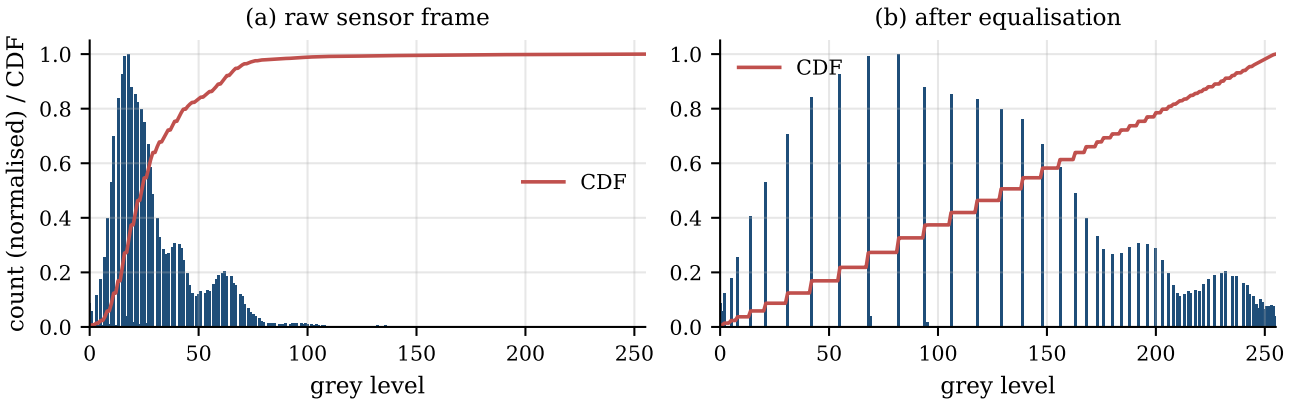
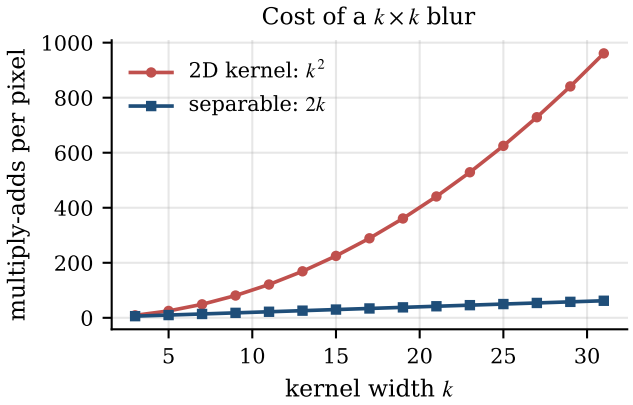


Figure 25.2: Image-processing stages on frames from the program (`) then P: K cycles the kernel, [ ] change γ).



(a) Histogram and CDF before and after equalisation.



(b) Separable convolution cost.

Figure 25.3: Histogram and convolution cost. Notice in (a) that the raw thermal frame uses a narrow band of grey levels (most pixels are cool ground); equalisation spreads them over the full range so the CDF becomes a straight line. Histogram equalisation is computed offline here (`tools/v10/plots.py`); it is not a switch in the program.

Gaussian ($\sigma = 1$, 5×5): raw and normalised

$$G_{\text{raw}} = \begin{bmatrix} 0.0183 & 0.0821 & 0.1353 & 0.0821 & 0.0183 \\ 0.0821 & 0.3679 & 0.6065 & 0.3679 & 0.0821 \\ 0.1353 & 0.6065 & 1 & 0.6065 & 0.1353 \\ 0.0821 & 0.3679 & 0.6065 & 0.3679 & 0.0821 \\ 0.0183 & 0.0821 & 0.1353 & 0.0821 & 0.0183 \end{bmatrix}, \quad G = G_{\text{raw}} / \sum = \begin{bmatrix} 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0219 & 0.0983 & 0.1621 & 0.0983 & 0.0219 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \end{bmatrix}$$

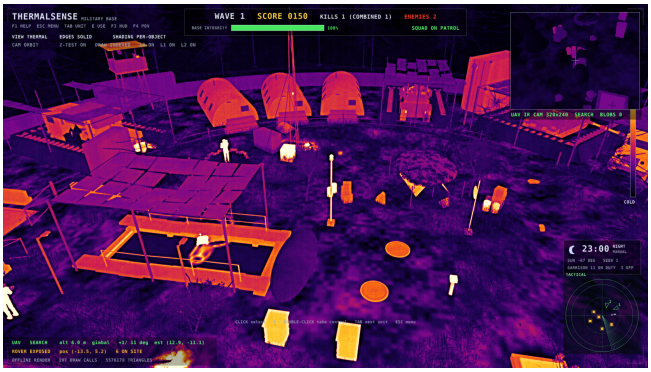
Thermal patch and the Sobel kernels

$$I = \begin{bmatrix} 0.2000 & 0.2200 & 0.2500 & 0.3000 & 0.3200 \\ 0.2100 & 0.2400 & 0.6000 & 0.6500 & 0.3400 \\ 0.2200 & 0.3000 & 0.8500 & 0.9000 & 0.4000 \\ 0.2000 & 0.2800 & 0.7000 & 0.7500 & 0.3600 \\ 0.1900 & 0.2100 & 0.2600 & 0.3100 & 0.3000 \end{bmatrix}, \quad S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

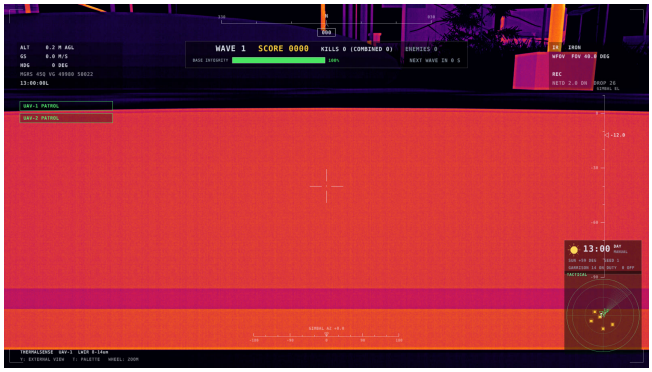
Valid 3×3 outputs, gradient magnitude and the thresholded edge map

$$G_x = \begin{bmatrix} 1.4600 & 1.5000 & -0.9000 \\ 2.1500 & 2.0800 & -1.5000 \\ 1.7000 & 1.6400 & -1.0900 \end{bmatrix}, \quad G_y = \begin{bmatrix} 0.7800 & 1.8800 & 1.8800 \\ 0.1700 & 0.3400 & 0.3200 \\ -0.8000 & -1.8600 & -1.8700 \end{bmatrix}, \quad \|\nabla I\| = \begin{bmatrix} 1.6553 & 2.4051 & 2.0843 \\ 2.1567 & 2.1076 & 1.5338 \\ 1.8788 & 2.4798 & 2.1645 \end{bmatrix}, \quad \text{edge} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Figure 25.4: Workbook sheet `Thermal_Kernels`: the normalised Gaussian for the chosen σ, Sobel kernels, and a 5×5 patch convolved step by step with one output pixel's `SUMPRODUCT` shown. (full sheet: Appendix A.19)



(a) External thermal view, ironbow palette.



(b) UAV gimbal by day: people classified, not engaged.

Figure 25.5: The thermal views. Notice in (b) the "PERS" brackets: blobs that are warm but have no hot engine core are classified as people and never targeted.

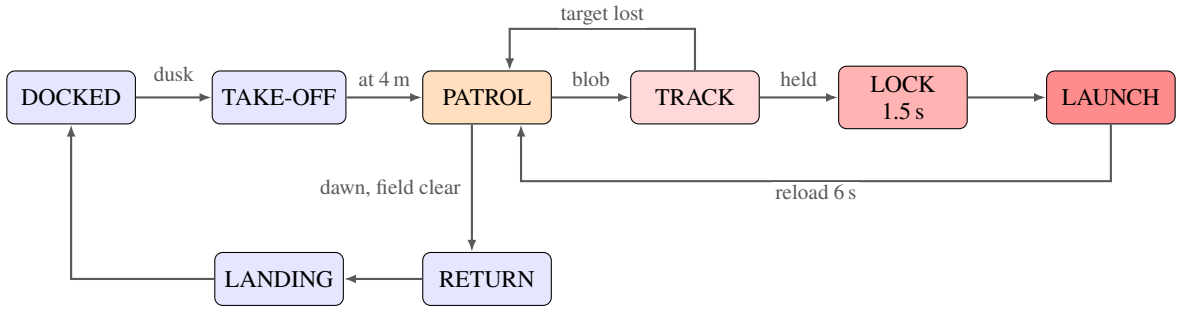
Where in the project. `src/vision/SensorModel.cpp (Eq. 25.1), CvPipeline.cpp, MultiTracker.cpp; src/math/ExplorerMath.cpp (kernels); src/rendering/ThermalPalette.h (4 palettes:` T `)`

Critique. The brief's live panel with all stages as tiles and sliders (key V) is only partly met: the stages exist (lab figures, Frame Explorer image lab with kernel and gamma controls), but not as one live tile strip with threshold and histogram-equalisation sliders. The CV runs on the CPU at 320×240 on a worker thread; the PSF blur is separable (two 3-tap passes instead of one 9-tap pass).

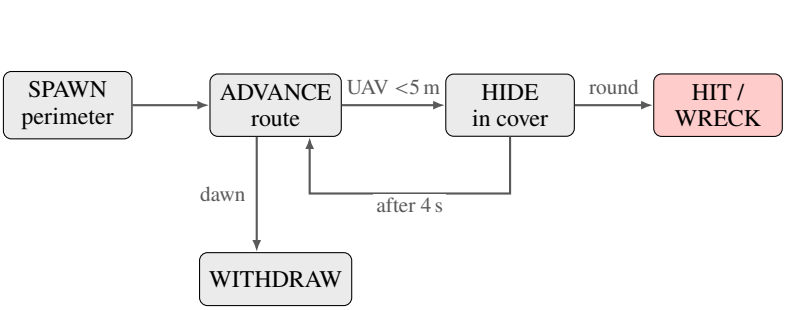
26 Game-mode AI

Builds on: ray casting (18), the thermal pipeline (25), paths (27).

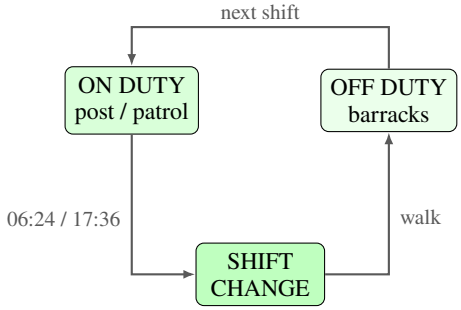
At night intruder rovers attack from the perimeter in waves; two UAVs patrol, detect them in their thermal images, hold a lock for 1.5 s and launch guided rounds. The player can join in by taking control of a soldier, a tank, the AA gun or a launcher. Hits are ray tests: each tick a projectile's path segment (previous to current position) is intersected with every target's bounding sphere and with the static solids' boxes, so a fast round cannot tunnel through a target between two ticks; an explosion then damages targets inside its blast radius.



(a) UAV: duty cycle and engagement (each UAV independently).

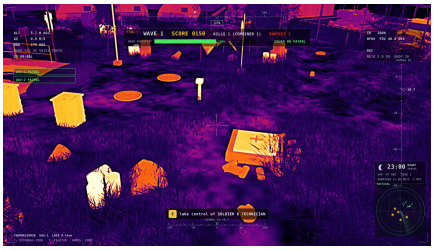


(b) Intruder rover.



(c) Soldier.

Figure 26.1: State diagrams. (a) a UAV (duty cycle `ai/SquadDuty.cpp`, engagement `ai/Squad.cpp`); each UAV runs it independently. (b) an intruder rover (`ai/HideController.cpp`); (c) a soldier's shift (`scene/Garrison.cpp`).



(a) Detect and track (gimbal).



(b) Ground fire.



(c) AA gun burst.



(d) Shift change at dusk.



(e) Dawn: survivors withdraw.

Figure 26.2: Game events captured by the program. Notice in (a) the HUD lock ring and the tracking ID from the CV tracker.

#	role	day	night
1	SENTRY	guard	guard
2	WATCH	guard	guard
3	GUNNER	walk	guard
4	MECHANIC	work	off
5	LOGISTICS	work	off
6	TECHNICIAN	work	work
7	PATROL	walk	guard
8	RUNNER	run	off

input = value copied from the code, white = formula

Figure 26.3: Workbook sheet `Duty_Cycle`: the 24-hour roster of the eight soldiers, day and night shift. (full sheet: Appendix A.20)

Where in the project. `src/ai/Squad.cpp`, `SquadDuty.cpp`, `DroneController.cpp`, `HideController.cpp`; `src/scene/Combat.cpp` (`segmentSphere`, `segmentBoxT`), `Mission.cpp` (`waves`, `score`), `Garrison.cpp` (`shifts`)

Critique. The brief asked for shifts every 6 in-game hours and an explicit “backup fire” call from the nearest soldiers. The code has two shifts (day 06:24–17:36 and night) with staggered changes, and AI soldiers do not fire: ground backup fire exists only when the player controls a soldier or a weapon. Both UAVs can independently detect, lock and fire, and they coordinate a combined strike when both hold the same track.

27 Line follower and MicroMouse logic

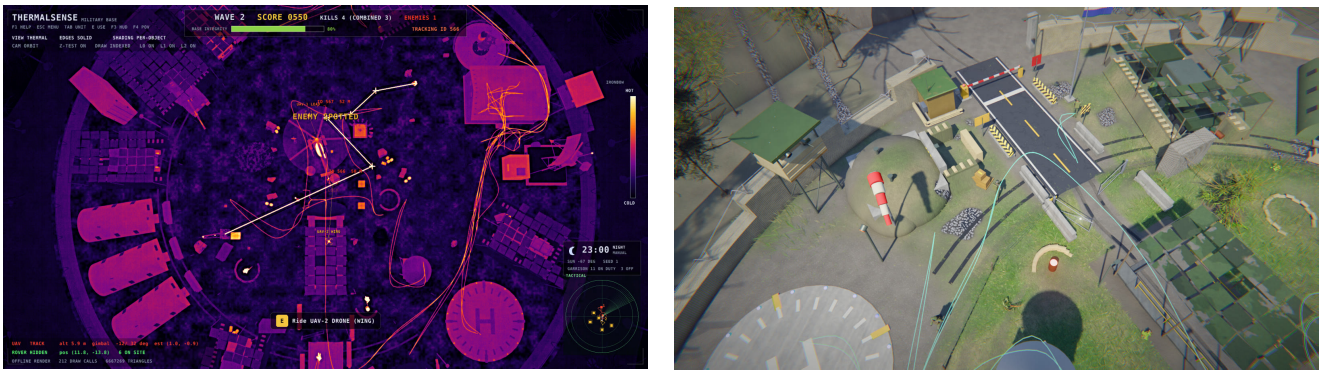
Builds on: grids, curves (22), ray casting for line of sight (18).

The intruder rovers are built like two classic robot-contest machines. **Planning (MicroMouse):** the 40 m×40 m area is divided into 80×80 cells of 0.5 m; every cell hidden from the UAVs by an occluder is a goal; a breadth-first flood fill spreads distances out from the goals, and the rover walks “downhill” to the neighbour with the smaller value, as a MicroMouse descends its maze towards the centre. **Driving (line-follower robot):** the planned cell path is smoothed into a virtual guide line; five reflectance sensors in front of the rover report whether they are over the line, and the weighted sensor error drives a PD controller that sets the left and right wheel speeds of a differential drive. The soldiers use A* on a 72×72 grid of 1 m cells with 0.40 m clearance, smoothed into

centripetal Catmull–Rom splines.

$$d(c) = 1 + \min_{n \in N_4(c)} d(n),\ d(\text{goal}) = 0; \qquad e = \frac{\sum_{i=1}^{5} w_i s_i}{\sum_i s_i},\ \ u = K_p e + K_d \dot{e},\ \ v_{L,R} = v \mp u \tag{27.1}$$

with w_i the sensors' lateral offsets, $s_i \in \{0, 1\}$ their readings, $K_p = 3.2$, $K_d = 0.25$.



(a) M: flood field, routes and guide lines.

(b) A soldier's A* patrol spline.

Figure 27.1: Path debug views drawn by the program. Notice in (a) that the routes bend towards cover (cells the UAVs cannot see) rather than taking the straight line, and in (b) that the spline rounds the grid path's corners while keeping clear of the walls.

Method	Result	Cost	Used by
Flood fill (BFS)	distance to the nearest goal from every cell; any start descends greedily	O(cells) per replan, then $O(1)$ per step	rovers
A*	one shortest path, start to goal	expands far fewer cells with a good heuristic	soldiers
PD line following	smooth tracking of a drawn line with sensor feedback	5 samples per tick	rovers
Catmull–Rom (centripetal)	smooth curve through the waypoints, no loops or cusps	a few evaluations per sample	soldiers

Table 27.1: Differences between the path methods.

Where in the project. `src/ai/HideController.h,cpp (flood fill, guide line, PD gains kKp / kKd), src/scene/Garrison.cpp SitePlanner (A*), src/math/Spline.cpp`

Critique. The rover's sensors are simulated against a line the planner drew, not against a painted line in the scene, so the line-follower layer adds realistic lag and overshoot rather than new information. On top of it, a deliberate lateral weave, speed wander and short hesitations (`weave`, `speedWander`, `hesitate`) keep the UAVs' tracker from extrapolating a rover along one straight, constant-speed line.

28 Hard problems and trade-offs

Each problem below was met in this code. For each: why the class method alone falls short, what the program does, and what was measured. Numbers come from the v9 milestone notes (`docs/v9_notes/v9-notes.md`) or this report's plots; measurements on a GPU shared with other jobs are marked as such there.

1. Depth precision and z-fighting

Problem. A 600 m far plane (hills, sky objects) and a 0.05 m near plane (first-person rifle). **Why the class method falls short.** A perspective Z-buffer stores z_w, hyperbolic in distance (Eq. 11.1); with $F/N = 12\,000$ the 24-bit buffer resolves only about 1 m at 100 m (Figure 11.1b). **Solution.** Layered details are separated in depth instead of being coplanar: on each UAV pad the painted ring sits 2 mm and the centre disc 4 mm above the slab

(`objects/UavPad.cpp`). That is enough close to the camera, where pads are seen. **Measured.** The plot quantifies it; raising N to 0.5 m would improve precision tenfold but clip the first-person weapon. Reversed-Z needs OpenGL 4.5 (not the 3.3 baseline).

2. Many UAV searchlights versus the light budget

Problem. Two UAV searchlights, the moon, a beacon and the practical lamps of the camp at once. **Why the class method falls short.** Eq. (15.1) loops over all lights per fragment, and a shadowed spot light needs its own shadow map each frame. **Solution.** One shadowed spot slot (L2, UAV-1's searchlight, PCF shadow map); UAV-2's searchlight is an unshadowed dynamic light at 0.6 of the intensity with a 26 m range; practicals are capped at 16 per frame (`kMaxPracticals`); the searchlights are switched off by day and when docked. **Measured.** The light count per fragment is bounded (1 key + 1 spot + 1 dynamic + $\leq$16 practicals); the night/day difference in the benchmark (Section 29) includes this cost.

3. Thousands of grass blades

Problem. A dense meadow in a 36 m disc. **Why the class method falls short.** One draw call per blade, or one 12-triangle cube per blade (the v8 version), gave 6.3–7.7 million triangles per frame. **Solution.** GPU instancing of a 5-triangle curved strip, culled per cell, with a rank-based distance LOD that submits only a prefix of each cell's shuffled blades. **Measured.** At most 1.82 million grass triangles, and 86–121 fps on the ten fixed shots (was 80–101), Windows exe, 1080p EPIC (v9 M1).

4. Thermal convolution cost

Problem. The sensor PSF blurs every UAV frame on the CPU, twice per tick. **Why the class method falls short.** A $k \times k$ convolution costs k^2 multiply-adds per pixel. **Solution.** The binomial kernel is separable, $[1\,2\,1]^\top [1\,2\,1]/16$: a row pass then a column pass, $2k$ instead of k^2 (Figure 25.3d); the whole sensor and CV chain runs on a worker thread at 320×240. **Measured.** With $k = 3$ the saving is $9 \rightarrow 6$; it grows to $49 \rightarrow 14$ for a 7×7 Gaussian. A threaded run is byte-identical to a serial one.

5. Shadow acne and peter-panning

Problem. Self-shadowing stripes on lit surfaces (acne) versus shadows detached from their casters (peter-panning). **Why the class method falls short.** A constant depth bias large enough to remove acne on sloped surfaces lifts contact shadows off the ground. **Solution.** A normal-offset bias that grows with the surface slope, $\mathbf{p}' = \mathbf{p} + \mathbf{N}\,\text{texel}\,(1 + 2\,\text{slope})$, plus screen-space contact shadows (v9 M8) that put back the short shadows at feet and wheels. **Measured.** Contact shadows cost 0.10–0.17 ms (v9 M8); before/after images in `docs/v9_notes/m8/feet_contact_off_on.jpg`.

6. Transparency and eye adaptation

Problem. Rotor blur discs, beams and particles are translucent; walking from daylight into the hall changes the scene brightness by several stops. **Solution.** Translucent parts are drawn after all opaque geometry with depth test on and depth writes off (`Renderer::drawTranslucent`), so they are hidden by solid objects in front but never hide each other. They are *not* sorted back to front: two overlapping blur discs can blend in the wrong order, which is acceptable for faint discs but would not be for stained glass; exposure adapts with two time constants (0.35 s brighter, 1.1 s darker) plus a +1.25 EV indoor bias. When the post mode changes (visible to thermal) the TAA history and the adaptation restart, which fixed a washed-out thermal gimbal view found in the real exe. **Measured.** Figure 23.1d–e.

Summary

Problem	Solution	Result
Z-fighting	decals lifted 2–4 mm; near 0.05 m kept for first person	quantified, Fig. 11.1
Light budget	1 shadowed spot + 1 dynamic + ≤16 practicals	bounded per-fragment cost
Grass	instanced 5-tri strips, cell culling, rank LOD	6.3–7.7 M → ≤1.82 M triangles
Convolution	separable PSF on a worker thread	$k^2 \to 2k$; threaded = serial, byte for byte
Shadow bias	slope-scaled normal offset + contact shadows	0.10–0.17 ms
Transparency, exposure	opaque first, no depth writes (unsorted); asymmetric adaptation	0.35 s / 1.1 s

Table 28.1: Hard problems at a glance.

Back matter

29 Performance evaluation

Setup. `TS_BENCH_V10=1 ts --headless --quality epic --size 1920x1080 --seed 1 --benchmark report/data/bench.json`: the same orbit camera for every configuration; each configuration first settles (2 s of simulation after a clock change), then is timed in 3 interleaved rounds of 10 s (30 s per configuration). A frame includes the simulation tick, both UAV camera renders, the main frame and the HUD. Mean and 95th-percentile frame times come from every timed frame; draw calls include shadow passes. The program writes `bench.json.csv`, copied to `report/data/bench.csv`. Headless runs use the RTX 4070 through the WSL2 d3d12 driver, which is slower than the native Windows driver (v9 M0 measured 54–73 fps headless against 80–101 fps in the native exe on the same shots).

Configuration	mean ms	p95 ms	fps	draws	triangles
EPIC day 10:30	14.87	17.71	67.3	256	5,725,520
EPIC night 23:00 (game: waves, UAVs)	16.38	20.82	61.0	970	5,798,500
EPIC day, thermal view	10.72	14.80	93.3	71	2,726,600
EPIC night, thermal view	12.62	17.62	79.2	67	2,499,130
EPIC day, shadows off	16.42	21.02	60.9	206	5,857,520
EPIC night, shadows off	17.18	22.03	58.2	905	5,867,990
EPIC day, non-indexed	54.47	59.47	18.4	4046	5,855,790
ULTRA day	16.76	22.08	59.7	257	4,307,730
HIGH day	15.11	17.25	66.2	169	1,411,360
LOW day	14.16	16.04	70.6	165	1,315,170
v1 look (no post) day	13.01	14.69	76.9	148	1,341,050

Table 29.1: Benchmark (`report/data/bench.csv`; columns as the program wrote them). Day = 10:30, no enemies, UAVs docked; night = 23:00 with intruder waves, both UAVs flying and searchlights on.

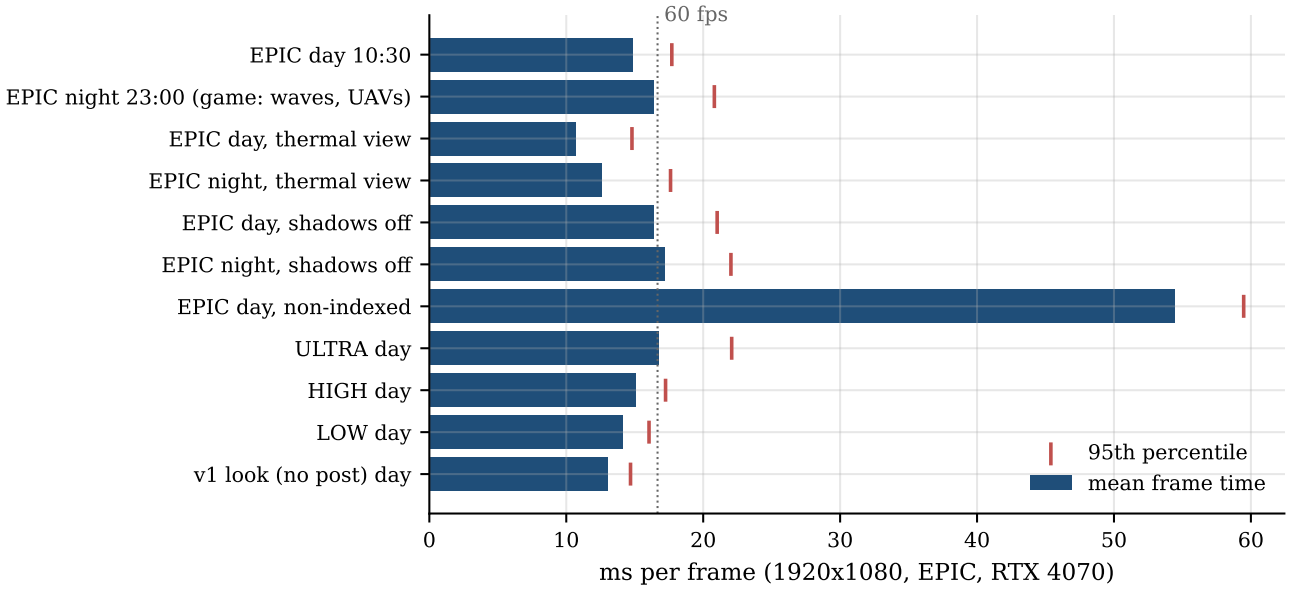


Figure 29.1: Frame time per configuration (bars: mean; ticks: 95th percentile; dotted line: 60 fps).

Analysis. Between the three rounds of one configuration the frame time varies by 0.1–6.6 ms, so differences of a few milliseconds are not significant. The night configuration (searchlights, beams, lamps, intruders, both UAV cameras rendering) costs +1.51 ms over day (14.87 to 16.38 ms). The thermal view frame is 4.15 ms shorter and draws 2.7 M triangles against 5.7 M in the visible view. Turning shadows off changes the day frame by +1.55 ms, within that variation. Non-indexed drawing is the one large effect: +39.60 ms (18.4 fps), with non-indexed meshes every triangle stores its own three vertices instead of sharing them (a sphere: 3672 vertices instead of 614, Table 6.1). Quality presets: LOW 14.16 ms, HIGH 15.11 ms, ULTRA 16.76 ms, EPIC 14.87 ms (ULTRA uses 8× MSAA, EPIC 4× plus TAA). In this headless setup much of a frame is CPU time: 5.61 ms simulation and UAV camera submission plus 9.25 ms rendering, of which 5.18 ms is the UAV images' sensor model and CV (synchronous in the benchmark; on a worker thread in the game). Fastest: EPIC day, thermal view at 93.3 fps.

config	avg_ms	1000 / avg_ms (fps)	fps check (diff)	triangles per draw
EPIC day 10:30	14.8698	67.2504	1.001×10^{-4}	22352.1282
EPIC night 23:00 (game: waves, UAVs)	16.3802	61.0493	1.564×10^{-5}	5975.1863
EPIC day, thermal view	10.7170	93.3097	-5.123×10^{-6}	38276.5582
EPIC night, thermal view	12.6203	79.2374	-1.809×10^{-4}	37309.3575
EPIC day, shadows off	16.4236	60.8880	9.045×10^{-5}	28473.8183
EPIC night, shadows off	17.1765	58.2191	-1.216×10^{-4}	6480.4223
EPIC day, non-indexed	54.4709	18.3584	2.624×10^{-5}	1447.3572
ULTRA day	16.7587	59.6705	9.950×10^{-5}	16746.2175
HIGH day	15.1059	66.1993	1.996×10^{-4}	8365.2507
LOW day	14.1625	70.6090	2.648×10^{-6}	7949.8168
v1 look (no post) day	13.0090	76.8699	-1.407×10^{-4}	9061.1486

input = value copied from the code, white = formula

Figure 29.2: Workbook sheet `Performance`: the same CSV imported, with a chart. (full sheet: Appendix A.21)

Per-pass costs from v9 (RTX 4070, 1080p EPIC). GTAO 0.53–1.71 ms (replaced SSAO, 0.39–0.59 ms); contact shadows 0.10–0.17 ms; PCSS added 1.0–1.9 ms to the day scene pass; vegetation 1.7–2.7 ms in gameplay views but 6.6 ms on the high overview (over its 3 ms budget); grass 0.4–1.5 ms. Measured on a GPU shared with other jobs.

30 Limitations and future work

- **Primitives:** no torus, wedge, hemisphere or helix generator; no level of detail for primitives (Section 22).

- **Teaching views not built:** interactive clip window, vertex/face normal lines, a live back-face toggle and a second camera's frustum drawn in the live scene (the Frame Explorer shows the latter two on a frozen frame).
- **Thermal panel:** the stages exist (lab figures, Frame Explorer image lab), but there is no single live tile strip with threshold and equalisation sliders; histogram equalisation and the optional DFT restoration are not in the program.
- **Moon:** a fixed-direction night light, not an orbiting body.
- **Modes and keys:** no dedicated observation/game switch (enemies follow the time of day, or `Shift+F2`); `N`/`D`, `G`, `H` and `F9` keep their earlier meanings.
- **AI:** two shifts instead of 6-hour shifts; AI soldiers do not fire, so "backup fire" exists only under player control.
- **Terrain (v9 M7):** the far hills are still scaled spheres; the planned heightfield ring with splatted CC0 materials was not built.
- **Budgets:** vegetation on the high overview (6.6 ms against 3 ms) and the static-camera shimmer target (2.6–6.6/255 against 1.5) are not met.
- **Depth precision:** reversed-Z would remove the z-fighting limit but needs OpenGL 4.5.

31 Conclusion

ThermalSense takes a small set of unit primitives through every stage of the graphics pipeline and makes each stage visible inside the running program. Table 31.1 maps each objective to the section that proves it.

Objective	Evidence	Section
O1 Modelling	7 primitives, 31 objects, 5,736 parts, *TRS* trace checked against the export	6–8, 19
O2 Pipeline stages	same scene in 5 projections, depth plots, clipping, HSR, raster, light terms, 3 shading models	9–17
O3 Ray tracing	5 render modes, ray tree, picking and line of sight	18, 24
O4 Day/night	solar model, derived light, sky, fog, exposure	21
O5 Thermal + vision	sensor model, CV pipeline, kernels, histograms	25
O6 Interaction	26 controllable units, AI state machines, flood fill + PD line follower, A*	23, 26, 27
O7 Measurement	benchmark CSV, per-pass GPU times, six hard problems with numbers	28, 29

Table 31.1: Objectives and where they are proved.

A 2-minute showcase video (1920×1080, 60 fps, EPIC) is rendered by the program itself with `ts --record FILE 120`: night wave, UAV tracking and strikes, first-person views, the ray-tracing modes and the day cycle. What is missing is listed honestly in Section 30.

References

[1] D. Hearn, M. P. Baker, and W. R. Carithers, *Computer Graphics with OpenGL.* Pearson, 4th ed., 2011.

[2] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*. Addison-Wesley, 2nd ed., 1996.

[3] T. Whitted, "An improved illumination model for shaded display," *Communications of the ACM*, vol. 23, no. 6, pp. 343–349, 1980.

[4] F. Kasten and A. T. Young, "Revised optical air mass tables and approximation formula," *Applied Optics*, vol. 28, no. 22, pp. 4735–4738, 1989.

[5] A. J. Preetham, P. Shirley, and B. Smits, "A practical analytic model for daylight," *Proc. SIGGRAPH*, pp. 91–100, 1999.

[6] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Pearson, 4th ed., 2018.

[7] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[8] T. Akenine-Möller, E. Haines, N. Hoffman, *et al.*, *Real-Time Rendering*. CRC Press, 4th ed., 2018.

[9] M. Segal and K. Akeley, "The OpenGL graphics system: A specification (version 3.3, core profile)." The Khronos Group, 2010.

[10] R. Fernando, “Percentage-closer soft shadows,” in *ACM SIGGRAPH Sketches*, 2005.

[11] G. Gribb and K. Hartmann, “Fast extraction of viewing frustum planes from the world-view-projection matrix,” in *Technical note*, 2001.

A Workbook sheets

Every sheet of `report/workbooks/cg_workbook.xlsx`, as LibreOffice prints it after recalculating every formula (vector pages; yellow cells are inputs copied from the code, white cells are formulas). The body of the report typesets the key blocks of each sheet as tables.

Primitives: vertex and triangle counts of the real mesh generators

Legend: yellow = input from the code; white = formula.

Source: src/primitives/{Cube,Sphere,Cylinder}.cpp, objects/Terrain.cpp createTerrainDisc, rendering/Grass.cpp; exported counts: ts --export-docs 'primitives'.

Primitive	Generator call	slices s	stacks / rings k	V formula	V	T formula	T	indices I = 3T	bytes indexed 24V+4I	bytes non-indexed 72T	V export	T export	dV	dT	instances (export)	scene triangles T export x inst
Cube	createCube()	-	-	6 faces x 4	24	6 faces x 2	12	36	720	864	24	12	0	0	2478	29736
Sphere	createSphere(36, 18)	36	18	(k-1)s + 2	614	2s(k-1)	1224	3672	29424	88128	614	1224	0	0	826	1011024
Cylinder	createCylinder(36) (capped)	36	-	4s + 2	146	4s	144	432	5232	10368	146	144	0	0	2133	307152
Cylinder (frustum)	createCylinder(36, 0.03, 0.5, false)	36	-	2s	72	2s	72	216	2592	5184	72	72	0	0	3	216
Cone	createCylinder(36, 0.5, 0, true)	36	-	3s + 1	109	2s	72	216	3480	5184	109	72	0	0	295	21240
Terrain disc	createTerrainDisc(130, 384, 36)	384	130	1 + k s	49921	s(2k-1)	99456	298368	2391576	7160832	49921	97440	0	2016	1	97440
Grass blade	Grass.cpp instanced strip (3 segments)	-	3	2k + 1	7	2k - 1	5	15	228	360	7	5	0	0	-	-
Scene total (library)															5736	1466808

dV = dT = 0: the closed-form counts match the exported meshes. Terrain dT = triangles whose centroid lies in the entrance-trench hole (dropped by createTerrainDisc).

Bytes: 24-byte vertex (position + normal, 6 floats), 4-byte indices; non-indexed = 3 vertices per triangle (72 bytes). Exported bytes_indexed / bytes_non_indexed agree.

Not primitives in this code (docs/audit.md section 2)

Shape	Status in the code
Cuboid	a scaled createCube() (S per part); not a separate generator
Hemisphere	NOT a primitive (only scaled / clipped spheres in places)
Torus	NOT a primitive: no generator exists
Wedge / prism	NOT a primitive: no generator exists
Helix	NOT a primitive: no generator exists

Parametric equations (local space, unit size)

Primitive	Point p(parameters)	Parameter ranges
Cube	p = 0.5 n + 0.5 (a u + b v), face frame (n, u, v) with u x v = n	6 faces, a, b in {-1, +1}; 2 CCW triangles (0,1,2), (0,2,3) per face
Sphere	p = 0.5 (sin(th) cos(ph), cos(th), sin(th) sin(ph)), normal = p / 0.5	th = pi k / stacks, k = 1..stacks-1; ph = 2 pi j / slices; 2 poles; south half mirrored (y -> -y)
Cylinder / frustum / cone	p = (r(z) cos(ph), r(z) sin(ph), z), r(-0.5) = r_bottom, r(+0.5) = r_top	ph = 2 pi j / slices; cylinder r = 0.5 / 0.5, frustum 0.5 / 0.03, cone 0.5 / 0 (apex); caps = fans
Terrain disc	p = (r cos(ph), h(x, z), r sin(ph)), r = 36 k / rings	k = 0..130, ph = 2 pi j / 384; h = terrainHeight(x, z); normal by central differences
Grass blade	p = (+-0.625 w(t), t - 0.5, 0), t in {0, 0.38, 0.72}; tip (0, 0.5, 0)	bent and scaled per instance in the vertex shader (wind, lean); one-sided strip

Figure A.1: Workbook sheet `Primitives` (smallest text 7.5 pt at print size).

Decomposition: objects as hierarchies of primitive parts

Legend: yellow = input from the code; white = formula.

Source: ts --export-docs 'components' (S, R_deg, T, parent, colour, specular, shininess) and 'objects'; objects/.cpp Builder. Rows below list every drawn part (leaf) of UAV-1 and of Soldier 1.*

1. Object-level primitive totals (whole scene, from the export)

Object	Cube	Sphere	Cylinder	Cylinder (frustum)	Cone	Terrain disc	parts (sum)	parts export	check	groups	animated nodes
Terrain	0	0	0	0	0	1	1	1	OK	1	0
Backdrop	0	24	10	0	86	0	120	120	OK	47	3
Environment	46	201	237	0	168	0	652	652	OK	89	2
Command Bunker	309	19	99	0	0	0	427	427	OK	18	18
Autonomous UAV	68	26	54	2	8	0	158	158	OK	34	26
Base	558	41	654	0	0	0	1253	1253	OK	33	0
Barracks hall	225	2	50	0	0	0	277	277	OK	5	1
UAV pads	4	8	6	0	0	0	18	18	OK	3	0
Target Rover	36	72	48	0	0	0	156	156	OK	12	48
Main Battle Tank	170	5	80	0	0	0	255	255	OK	29	118
Scout Helicopter	40	21	28	1	1	0	91	91	OK	19	17
AA Gatling Gun	19	5	54	0	0	0	78	78	OK	5	26
Air-defence radar	62	3	42	0	1	0	108	108	OK	4	4
SAM launcher	74	5	32	0	1	0	112	112	OK	9	10
MLRS rocket truck	45	9	76	0	6	0	136	136	OK	22	28
Main Battle Tank 2	170	5	80	0	0	0	255	255	OK	29	118
Main Battle Tank 3	170	5	80	0	0	0	255	255	OK	29	118
Ammunition bunker	19	10	14	0	0	0	43	43	OK	2	0
Camp camouflage	277	48	309	0	24	0	658	658	OK	21	0
Boom barrier	17	2	1	0	0	0	20	20	OK	2	2
Windsock	1	0	14	0	0	0	15	15	OK	7	6
Flag	40	2	5	0	0	0	47	47	OK	14	13
Soldier 1	16	39	20	0	0	0	75	75	OK	20	15
Soldier 2	16	39	20	0	0	0	75	75	OK	20	15
Soldier 3	16	39	20	0	0	0	75	75	OK	20	15
Soldier 4	16	39	20	0	0	0	75	75	OK	20	15
Soldier 5	16	39	20	0	0	0	75	75	OK	20	15
Soldier 6	16	39	20	0	0	0	75	75	OK	20	15
Soldier 7	16	39	20	0	0	0	75	75	OK	20	15
Soldier 8	16	39	20	0	0	0	75	75	OK	20	15
Sky Clock	0	1	0	0	0	0	1	1	OK	1	0
Scene total	2478	826	2133	3	295	1	5736	5736		595	678
Instances (export)	2478	826	2133	3	295	1	5736	5736			
check	OK	OK	OK	OK	OK	OK	OK	OK			

2. Parts per primitive (COUNTIFS ove

Primitive	UAV-1 COUNTIFS
Cube	34
Sphere	13
Cylinder	27
Cylinder (frustum)	1
Cone	4
Total parts	79

UAV-1 export = controllable 'UAV-1 D

the object 'Autonomous UAV' below (

Figure A.2: Workbook sheet `Decomposition` (smallest text 7.4 pt at print size).

3. UAV-1 (root UAV-00 'Autonomous UAV'): 79 drawn parts. M = T Ry Rx Rz S, world = parent_world M (the export's object 'Autonomous UAV' holds 2 UAVs).

ID	Part	Primitive	Parent (group)	Sx	Sy	Sz	Rx (deg)	Ry (deg)	Rz (deg)	Tx (m)	Ty (m)	Tz (m)	R	G
UAV-01	Fuselage	Cube	Autonomous UAV (UAV-00)	0.1600	0.0700	0.2600	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0800	0.0800
UAV-02	Canopy	Sphere	Autonomous UAV (UAV-00)	0.1500	0.0800	0.2200	0.0000	0.0000	0.0000	0.0000	0.0400	0.0100	0.6200	0.6400
UAV-03	Battery	Cube	Autonomous UAV (UAV-00)	0.1000	0.0400	0.1600	0.0000	0.0000	0.0000	0.0000	-0.0550	-0.0200	0.1400	0.1500
UAV-04	GPS mast	Cylinder	Autonomous UAV (UAV-00)	0.0080	0.0080	0.0500	90.0000	0.0000	0.0000	0.0000	0.0700	-0.0700	0.0800	0.0800
UAV-05	GPS puck	Cylinder	Autonomous UAV (UAV-00)	0.0500	0.0500	0.0120	90.0000	0.0000	0.0000	0.0000	0.0980	-0.0700	0.6200	0.6400
UAV-06	Arm Front-left	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	45.0000	0.0000	0.1061	0.0000	0.1061	0.0800	0.0800
UAV-08	Motor	Cylinder	Motor mount Front-left (UAV-07)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-10	Prop hub	Sphere	Rotor (UAV-09)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-11	Prop blade	Cube	Rotor (UAV-09)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-12	Rotor blur disc	Cylinder	Motor mount Front-left (UAV-07)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-13	Nav LED	Sphere	Motor mount Front-left (UAV-07)	0.0180	0.0180	0.0180	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	1.0000	0.1000
UAV-84	Motor cooling ring	Cylinder	Motor mount Front-left (UAV-07)	0.0500	0.0500	0.0100	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-14	Arm Front-right	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	-45.0000	0.0000	-0.1061	0.0000	0.1061	0.0800	0.0800
UAV-16	Motor	Cylinder	Motor mount Front-right (UAV-15)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-18	Prop hub	Sphere	Rotor (UAV-17)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-19	Prop blade	Cube	Rotor (UAV-17)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-20	Rotor blur disc	Cylinder	Motor mount Front-right (UAV-15)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-21	Nav LED	Sphere	Motor mount Front-right (UAV-15)	0.0180	0.0180	0.0180	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	0.1000	1.0000
UAV-85	Motor cooling ring	Cylinder	Motor mount Front-right (UAV-15)	0.0500	0.0500	0.0100	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-22	Arm Rear-left	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	135.0000	0.0000	0.1061	0.0000	-0.1061	0.0800	0.0800
UAV-24	Motor	Cylinder	Motor mount Rear-left (UAV-23)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-26	Prop hub	Sphere	Rotor (UAV-25)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-27	Prop blade	Cube	Rotor (UAV-25)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-28	Rotor blur disc	Cylinder	Motor mount Rear-left (UAV-23)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-86	Motor cooling ring	Cylinder	Motor mount Rear-left (UAV-23)	0.0500	0.0500	0.0100	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-87	Status LED	Sphere	Motor mount Rear-left (UAV-23)	0.0160	0.0160	0.0160	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	1.0000	0.5500
UAV-29	Arm Rear-right	Cube	Autonomous UAV (UAV-00)	0.0250	0.0180	0.2200	0.0000	-135.0000	0.0000	-0.1061	0.0000	-0.1061	0.0800	0.0800
UAV-31	Motor	Cylinder	Motor mount Rear-right (UAV-30)	0.0450	0.0450	0.0350	90.0000	0.0000	0.0000	0.0000	0.0200	0.0000	0.4500	0.4600
UAV-33	Prop hub	Sphere	Rotor (UAV-32)	0.0250	0.0250	0.0250	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4500	0.4600
UAV-34	Prop blade	Cube	Rotor (UAV-32)	0.2000	0.0040	0.0180	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1000	0.1000
UAV-35	Rotor blur disc	Cylinder	Motor mount Rear-right (UAV-30)	0.2050	0.2050	0.0020	90.0000	0.0000	0.0000	0.0000	0.0450	0.0000	0.1000	0.1000
UAV-88	Motor cooling ring	Cylinder	Motor mount Rear-right (UAV-30)	0.0500	0.0500	0.0100	90.0000	0.0000	0.0000	0.0000	0.0040	0.0000	0.1200	0.1200
UAV-89	Status LED	Sphere	Motor mount Rear-right (UAV-30)	0.0160	0.0160	0.0160	0.0000	0.0000	0.0000	0.0000	-0.0120	0.0000	1.0000	0.5500
UAV-36	Strobe	Sphere	Autonomous UAV (UAV-00)	0.0220	0.0160	0.0220	0.0000	0.0000	0.0000	0.0000	0.0200	-0.1350	1.0000	1.0000
UAV-37	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	-0.0600	-0.0800	-0.0700	0.0800	0.0800
UAV-38	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	-0.0600	-0.0800	0.0700	0.0800	0.0800
UAV-39	Landing skid	Cylinder	Autonomous UAV (UAV-00)	0.0140	0.0140	0.2400	0.0000	0.0000	0.0000	-0.0600	-0.1250	0.0000	0.0800	0.0800
UAV-40	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	0.0600	-0.0800	-0.0700	0.0800	0.0800
UAV-41	Landing strut	Cylinder	Autonomous UAV (UAV-00)	0.0120	0.0120	0.0900	90.0000	0.0000	0.0000	0.0600	-0.0800	0.0700	0.0800	0.0800
UAV-42	Landing skid	Cylinder	Autonomous UAV (UAV-00)	0.0140	0.0140	0.2400	0.0000	0.0000	0.0000	0.0600	-0.1250	0.0000	0.0800	0.0800
UAV-44	Gimbal yoke	Cube	Gimbal yaw (UAV-43)	0.0700	0.0300	0.0500	0.0000	0.0000	0.0000	0.0000	-0.0150	0.0000	0.0800	0.0800
UAV-46	Lamp housing	Cylinder	Gimbal pitch (UAV-45)	0.0600	0.0600	0.0800	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3000	0.3100
UAV-47	Lens	Sphere	Gimbal pitch (UAV-45)	0.0500	0.0500	0.0150	0.0000	0.0000	0.0000	0.0000	0.0000	0.0450	1.0000	0.9700

Figure A.3: Workbook sheet `Decomposition`, page 2 (smallest text 7.0 pt at print size).

UAV-48	Thermal camera	Cube	Gimbal pitch (UAV-45)	0.0300	0.0250	0.0350	0.0000	0.0000	0.0000	0.0000	0.0420	0.0000	0.0800	0.0800
UAV-83	Light cone	Cylinder (frustum)	Gimbal pitch (UAV-45)	4.8588	4.8588	9.7437	0.0000	0.0000	0.0000	0.0000	0.0000	4.9169	1.0000	0.9500
UAV-94	Camera window	Sphere	Gimbal pitch (UAV-45)	0.0180	0.0180	0.0060	0.0000	0.0000	0.0000	0.0000	0.0420	0.0180	0.0500	0.0500
UAV-50	Rail	Cube	Launcher rail L (UAV-49)	0.0120	0.0120	0.2200	0.0000	0.0000	0.0000	0.0000	0.0120	0.0000	0.2000	0.2100
UAV-51	Rail strut	Cube	Launcher rail L (UAV-49)	0.0080	0.0200	0.0080	0.0000	0.0000	0.0000	0.0000	0.0250	0.0500	0.2000	0.2100
UAV-53	Round body	Cylinder	Round (UAV-52)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-54	Round nose	Cone	Round (UAV-52)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-55	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-56	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-57	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-58	Round fin	Cube	Round (UAV-52)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-60	Round body	Cylinder	Round (UAV-59)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-61	Round nose	Cone	Round (UAV-59)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-62	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-63	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-64	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-65	Round fin	Cube	Round (UAV-59)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-67	Rail	Cube	Launcher rail R (UAV-66)	0.0120	0.0120	0.2200	0.0000	0.0000	0.0000	0.0000	0.0120	0.0000	0.2000	0.2100
UAV-68	Rail strut	Cube	Launcher rail R (UAV-66)	0.0080	0.0200	0.0080	0.0000	0.0000	0.0000	0.0000	0.0250	0.0500	0.2000	0.2100
UAV-70	Round body	Cylinder	Round (UAV-69)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-71	Round nose	Cone	Round (UAV-69)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-72	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-73	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-74	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-75	Round fin	Cube	Round (UAV-69)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-77	Round body	Cylinder	Round (UAV-76)	0.0220	0.0220	0.1700	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3200	0.3400
UAV-78	Round nose	Cone	Round (UAV-76)	0.0220	0.0220	0.0400	0.0000	0.0000	0.0000	0.0000	0.0000	0.1050	0.0500	0.0500
UAV-79	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	45.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-80	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	135.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-81	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	225.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-82	Round fin	Cube	Round (UAV-76)	0.0020	0.0300	0.0300	0.0000	0.0000	315.0000	0.0000	0.0000	-0.0700	0.2000	0.2100
UAV-90	Telemetry antenna	Cylinder	Autonomous UAV (UAV-00)	0.0050	0.0050	0.0900	-35.0000	0.0000	0.0000	-0.0500	-0.0400	-0.1300	0.0800	0.0800
UAV-91	Telemetry antenna	Cylinder	Autonomous UAV (UAV-00)	0.0050	0.0050	0.0900	-35.0000	0.0000	0.0000	0.0500	-0.0400	-0.1300	0.0800	0.0800
UAV-92	Battery strap	Cube	Autonomous UAV (UAV-00)	0.1060	0.0440	0.0120	0.0000	0.0000	0.0000	0.0000	-0.0550	-0.0700	0.5500	0.2200
UAV-93	Battery strap	Cube	Autonomous UAV (UAV-00)	0.1060	0.0440	0.0120	0.0000	0.0000	0.0000	0.0000	-0.0550	0.0300	0.5500	0.2200
UAV-95	Arming LED	Sphere	Autonomous UAV (UAV-00)	0.0120	0.0080	0.0120	0.0000	0.0000	0.0000	0.0400	0.0750	-0.0300	0.2000	0.6000

Figure A.4: Workbook sheet `Decomposition`, page 3 (smallest text 7.4 pt at print size).

4. Soldier 1 (root SLD1-00): 75 drawn parts (joint angles live in the parent groups).

ID	Part	Primitive	Parent (group)	Sx	Sy	Sz	Rx (deg)	Ry (deg)	Rz (deg)	Tx (m)	Ty (m)	Tz (m)	R	G
SLD1-02	Hips	Sphere	Pelvis (SLD1-01)	0.3778	0.2200	0.2400	0.0000	0.0000	0.0000	0.0000	0.0400	0.0000	0.2703	0.2726
SLD1-03	Belt	Cylinder	Pelvis (SLD1-01)	0.3883	0.2500	0.0700	90.0000	0.0000	0.0000	0.0000	0.1200	0.0000	0.2700	0.2350
SLD1-05	Hip joint	Sphere	Hip L (SLD1-04)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-06	Thigh	Cylinder	Hip L (SLD1-04)	0.1500	0.1500	0.4600	90.0000	0.0000	0.0000	0.0000	-0.2300	0.0000	0.2703	0.2726
SLD1-08	Knee joint	Sphere	Knee L (SLD1-07)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-09	Shin	Cylinder	Knee L (SLD1-07)	0.1200	0.1200	0.4400	90.0000	0.0000	0.0000	0.0000	-0.2200	0.0000	0.2703	0.2726
SLD1-11	Ankle joint	Sphere	Ankle L (SLD1-10)	0.1200	0.1200	0.1200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0750	0.0550
SLD1-12	Boot	Sphere	Ankle L (SLD1-10)	0.1300	0.1100	0.3000	0.0000	0.0000	0.0000	0.0000	-0.0250	0.0600	0.0750	0.0550
SLD1-76	Calf	Sphere	Knee L (SLD1-07)	0.1300	0.2200	0.1400	0.0000	0.0000	0.0000	0.0000	-0.1300	-0.0150	0.2703	0.2726
SLD1-77	Knee pad	Sphere	Knee L (SLD1-07)	0.1150	0.1300	0.0600	0.0000	0.0000	0.0000	0.0000	-0.0200	0.0600	0.1000	0.1000
SLD1-75	Thigh muscle	Sphere	Hip L (SLD1-04)	0.1750	0.3100	0.1800	0.0000	0.0000	0.0000	0.0000	-0.1500	0.0100	0.2703	0.2726
SLD1-14	Hip joint	Sphere	Hip R (SLD1-13)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-15	Thigh	Cylinder	Hip R (SLD1-13)	0.1500	0.1500	0.4600	90.0000	0.0000	0.0000	0.0000	-0.2300	0.0000	0.2703	0.2726
SLD1-17	Knee joint	Sphere	Knee R (SLD1-16)	0.1500	0.1500	0.1500	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.2703	0.2726
SLD1-18	Shin	Cylinder	Knee R (SLD1-16)	0.1200	0.1200	0.4400	90.0000	0.0000	0.0000	0.0000	-0.2200	0.0000	0.2703	0.2726
SLD1-20	Ankle joint	Sphere	Ankle R (SLD1-19)	0.1200	0.1200	0.1200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0750	0.0550
SLD1-21	Boot	Sphere	Ankle R (SLD1-19)	0.1300	0.1100	0.3000	0.0000	0.0000	0.0000	0.0000	-0.0250	0.0600	0.0750	0.0550
SLD1-81	Calf	Sphere	Knee R (SLD1-16)	0.1300	0.2200	0.1400	0.0000	0.0000	0.0000	0.0000	-0.1300	-0.0150	0.2703	0.2726
SLD1-82	Knee pad	Sphere	Knee R (SLD1-16)	0.1150	0.1300	0.0600	0.0000	0.0000	0.0000	0.0000	-0.0200	0.0600	0.1000	0.1000
SLD1-80	Thigh muscle	Sphere	Hip R (SLD1-13)	0.1750	0.3100	0.1800	0.0000	0.0000	0.0000	0.0000	-0.1500	0.0100	0.2703	0.2726
SLD1-23	Abdomen	Sphere	Spine (SLD1-22)	0.3568	0.3000	0.2400	0.0000	0.0000	0.0000	0.0000	0.1200	0.0000	0.3003	0.3029
SLD1-25	Torso	Sphere	Chest (SLD1-24)	0.4408	0.3500	0.2600	0.0000	0.0000	0.0000	0.0000	0.1000	0.0000	0.3003	0.3029
SLD1-26	Backpack	Cube	Chest (SLD1-24)	0.3200	0.3800	0.1600	0.0000	0.0000	0.0000	0.0000	0.0700	-0.2000	0.2902	0.2824
SLD1-27	Bedroll	Cylinder	Chest (SLD1-24)	0.1100	0.1100	0.3400	0.0000	90.0000	0.0000	0.0000	0.3100	-0.2100	0.2902	0.2824
SLD1-30	Rifle receiver	Cube	Rifle (held) (SLD1-28)	0.0600	0.1100	0.5200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0700	0.0700
SLD1-31	Rifle barrel	Cylinder	Rifle (held) (SLD1-28)	0.0350	0.0350	0.4000	0.0000	0.0000	0.0000	0.0000	0.0200	0.4500	0.0700	0.0700
SLD1-32	Rifle magazine	Cube	Rifle (held) (SLD1-28)	0.0400	0.1600	0.0700	15.0000	0.0000	0.0000	0.0000	-0.1100	0.1000	0.0700	0.0700
SLD1-89	Pistol grip	Cube	Rifle (held) (SLD1-28)	0.0350	0.1000	0.0450	-20.0000	0.0000	0.0000	0.0000	-0.0900	-0.1300	0.0700	0.0700
SLD1-90	Handguard	Cube	Rifle (held) (SLD1-28)	0.0580	0.0680	0.2400	0.0000	0.0000	0.0000	0.0000	0.0000	0.3400	0.0700	0.0700
SLD1-91	Optic	Cylinder	Rifle (held) (SLD1-28)	0.0420	0.0420	0.1300	0.0000	0.0000	0.0000	0.0000	0.0850	0.0200	0.0700	0.0700
SLD1-33	Rifle receiver	Cube	Rifle (slung) (SLD1-29)	0.0600	0.1100	0.5200	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0700	0.0700
SLD1-34	Rifle barrel	Cylinder	Rifle (slung) (SLD1-29)	0.0350	0.0350	0.4000	0.0000	0.0000	0.0000	0.0000	0.0200	0.4500	0.0700	0.0700
SLD1-35	Rifle magazine	Cube	Rifle (slung) (SLD1-29)	0.0400	0.1600	0.0700	15.0000	0.0000	0.0000	0.0000	-0.1100	0.1000	0.0700	0.0700
SLD1-92	Pistol grip	Cube	Rifle (slung) (SLD1-29)	0.0350	0.1000	0.0450	-20.0000	0.0000	0.0000	0.0000	-0.0900	-0.1300	0.0700	0.0700
SLD1-93	Handguard	Cube	Rifle (slung) (SLD1-29)	0.0580	0.0680	0.2400	0.0000	0.0000	0.0000	0.0000	0.0000	0.3400	0.0700	0.0700
SLD1-94	Optic	Cylinder	Rifle (slung) (SLD1-29)	0.0420	0.0420	0.1300	0.0000	0.0000	0.0000	0.0000	0.0850	0.0200	0.0700	0.0700
SLD1-36	Carried crate	Cube	Chest (SLD1-24)	0.4600	0.3200	0.3600	0.0000	0.0000	0.0000	0.0000	-0.1000	0.4000	0.4200	0.3300
SLD1-38	Neck	Cylinder	Neck (SLD1-37)	0.1250	0.1150	0.1100	90.0000	0.0000	0.0000	0.0000	0.0350	0.0000	0.3760	0.2629
SLD1-40	Head	Sphere	Head (SLD1-39)	0.1800	0.2250	0.2050	0.0000	0.0000	0.0000	0.0000	0.0800	0.0000	0.3760	0.2629
SLD1-41	Helmet	Sphere	Head (SLD1-39)	0.2450	0.1900	0.2650	-6.0000	0.0000	0.0000	0.0000	0.1450	-0.0100	0.2455	0.2644
SLD1-42	Helmet brim	Cylinder	Head (SLD1-39)	0.2550	0.2800	0.0220	84.0000	0.0000	0.0000	0.0000	0.1380	-0.0050	0.1719	0.1851
SLD1-61	Jaw	Sphere	Head (SLD1-39)	0.1300	0.0850	0.1150	0.0000	0.0000	0.0000	0.0000	0.0180	0.0370	0.3760	0.2629
SLD1-62	Nose	Sphere	Head (SLD1-39)	0.0320	0.0500	0.0450	-15.0000	0.0000	0.0000	0.0000	0.0620	0.0980	0.3760	0.2629

Figure A.5: Workbook sheet `Decomposition`, page 4 (smallest text 7.0 pt at print size).

SLD1-63	Eye	Sphere	Head (SLD1-39)	0.0240	0.0160	0.0120	0.0000	0.0000	0.0000	0.0330	0.0930	0.0940	0.0350	0.0300
SLD1-64	Ear	Sphere	Head (SLD1-39)	0.0220	0.0580	0.0420	0.0000	0.0000	0.0000	0.0890	0.0750	-0.0050	0.3760	0.2629
SLD1-65	Chin strap	Cylinder	Head (SLD1-39)	0.0120	0.0120	0.1695	58.8041	-37.1309	0.0000	0.0615	0.0525	0.0250	0.0900	0.0850
SLD1-66	Eye	Sphere	Head (SLD1-39)	0.0240	0.0160	0.0120	0.0000	0.0000	0.0000	-0.0330	0.0930	0.0940	0.0350	0.0300
SLD1-67	Ear	Sphere	Head (SLD1-39)	0.0220	0.0580	0.0420	0.0000	0.0000	0.0000	-0.0890	0.0750	-0.0050	0.3760	0.2629
SLD1-68	Chin strap	Cylinder	Head (SLD1-39)	0.0120	0.0120	0.1695	58.8041	37.1309	0.0000	-0.0615	0.0525	0.0250	0.0900	0.0850
SLD1-69	Goggles	Cube	Head (SLD1-39)	0.1500	0.0320	0.0300	-35.0000	0.0000	0.0000	0.0000	0.2150	0.0920	0.0400	0.0500
SLD1-70	NVG mount	Cube	Head (SLD1-39)	0.0400	0.0350	0.0250	-10.0000	0.0000	0.0000	0.0000	0.1630	0.1280	0.0700	0.0700
SLD1-71	Trapezius	Sphere	Neck (SLD1-37)	0.2000	0.0850	0.1500	0.0000	0.0000	-20.0000	0.0850	-0.0200	-0.0200	0.3003	0.3029
SLD1-73	Trapezius	Sphere	Neck (SLD1-37)	0.2000	0.0850	0.1500	0.0000	0.0000	20.0000	-0.0850	-0.0200	-0.0200	0.3003	0.3029
SLD1-44	Shoulder joint	Sphere	Shoulder L (SLD1-43)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-45	Upper arm	Cylinder	Shoulder L (SLD1-43)	0.1300	0.1300	0.3000	90.0000	0.0000	0.0000	0.0000	-0.1500	0.0000	0.3183	0.3211
SLD1-47	Elbow joint	Sphere	Elbow L (SLD1-46)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-48	Forearm	Cylinder	Elbow L (SLD1-46)	0.1100	0.1100	0.2600	90.0000	0.0000	0.0000	0.0000	-0.1300	0.0000	0.3183	0.3211
SLD1-50	Hand	Sphere	Wrist L (SLD1-49)	0.0900	0.1200	0.0800	0.0000	0.0000	0.0000	0.0000	-0.0500	0.0000	0.1100	0.1000
SLD1-78	Forearm muscle	Sphere	Elbow L (SLD1-46)	0.1200	0.1800	0.1200	0.0000	0.0000	0.0000	0.0000	-0.0700	0.0000	0.3183	0.3211
SLD1-79	Elbow pad	Sphere	Elbow L (SLD1-46)	0.1000	0.1100	0.0550	0.0000	0.0000	0.0000	0.0000	-0.0100	-0.0550	0.1000	0.1000
SLD1-72	Deltoid	Sphere	Shoulder L (SLD1-43)	0.1550	0.1700	0.1500	0.0000	0.0000	0.0000	0.0100	-0.0350	0.0000	0.3183	0.3211
SLD1-52	Shoulder joint	Sphere	Shoulder R (SLD1-51)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-53	Upper arm	Cylinder	Shoulder R (SLD1-51)	0.1300	0.1300	0.3000	90.0000	0.0000	0.0000	0.0000	-0.1500	0.0000	0.3183	0.3211
SLD1-55	Elbow joint	Sphere	Elbow R (SLD1-54)	0.1300	0.1300	0.1300	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3183	0.3211
SLD1-56	Forearm	Cylinder	Elbow R (SLD1-54)	0.1100	0.1100	0.2600	90.0000	0.0000	0.0000	0.0000	-0.1300	0.0000	0.3183	0.3211
SLD1-58	Hand	Sphere	Wrist R (SLD1-57)	0.0900	0.1200	0.0800	0.0000	0.0000	0.0000	0.0000	-0.0500	0.0000	0.1100	0.1000
SLD1-59	Wrench	Cylinder	Wrist R (SLD1-57)	0.0300	0.0300	0.2600	0.0000	0.0000	0.0000	0.0000	-0.0600	0.1000	0.6200	0.6300
SLD1-83	Forearm muscle	Sphere	Elbow R (SLD1-54)	0.1200	0.1800	0.1200	0.0000	0.0000	0.0000	0.0000	-0.0700	0.0000	0.3183	0.3211
SLD1-84	Elbow pad	Sphere	Elbow R (SLD1-54)	0.1000	0.1100	0.0550	0.0000	0.0000	0.0000	0.0000	-0.0100	-0.0550	0.1000	0.1000
SLD1-74	Deltoid	Sphere	Shoulder R (SLD1-51)	0.1550	0.1700	0.1500	0.0000	0.0000	0.0000	-0.0100	-0.0350	0.0000	0.3183	0.3211
SLD1-60	Plate carrier	Cube	Chest (SLD1-24)	0.3463	0.2700	0.2950	0.0000	0.0000	0.0000	0.0000	0.0700	0.0050	0.2700	0.2350
SLD1-85	Magazine pouches	Cube	Chest (SLD1-24)	0.2500	0.1100	0.0600	0.0000	0.0000	0.0000	0.0000	0.0000	0.1750	0.2700	0.2350
SLD1-86	Admin pouch	Cube	Chest (SLD1-24)	0.1300	0.0700	0.0350	0.0000	0.0000	0.0000	0.0000	0.1150	0.1660	0.2295	0.1998
SLD1-87	Radio	Cube	Chest (SLD1-24)	0.0500	0.1500	0.0800	0.0000	0.0000	0.0000	0.1900	0.0400	-0.0500	0.1000	0.1100
SLD1-88	Radio antenna	Cylinder	Chest (SLD1-24)	0.0080	0.0080	0.3693	-81.2925	153.4349	0.0000	0.2025	0.2975	-0.0950	0.0700	0.0700

Figure A.6: Workbook sheet `Decomposition`, page 5 (smallest text 7.4 pt at print size).

UAV-1 export	check	Soldier 1 COUNTIFS	Soldier 1 export	check
34	OK	16	16	OK
13	OK	39	39	OK
27	OK	20	20	OK
1	OK	0	0	OK
4	OK	0	0	OK
79	OK	75	75	OK

Figure A.7: Workbook sheet `Decomposition`, page 6 (smallest text 9.4 pt at print size).

B	ks	shininess n
0.0900	0.6000	64.0000
0.6600	0.8500	96.0000
0.2000	0.4000	48.0000
0.0900	0.6000	64.0000
0.6600	0.8500	96.0000
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.0500	0.4000	64.0000
0.1300	0.6000	64.0000
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.2500	0.4000	64.0000
0.1300	0.6000	64.0000
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.1300	0.6000	64.0000
0.0800	0.2950	111.4300
0.0900	0.6000	64.0000
0.5000	0.9000	128.0000
0.5000	0.9000	128.0000
0.1100	0.3000	32.0000
0.1100	0.2000	16.0000
0.1300	0.6000	64.0000
0.0800	0.2950	111.4300
1.0000	0.5000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.3300	0.8000	96.0000
0.8500	1.0000	128.0000

Figure A.8: Workbook sheet `Decomposition`, page 7 (smallest text 7.1 pt at print size).

0.0900	0.6000	64.0000
0.8000	0.0000	1.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2800	0.5000	64.0000
0.0600	0.9000	128.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.2000	0.5000	64.0000
0.0900	0.6000	64.0000
0.0900	0.6000	64.0000
0.0500	0.1000	3.0314
0.0500	0.1000	3.0314
1.0000	0.2950	111.4300

Figure A.9: Workbook sheet `Decomposition`, page 8 (smallest text 8.9 pt at print size).

B	ks	shininess n
0.1940	0.0520	2.0000
0.1550	0.1240	4.7240
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.0400	0.1750	12.1257
0.0400	0.1750	12.1257
0.1940	0.0520	2.0000
0.0850	0.1000	3.0314
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.1940	0.0520	2.0000
0.0400	0.1750	12.1257
0.0400	0.1750	12.1257
0.1940	0.0520	2.0000
0.0850	0.1000	3.0314
0.1940	0.0520	2.0000
0.2156	0.0550	2.0000
0.2156	0.0550	2.0000
0.1825	0.0700	2.0000
0.1825	0.0700	2.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.0750	0.4350	16.0000
0.2000	0.1150	4.0000
0.1904	0.1750	12.1257
0.1904	0.1750	12.1257
0.1889	0.0550	2.0000
0.1322	0.1980	9.1896
0.1904	0.1750	12.1257
0.1904	0.1750	12.1257

Figure A.10: Workbook sheet `Decomposition`, page 9 (smallest text 7.1 pt at print size).

0.0250	0.2800	84.4485
0.1904	0.1750	12.1257
0.0600	0.0850	2.2974
0.0250	0.2800	84.4485
0.1904	0.1750	12.1257
0.0600	0.0850	2.2974
0.0600	0.3465	194.0120
0.0750	0.4350	16.0000
0.2156	0.0550	2.0000
0.2156	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.6600	0.5110	48.5029
0.2285	0.0550	2.0000
0.0850	0.1000	3.0314
0.2285	0.0550	2.0000
0.1550	0.1240	4.7240
0.1550	0.1240	4.7240
0.1318	0.1240	4.7240
0.0800	0.1710	5.2780
0.0750	0.4350	16.0000

Figure A.11: Workbook sheet `Decomposition`, page 10 (smallest text 9.4 pt at print size).

Transform_Trace: one real UAV vertex from local space to the window

Legend: yellow = input from the code; white = formula.

Source: ts --export-docs component UAV-14 'Arm Front-right' (S, R_deg, T, local_matrix, world_matrix, parent UAV-00 world_matrix); objects/Builder M = T Ry Rx Rz S; View_Projection sheet (V, P, viewport).

Part	UAV-14 Arm Front-right (Cube, createCube)			
	x	y	z	w
S (scale)	0.0250	0.0180	0.2200	
R_deg (pitch x, yaw y, roll z)	0	-45	0	
T (m)	-0.1061	0	0.1061	
local vertex p (cube corner)	0.5000	0.5000	0.5000	1

Parent	UAV-00 Autonomous UAV (root, R = (0, 180, 0), T = (0, 7.5, 12.5))

createCube() vertex 3 = (0.5, 0.5, 0.5), +X face (mesh_samples in the export)

S = scale(Sx, Sy, Sz)

0.0250	0	0	0
0	0.0180	0	0
0	0	0.2200	0
0	0	0	1

Rx(pitch)

1	0	0	0
0	1.0000	0.0000	0
0	0.0000	1.0000	0
0	0	0	1

Ry(yaw)

0.7071	0	-0.7071	0
0	1	0	0
0.7071	0	0.7071	0
0	0	0	1

Rz(roll)

1.0000	0.0000	0	0
0.0000	1.0000	0	0
0	0	1	0
0	0	0	1

T = translate(Tx, Ty, Tz)

1	0	0	-0.1061
0	1	0	0.0000
0	0	1	0.1061
0	0	0	1

Ry Rx

0.7071	0.0000	-0.7071	0.0000
0.0000	1.0000	0.0000	0.0000
0.7071	0.0000	0.7071	0.0000
0.0000	0.0000	0.0000	1.0000

R = Ry Rx Rz

0.7071	0.0000	-0.7071	0.0000
0.0000	1.0000	0.0000	0.0000
0.7071	0.0000	0.7071	0.0000
0.0000	0.0000	0.0000	1.0000

R S

0.0177	0.0000	-0.1556	0.0000
0.0000	0.0180	0.0000	0.0000
0.0177	0.0000	0.1556	0.0000
0.0000	0.0000	0.0000	1.0000

M = T R S (local matrix)

0.0177	0.0000	-0.1556	-0.1061
0.0000	0.0180	0.0000	0.0000
0.0177	0.0000	0.1556	0.1061
0.0000	0.0000	0.0000	1.0000

M exported (local_matrix)

0.0177	0	-0.1556	-0.1061
0	0.0180	0	0
0.0177	0	0.1556	0.1061
0	0	0	1

\|M - M exported\|

0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000

Parent world (UAV-00, exported)

-1	0	0.0000	0
0	1	0	7.5000
0.0000	0	-1	12.5000
0	0	0	1

World = Parent M

-0.0177	0.0000	0.1556	0.1061
0.0000	0.0180	0.0000	7.5000
-0.0177	0.0000	-0.1556	12.3939
0.0000	0.0000	0.0000	1.0000

World exported (world_matrix)

-0.0177	0	0.1556	0.1061
0	0.0180	0	7.5000
-0.0177	0	-0.1556	12.3939
0	0	0	1

\|World - World exported\|

0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000

View V (View_Projection sheet)

0.8480	0.0000	-0.5299	0.0000
-0.2406	0.8910	-0.3850	-0.7128
0.4722	0.4540	0.7556	-34.3632
0.0000	0.0000	0.0000	1.0000

Projection P (View_Projection sheet)

1.2063	0.0000	0.0000	0.0000
0.0000	2.1445	0.0000	0.0000
0.0000	0.0000	-1.0002	-0.1000
0.0000	0.0000	-1.0000	0.0000

Vertex pipeline (column vectors)

stage	x	y	z	w
local p	0.5000	0.5000	0.5000	1.0000
world = World p	0.1750	7.5090	12.3073	1.0000
eye = V world	-6.3735	1.1973	-21.5719	1.0000
clip = P eye	-7.6882	2.5676	21.4755	21.5719
NDC = clip / w	-0.3564	0.1190	0.9955	1.0000
window (x px, y px top-left, depth, -)	617.8569	475.7275	0.9978	-

Checks

check	value
max |M - export|	1.80E-08
max |World - export|	2.09E-07
inside clip volume |x|,|y|,|z| <= w	yes
eye-space distance -z_eye (m)	21.5719

Exported matrices are float32: differences of ~1e-7 are rounding. Window: x = (ndc.x/2+0.5) W, y = (1-(ndc.y/2+0.5)) H, d = ndc.z/2+0.5 (glDepthRange 0..1)

Figure A.12: Workbook sheet `Transform_Trace` (smallest text 7.3 pt at print size).

View_Projection: default orbit camera, lookAt basis, perspective and depth

Legend: yellow = input from the code; white = formula.

Source: scene/Camera.h (target, yaw 32, pitch 27, distance 34, fov 50), Camera.cpp compute() (orbit eye), Math3D.cpp lookAt / perspective, near 0.05 far 600, 24-bit depth (FrameBuffers.cpp).

Input / derived	x	y	z
target (m)	0.0000	0.8000	0.0000
up	0.0000	1.0000	0.0000
yaw (deg)	32		
pitch (deg)	27		
distance (m)	34		
fov y (deg)	50		
width (px)	1920		
height (px)	1080		
near (m)	0.0500		
far (m)	600		
near, comparison (m)	0.5000		
aspect = width / height	1.7778		
eye = target + d (cos p sin y, sin p, cos p	16.0535	16.2357	25.6910
eye - target	16.0535	15.4357	25.6910
n = normalize(eye - target)	0.4722	0.4540	0.7556
up x n	0.7556	0.0000	-0.4722
u = normalize(up x n)	0.8480	0.0000	-0.5299
v = n x u	-0.2406	0.8910	-0.3850
code: s = normalize(f x up), f = -n; |s - u|	0.0000		
u . v, u . n, v . n (orthogonal: 0)	0.0000	0.0000	0.0000

Depth mapping (24-bit fixed point: step = 1 / (2^24 - 1))

z_eye (m)	z_ndc (near 0.05)	depth d (near 0.05)	metres per depth step (near 0.05)	z_ndc (near 0.5)	depth d (near 0.5)	metres per depth step (near 0.5)
-0.0500	-1.0000	0.0000	2.98E-09	clipped	clipped	clipped
-0.1000	0.0001	0.5000	1.19E-08	clipped	clipped	clipped
-0.5000	0.8002	0.9001	2.98E-07	-1.0000	0.0000	2.98E-08
-1.0000	0.9002	0.9501	1.19E-06	0.0008	0.5004	1.19E-07
-2.0000	0.9502	0.9751	4.77E-06	0.5013	0.7506	4.76E-07
-5.0000	0.9802	0.9901	2.98E-05	0.8015	0.9008	2.98E-06
-10.0000	0.9902	0.9951	1.19E-04	0.9016	0.9508	1.19E-05
-34.0000	0.9972	0.9986	1.38E-03	0.9722	0.9861	1.38E-04
-50.0000	0.9982	0.9991	2.98E-03	0.9817	0.9908	2.98E-04
-100.0000	0.9992	0.9996	1.19E-02	0.9917	0.9958	1.19E-03
-200.0000	0.9997	0.9998	4.77E-02	0.9967	0.9983	4.76E-03
-400.0000	0.9999	1.0000	1.91E-01	0.9992	0.9996	1.91E-02
-600.0000	1.0000	1.0000	4.29E-01	1.0000	1.0000	4.29E-02

Summary

depth step at 100 m (m)	1.19E-02	1.19E-03
ratio near 0.05 / near 0.5 at 100 m	10.0075	
share of [0, 1] depth used by the first 1 m	0.9501	0.5004
share of depth used beyond 10 m	0.0049	0.0492

d = (F / (F - N)) (1 - N / \|z\|): precision falls with z^2.

Step size dz = step (F - N) z^2 / (F N): ten times the near plane = ten times finer steps.

Two surfaces closer than one step at 100 m share a depth value: z-fighting.

The code keeps near 0.05 m for the first-person views; 24-bit depth.

View V = lookAt(eye, target, up): rows u, v, n; last column

row 0 (u)	0.8480	0.0000	-0.5299	0.0000
row 1 (v)	-0.2406	0.8910	-0.3850	-0.7128
row 2 (n)	0.4722	0.4540	0.7556	-34.3632
row 3	0	0	0	1

t = 1 / tan(fov / 2)	2.1445

Perspective P (Math3D perspective, OpenGL clip z in [-w, w])

row 0	1.2063	0	0	0
row 1	0	2.1445	0	0
row 2	0	0	-1.0002	-0.1000
row 3	0	0	-1	0

Code (lookAt): f = normalize(target - eye), s = normalize(f x up), u' = s x f; rows s, u', -f. Since f = -n: s = u and u' = v, the same matrix.

Figure A.13: Workbook sheet `View_Projection` (smallest text 7.6 pt at print size).

Clipping: Cohen-Sutherland, Liang-Barsky, Sutherland-Hodgman (textbook worked example)

Legend: yellow = input from the code; white = formula.

Source: Worked textbook example (window and lines chosen for teaching). The code itself clips on the GPU (clip volume -w <= x, y, z <= w) and culls on the CPU with math/Frustum (6 planes, sphere test).

Window	xmin	0	xmax	10	ymin	0	ymax	8

Outcode bits T B R L = (y > ymax)(y < ymin)(x > xmax)(x < xmin); code = 8T + 4B + 2R + L

1. Cohen-Sutherland outcodes

line	x0	y0	x1	y1	T0	B0	R0	L0	code0	T1	B1	R1	L1	code1	bits 0	bits 1	AND of codes	decision
L1	2	2	8	6	0	0	0	0	0	0	0	0	0	0	0000	0000	0	accept (both inside)
L2	-2	-1	-1	9	0	1	0	1	5	1	0	0	1	9	0101	1001	1	reject (same side)
L3	-3	4	6	12	0	0	0	1	1	1	0	0	0	8	0001	1000	0	clip (subdivide)
L4	-2	3	12	5	0	0	0	1	1	0	0	1	0	2	0001	0010	0	clip (subdivide)

2. Liang-Barsky: x = x0 + t dx, y = y0 + t dy; p1 = -dx, q1 = x0 - xmin; p2 = dx, q2 = xmax - x0; p3 = -dy, q3 = y0 - ymin; p4 = dy, q4 = ymax - y0

line	dx	dy	p1	p2	p3	p4	q1	q2	q3	q4	t enter = max(0, q/p, p<0)	t exit = min(1, q/p, p>0)	parallel outside	visible	x0'	y0'	x1'	y1'
L1	6	4	-6	6	-4	4	2	8	2	6	0.0000	1.0000	no	yes	2.0000	2.0000	8.0000	6.0000
L2	1	10	-1	1	-10	10	-2	12	-1	9	2.0000	0.9000	no	no	-	-	-	-
L3	9	8	-9	9	-8	8	-3	13	4	4	0.3333	0.5000	no	yes	0.0000	6.6667	1.5000	8.0000
L4	14	2	-14	14	-2	2	-2	12	3	5	0.1429	0.8571	no	yes	0.0000	3.2857	10.0000	4.7143

3. Sutherland-Hodgman: triangle clipped against the 4 window edges in turn (output list -> next stage input)

Triangle	A	-2	3	B	7	-2	C	12	10

Stage 1: left x >= xmin

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	-2.0000	3.0000	7.0000	-2.0000	0	1	0.0000	1.8889	I, P
2-3	7.0000	-2.0000	12.0000	10.0000	1	1	-	-	P
3-1	12.0000	10.0000	-2.0000	3.0000	1	0	0.0000	4.0000	I
out	(0, 1.89) (7, -2) (12, 10) (0, 4)								

4 vertices

Stage 2: right x <= xmax

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	0.0000	1.8889	7.0000	-2.0000	1	1	-	-	P
2-3	7.0000	-2.0000	12.0000	10.0000	1	0	10.0000	5.2000	I
3-4	12.0000	10.0000	0.0000	4.0000	0	1	10.0000	9.0000	I, P
4-1	0.0000	4.0000	0.0000	1.8889	1	1	-	-	P
out	(7, -2) (10, 5.2) (10, 9) (0, 4) (0, 1.89)								

5 vertices

Stage 3: bottom y >= ymin

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	7.0000	-2.0000	10.0000	5.2000	0	1	7.8333	0.0000	I, P
2-3	10.0000	5.2000	10.0000	9.0000	1	1	-	-	P
3-4	10.0000	9.0000	0.0000	4.0000	1	1	-	-	P
4-5	0.0000	4.0000	0.0000	1.8889	1	1	-	-	P
5-1	0.0000	1.8889	7.0000	-2.0000	1	0	3.4000	0.0000	I
out	(7.83, 0) (10, 5.2) (10, 9) (0, 4) (0, 1.89) (3.4, 0)								

6 vertices

Stage 4: top y <= ymax

edge	Sx	Sy	Px	Py	S in	P in	Ix	Iy	output
1-2	7.8333	0.0000	10.0000	5.2000	1	1	-	-	P
2-3	10.0000	5.2000	10.0000	9.0000	1	0	10.0000	8.0000	I
3-4	10.0000	9.0000	0.0000	4.0000	0	1	8.0000	8.0000	I, P
4-5	0.0000	4.0000	0.0000	1.8889	1	1	-	-	P
5-6	0.0000	1.8889	3.4000	0.0000	1	1	-	-	P
6-1	3.4000	0.0000	7.8333	0.0000	1	1	-	-	P
out	(10, 5.2) (10, 8) (8, 8) (0, 4) (0, 1.89) (3.4, 0) (7.83, 0)								

7 vertices

Each stage keeps the output-list rule: in-in -> P; in-out -> I; out-in -> I, P; out-out -> none. The final list is the clipped polygon (fan-triangulated afterwards).

Figure A.14: Workbook sheet `Clipping` (smallest text 7.1 pt at print size).

Illumination: ambient + Lambert diffuse + Blinn-Phong specular at one roof point

Legend: yellow = input from the code; white = formula.

Source: scene/DayCycle (solar model, transmittance, 3.2 smoothstep(-1, 10, e), ambient mixLog); export 'lights'; ShaderSources.cpp brdf(); BNK-171 'Roof slab A'.

Quantity	Noon	Sunset	Night (moon)	UAV spot (beam axis)	UAV spot (roof point)
hour (clock)	12.00	17.50	23.00	23.00	23.00
H = 15 (hour - 12) (deg)	0.0000	82.5000	165.0000	165.0000	165.0000
sin e = sin phi sin d + cos phi cos d cos H	0.8878	0.0889	-0.9187	-0.9187	-0.9187
sun elevation e (deg)	62.6000	5.0981	-66.7312	-66.7312	-66.7312
daylight w = smoothstep(-8, 6, e)	1.0000	0.9881	0.0000	0.0000	0.0000
surface point P.x (m)	0.0000	0.0000	0.0000	0.0000	0.0000
surface point P.y (m)	-0.4500	-0.4500	-0.4500	-0.4500	-0.4500
surface point P.z (m)	-1.3000	-1.3000	-1.3000	5.8195	-1.3000
to light (unnormalised) x	0.0000	-0.9883	0.5598	0.0000	0.0000
to light (unnormalised) y	0.8878	0.0889	0.3868	7.8060	7.8060
to light (unnormalised) z	-0.4602	-0.1244	-0.7328	6.5505	13.6700
distance d = |to light| (m)	1.0000	1.0000	1.0000	10.1903	15.7417
L.x (unit)	0.0000	-0.9883	0.5598	0.0000	0.0000
L.y (unit)	0.8878	0.0889	0.3868	0.7660	0.4959
L.z (unit)	-0.4602	-0.1244	-0.7328	0.6428	0.8684
angle to spot axis (deg)	-	-	-	0.4138	20.2741
spot cone c^2, c = clamp((cos a - cos o)/(cos i - co	-	-	-	1.0000	0.0000
attenuation 1 / (1 + kl d + kq d^2)	-	-	-	0.2162	0.1084
air mass m (Kasten-Young)	1.1257	10.1423	-	-	-
light intensity (scalar)	3.2000	1.8600	0.9500	1.0811	0.0000
N . L (clamped at 0)	0.8878	0.0889	0.3868	0.7660	0.4959
L + V (H before normalising) x	0.4514	-0.5368	1.0112	0.5261	0.4514
L + V y	1.3570	0.5581	0.8560	1.3129	0.9651
L + V z	0.2988	0.6346	0.0262	1.2941	1.6274
N . H	0.9288	0.5574	0.6460	0.6848	0.4962
(N . H)^n (0 when N . L = 0)	0.8627	0.3107	0.4173	0.4690	0.2462

Per-channel terms (light colour = transmittance for the sun)

Term (channel)	Noon	Sunset	Night (moon)	UAV spot (beam axis)	UAV spot (roof point)
Il = light colour x intensity R	3.1760	1.0747	0.5225	1.0811	0.0000
Il = light colour x intensity G	3.1560	0.6804	0.5890	1.0271	0.0000
Il = light colour x intensity B	3.1166	0.2727	0.7600	0.8865	0.0000
ambient Ia R	0.3400	0.3330	0.0600	0.0600	0.0600
ambient Ia G	0.3600	0.3527	0.0650	0.0650	0.0650
ambient Ia B	0.4000	0.3927	0.0850	0.0850	0.0850
ambient kd Ia R	0.1020	0.0999	0.0180	0.0180	0.0180
ambient kd Ia G	0.1080	0.1058	0.0195	0.0195	0.0195
ambient kd Ia B	0.1120	0.1100	0.0238	0.0238	0.0238

Input (from the code)	x / R	y / G	z / B
latitude phi, declination delta (deg)	22.8000	-4.6000	
sun intensity, key above (deg)	3.2000	-1.0000	
optical depth tau (R, G, B)	0.0600	0.1100	0.2100
moon direction (travel)	-0.5598	-0.3868	0.7328
moon colour	0.5500	0.6200	0.8000
moon intensity	0.9500		
spot position (m)	0.0000	7.3560	12.3700
spot direction	0.0000	-0.7660	-0.6428
spot colour	1	0.9500	0.8200
spot intensity, inner, outer (deg)	5	9	14
spot attenuation linear, quadratic	0.0500	0.0300	
night ambient (LightRig)	0.0600	0.0650	0.0850
day ambient (kDayAmbient)	0.3400	0.3600	0.4000
roof albedo kd (BNK-171 colour)	0.3000	0.3000	0.2800
roof ks, shininess n	0.0760	2	
roof point P (top face, m)	0	-0.4500	-1.3000
normal N	0	1	0
viewer: orbit eye (View_Projection)	16.0535	16.2357	25.6910

Model (display-referred Blinn-Phong path), per channel:
I = kd Ia + kd Il (N.L) + ks Il (N.H)^n
Not modelled: shadows, AO, relief, HDR sky SH ambient, moon tint.
Spot cone (shader): c = clamp((cos a - cos outer) / (cos inner - cos outer)), weight c^2 (not smoothstep).
Roof point: 20 deg off the spot axis at t = 0, outside the 14 deg cone. Column E: the up-facing point on the beam axis at roof height.

Sun key direction is quantised to 1 deg in the code.

Figure A.15: Workbook sheet `Illumination` (smallest text 7.1 pt at print size).

diffuse kd Il (N.L) R	0.8459	0.0286	0.0606	0.2484	0.0000
diffuse kd Il (N.L) G	0.8406	0.0181	0.0683	0.2360	0.0000
diffuse kd Il (N.L) B	0.7748	0.0068	0.0823	0.1901	0.0000
specular ks Il (N.H)^n R	0.2082	0.0254	0.0166	0.0385	0.0000
specular ks Il (N.H)^n G	0.2069	0.0161	0.0187	0.0366	0.0000
specular ks Il (N.H)^n B	0.2043	0.0064	0.0241	0.0316	0.0000
total I R	**1.1561**	**0.1539**	**0.0952**	**0.3050**	**0.0180**
total I G	**1.1555**	**0.1400**	**0.1065**	**0.2921**	**0.0195**
total I B	**1.0911**	**0.1232**	**0.1302**	**0.2455**	**0.0238**
luminance Y of total (Rec. 709)	**1.1510**	**0.1418**	**0.1058**	**0.2915**	**0.0195**

Ambient Ia = night^(1-w) day^w (DayCycle mixLog); spot columns: beams on at night only (sun below -2 deg). Linear HDR units (the sun's irradiance is 3.2).

Figure A.16: Workbook sheet `Illumination`, page 2 (smallest text 9.8 pt at print size).

Color_Interp: barycentric weights, Gouraud vs Phong, perspective-correct interpolation, sRGB blending, sky colour over the day

Legend: yellow = input from the code; white = formula.

Source: math/ExplorerMath edgeFunction / interpolateAffine / interpolatePerspective / srgbEncode (sections A-D: worked example values); section E: DayCycle solar model + export v6.day_cycle.grid (sky_zenith).

A. Screen triangle (worked example) with per-vertex data

vertex	x (px)	y (px)	clip w	tex u	N.x	N.y	N.z	I vertex
V0	100	100	1.0000	0.0000	-0.6012	-0.4008	0.6913	0.4148
V1	400	150	4.0000	1.0000	0.6007	-0.3004	0.7409	0.4445
V2	220	380	10.0000	0.5000	0.0000	0.6000	0.8000	0.4801
light	L = V = (0, 0, 1), H = (0, 0, 1)							
kd, ks, n	0.6000	0.6000	40					
2 x area (edge fn)	78000							

B. Barycentric weights at sample points: w0 = E12(p) / area, w1 = E20(p) / area, w2 = E01(p) / area

sample	px	py	w0	w1	w2	sum	inside
centroid	240	210	0.3333	0.3333	0.3333	1.0000	yes
near V1	360	160	0.0949	0.8410	0.0641	1.0000	yes
edge V0-V2 mid	160	240	0.5000	0.0000	0.5000	1.0000	yes
centre-left	200	180	0.5205	0.2359	0.2436	1.0000	yes
outside	420	330	-0.4744	0.7949	0.6795	1.0000	no

C. Gouraud (interpolate vertex I) vs Phong (interpolate N, normalise, then light)

sample	I Gouraud	N.x	N.y	N.z	|N|	I Phong	Phong - Gouraud
centroid	0.4465	-0.0002	-0.0337	0.7441	0.7448	1.1752	0.7288
near V1	0.4440	0.4482	-0.2522	0.7400	0.9011	0.4929	0.0489
edge V0-V2 mid	0.4474	-0.3006	0.0996	0.7457	0.8101	0.5740	0.1266
centre-left	0.4377	-0.1712	-0.1333	0.7295	0.7611	0.6851	0.2474

The highlight sits inside the triangle (interpolated N close to H): Phong finds it, Gouraud misses it.

D. Texture coordinate u: affine vs perspective-correct (clip w per vertex)

sample	u affine	u persp.	error
centroid	0.5000	0.2222	0.2778
near V1	0.8731	0.6852	0.1879
edge V0-V2 mid	0.2500	0.0455	0.2045
centre-left	0.3577	0.1178	0.2399

Perspective-correct: interpolate u / w and 1 / w linearly in screen space, then divide.

E. Blending two sRGB colours: naive (in sRGB) vs correct (in linear light)

colour	R (0-255)	G	B	R linear	G linear	B linear		
A	255	64	0	1.0000	0.0513	0.0000		
B	0	128	255	0.0000	0.2159	1.0000		
t	naive R	naive G	naive B	linear R	linear G	linear B	Y naive	Y linear
0.00	255.0	64.0	0.0	255.0	64.0	0.0	0.2493	0.2493
0.25	191.3	80.0	63.8	224.6	85.7	137.0	0.1721	0.2436
0.50	127.5	96.0	127.5	187.5	102.3	187.5	0.1446	0.2379
0.75	63.8	112.0	191.3	137.0	116.0	224.6	0.1644	0.2323
1.00	0.0	128.0	255.0	0.0	128.0	255.0	0.2266	0.2266

sRGB decode: c <= 0.04045 ? c/12.92 : ((c+0.055)/1.055)^2.4; encode = srgbEncode (ExplorerMath).

Y = relative luminance (Rec. 709). The naive mid-point is darker than the light-correct one.

F. Sky zenith colour over the day (solar model formulas; sky colour from the export) phi, delta: 22.8000 -4.6000

hour	e formula (deg)	e export (deg)	diff	daylight w	phase (no hysteresis)	zenith R	zenith G	zenith B	Y zenith	phase export
0.00	-71.8000	-71.8000	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
4.00	-29.3751	-29.3751	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
5.50	-8.6859	-8.6859	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
6.00	-1.7809	-1.7810	0.0000	0.4167	DAWN	0.0617	0.0753	0.1058	0.0746	DAWN
6.50	5.0981	5.0981	0.0000	0.9881	DAY	0.1885	0.2126	0.2329	0.2089	DAY
7.00	11.9319	11.9319	0.0000	1.0000	DAY	0.2206	0.2653	0.3436	0.2615	DAY
9.00	38.2196	38.2196	0.0000	1.0000	DAY	0.2852	0.4034	0.6962	0.3994	DAY
12.00	62.6000	62.6000	0.0000	1.0000	DAY	0.3089	0.4463	0.7643	0.4400	DAY
15.00	38.2196	38.2196	0.0000	1.0000	DAY	0.2852	0.4034	0.6962	0.3994	DAY
17.00	11.9319	11.9319	0.0000	1.0000	DAY	0.2206	0.2653	0.3436	0.2615	DAY
17.50	5.0981	5.0981	0.0000	0.9881	DAY	0.1885	0.2126	0.2329	0.2089	DAY
18.00	-1.7809	-1.7810	0.0000	0.4167	DUSK	0.0617	0.0753	0.1058	0.0746	DUSK
18.50	-8.6859	-8.6859	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
20.00	-29.3751	-29.3751	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT
22.00	-55.7779	-55.7779	0.0000	0.0000	NIGHT	0.0300	0.0400	0.0750	0.0404	NIGHT

Day sky: Preetham analytic sky (math/SkyModel), blended with the night preset in log space (mixLog, weight w).

The export applies 0.25 deg hysteresis to the phase, so DAWN / DUSK can last a little longer.

Figure A.17: Workbook sheet `Color_Interp` (smallest text 7.2 pt at print size).

Ray_Tracing: intersections, reflection, refraction, recursion (worked example)

Legend: yellow = input from the code; white = formula.

Source: Worked textbook example. The code: raytrace/Intersect (sphere / box / heightfield), Optics.h reflectDir, refractDir, fresnelDielectric; RayTracer.h whittedDepth = 5 (';' render modes).

1. Ray-sphere	x	y	z
ray origin o	0	2	-10
direction (unnormalised)	0.1000	-0.0500	1
d = normalize	0.0994	-0.0497	0.9938
sphere centre c	0	1	0
radius r	1.5000		
oc = o - c	0.0000	1.0000	-10.0000
a = d . d	1.0000		
b = 2 d . oc	-19.9755		
c' = oc . oc - r^2	98.7500		
discriminant b^2 - 4 a c'	4.0222		
t1 = (-b - sqrt(disc)) / 2a (near)	8.9850		
t2 = (-b + sqrt(disc)) / 2a (far)	10.9905		
hit p = o + t1 d	0.8929	1.5535	-1.0706
normal n = (p - c) / r	0.5953	0.3690	-0.7138
d . n	-0.6685		
reflection r = d - 2 (d . n) n	0.8953	0.4437	0.0395
check |r| (= 1)	1.0000		

2. Ray-plane (ground y = 0)	x	y	z
ray origin o	0	5	0
direction (unnormalised)	1	-1	0.5000
d = normalize	0.6667	-0.6667	0.3333
plane normal n	0	1	0
point on plane p0	0	0	0
n . d	-0.6667		
t = n . (p0 - o) / (n . d)	7.5000		
hit p = o + t d	5.0000	0.0000	2.5000
reflection r = d - 2 (d . n) n	0.6667	0.6667	0.3333

3. Snell refraction (air -> glass)	x	y	z
incident d (unnormalised)	0.5000	-1	0
d = normalize	0.4472	-0.8944	0.0000
normal n (facing against d)	0	1	0
n1, n2	1.0000	1.5000	
eta = n1 / n2	0.6667		
cos i = -d . n	0.8944		
k = 1 - eta^2 (1 - cos^2 i)	0.9111		
t = eta d + (eta cos i - sqrt k) n	0.2981	-0.9545	0.0000
incidence angle (deg)	26.5651		
refraction angle (deg)	17.3461		
Snell check n1 sin i - n2 sin t	0.0000		
Fresnel R (exact, fresnelDielectric)	0.0409		
Schlick R0 = ((n1 - n2) / (n1 + n2))^2	0.0400		

4. Total internal reflection (glass -> air, eta = 1.5)

incidence angle (deg)	k = 1 - eta^2 sin^2 i	TIR?	Fresnel R
critical angle asin(1/1.5)	41.8103		
20	0.7368	no	0.0417
30	0.4375	no	0.0552
41	0.0316	no	0.3798
42	-0.0074	yes	1.0000
60	-0.6875	yes	1.0000

5. Whitted recursion: weight of bounce k = kr^k

reflectance kr, depth limit (whittedDepth)	0.5000	5	
bounce k	kr^k	cumulative	share of the infinite sum
0	1.0000	1.0000	0.5000
1	0.5000	1.5000	0.7500
2	0.2500	1.7500	0.8750
3	0.1250	1.8750	0.9375
4	0.0625	1.9375	0.9688
5	0.0313	1.9688	0.9844
6	0.0000	1.9688	0.9844

Bounces past the limit (k > 5) contribute 0; the lost share of the infinite sum is kr^6.

Figure A.18: Workbook sheet `Ray_Tracing` (smallest text 8.0 pt at print size).

Thermal_Kernels: Gaussian and Sobel kernels on a 5x5 thermal patch

Legend: yellow = input from the code; white = formula.

Source: math/ExplorerMath.cpp kernel(): Gaussian 5x5 = binomial [1 4 6 4 1] (x) [1 4 6 4 1] / 256 (sigma ~ 1), Sobel Gx / Gy, convolve() (correlation, kernel row 0 = row above, clamp-to-edge), applyGamma v^(1/g).

sigma	1.0000	size 5 (offsets -2..2)

threshold T on |G|	2.0000

Gaussian exp(-(x^2+y^2)/(2 s^2))

0.0183	0.0821	0.1353	0.0821	0.0183
0.0821	0.3679	0.6065	0.3679	0.0821
0.1353	0.6065	1.0000	0.6065	0.1353
0.0821	0.3679	0.6065	0.3679	0.0821
0.0183	0.0821	0.1353	0.0821	0.0183
sum	1.0000			

normalised (sum = 1)

0.0030	0.0133	0.0219	0.0133	0.0030
0.0133	0.0596	0.0983	0.0596	0.0133
0.0219	0.0983	0.1621	0.0983	0.0219
0.0133	0.0596	0.0983	0.0596	0.0133
0.0030	0.0133	0.0219	0.0133	0.0030
max |normalised - code|				0.0215

code: b_y b_x / 256

0.0039	0.0156	0.0234	0.0156	0.0039
0.0156	0.0625	0.0938	0.0625	0.0156
0.0234	0.0938	0.1406	0.0938	0.0234
0.0156	0.0625	0.0938	0.0625	0.0156
0.0039	0.0156	0.0234	0.0156	0.0039
sum	1.0000			

binomial
1
4
6
4
1

Gamma: out = v^(1/g)

v	g = 0.5	g = 1.0	g = 2.2
g:	0.5	1.0	2.2
0.0	0.0000	0.0000	0.0000
0.1	0.0100	0.1000	0.3511
0.2	0.0400	0.2000	0.4812
0.3	0.0900	0.3000	0.5785
0.4	0.1600	0.4000	0.6594
0.5	0.2500	0.5000	0.7297
0.6	0.3600	0.6000	0.7928
0.7	0.4900	0.7000	0.8503
0.8	0.6400	0.8000	0.9035
0.9	0.8100	0.9000	0.9532
1.0	1.0000	1.0000	1.0000

g > 1 lifts dark tones (applyGamma).

5x5 thermal patch (worked example, 0..1)

0.20	0.22	0.25	0.30	0.32
0.21	0.24	0.60	0.65	0.34
0.22	0.30	0.85	0.90	0.40
0.20	0.28	0.70	0.75	0.36
0.19	0.21	0.26	0.31	0.30

Sobel Gx

-1	0	1
-2	0	2
-1	0	1

Sobel Gy

-1	-2	-1
0	0	0
1	2	1

Gx step: patch x kernel at the centre

-0.2400	0.0000	0.6500
-0.6000	0.0000	1.8000
-0.2800	0.0000	0.7500
sum	2.0800	SUMPRODUCT:

2.0800

Gaussian 5x5 at the centre = SUMPRODUCT(patch, normalised kernel)	0.5545	code kernel:	0.5379	centre input:	0.8500

Gx (3x3 valid output)

1.4600	1.5000	-0.9000
2.1500	2.0800	-1.5000
1.7000	1.6400	-1.0900

Gy (3x3 valid output)

0.7800	1.8800	1.8800
0.1700	0.3400	0.3200
-0.8000	-1.8600	-1.8700

\|G\| = sqrt(Gx^2 + Gy^2)

1.6553	2.4051	2.0843
2.1567	2.1076	1.5338
1.8788	2.4798	2.1645

edge = \|G\| >= T

0	1	1
1	1	0
0	1	1

Valid region: output (i, j) uses patch rows i..i+2, cols j..j+2 (the code clamps at the border).

The Frame Explorer shows Gx, Gy with bias 0.5 (zero = mid-grey); Sobel \|G\| uses both kernels.

Figure A.19: Workbook sheet `Thermal_Kernels` (smallest text 7.5 pt at print size).

Duty_Cycle: 24-hour roster of the 8 on-duty soldiers (2 shifts, not 6-hour shifts)

Legend: yellow = input from the code; white = formula.

Source: scene/Garrison.h kDayShiftStart 6.4, kDayShiftEnd 17.6, dayShift(); Garrison.cpp makePrograms() and shiftDelay = 1 + 2.5 rank s; rosters from ts --export-docs v6.garrison.soldiers.

day start (h)	6.40	day end (h)	17.60	day shift	06:24-17:36	hours	11.20	night hours	12.80	DayBreak, Dus	6.45	17.60

#	role	day roster (export)	night roster (export)	day	night
1	SENTRY	guard: inner gate and gate road (INNER GATE -> GATE ROAD)	guard: inner gate (INNER GATE)	guard	guard
2	WATCH	guard: watchtower and east wood look-out (WATCHTOWER -> EAST WOOD)	guard: sandbag guard post (SANDBAG POST)	guard	guard
3	GUNNER	walk: patrol loop on the east apron	guard: machine-gun nest (MG NEST)	walk	guard
4	MECHANIC	work: wrench at the parked tank and the helicopter (MBT-2 ENGINE DECK -> HELICOPTER)	off duty: bunk room (south bunk) (BUNK ROOM)	work	off
5	LOGISTICS	work: carry crates from the depot to the ammunition bunker (FUEL DEPOT -> AMMO BUNKER)	off duty: bunk room (mess table) (MESS BENCH)	work	off
6	TECHNICIAN	work: inspect the radar and the AA gun (RADAR -> AA GUN PIT)	work: plotting table in the command room (COMMAND ROOM)	work	work
7	PATROL	walk: patrol loop inside the compound	guard: east guard tower (GUARD TOWER E)	walk	guard
8	RUNNER	run: running loop on the south apron	off duty: bunk room (north bunk) (BUNK ROOM)	run	off

24-hour grid: state at the middle of each hour = IF(day start <= h + 0.5 < day end, day kind, night kind)

#	role	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1	SENTRY	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard
2	WATCH	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard	guard
3	GUNNER	guard	guard	guard	guard	guard	guard	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	guard	guard	guard	guard	guard	guard
4	MECHANIC	off	off	off	off	off	off	work	work	work	work	work	work	work	work	work	work	work	work	off	off	off	off	off	off
5	LOGISTICS	off	off	off	off	off	off	work	work	work	work	work	work	work	work	work	work	work	work	off	off	off	off	off	off
6	TECHNICIAN	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work	work
7	PATROL	guard	guard	guard	guard	guard	guard	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	walk	guard	guard	guard	guard	guard	guard
8	RUNNER	off	off	off	off	off	off	run	run	run	run	run	run	run	run	run	run	run	run	off	off	off	off	off	off
on duty (not off)		5	5	5	5	5	5	8	8	8	8	8	8	8	8	8	8	8	8	5	5	5	5	5	5
shift		NIGHT	NIGHT	NIGHT	NIGHT	NIGHT	NIGHT	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	DAY	NIGHT	NIGHT	NIGHT	NIGHT	NIGHT	NIGHT

Colours: grey off duty, red guard, blue work, green walk (patrol), yellow run.

Staggered shift change: departure delay = 1 + 2.5 rank (s, simulation time; rank = seeded order)

rank	0	1	2	3	4	5	6	7
delay (s)	1.0	3.5	6.0	8.5	11.0	13.5	16.0	18.5

The day shift follows the sun (DayBreak about 06:27, Dusk 17:36 at latitude 22.8 deg). Two shifts; the spec's 6-hour shifts are not in the code.

Figure A.20: Workbook sheet `Duty_Cycle` (smallest text 7.2 pt at print size).

Performance: benchmark import (report/data/bench.csv)

Legend: yellow = input from the code; white = formula.

Source: report/data/bench.csv written by the benchmark run (ts --benchmark / --bench-shots); this sheet is rebuilt by tools/v10/build_cg_workbook.py.

IMPORT AREA: rows below are copied from bench.csv (yellow); derived columns are formulas.

config	avg_ms	p95_ms	avg_fps	draw_calls	triangles	1000 / avg_ms (fps)	fps check (diff)	triangles per draw
EPIC day 10:30	14.87	17.71	67.25	256	5725520	67.3	0.0	22352
EPIC night 23:00 (game: waves, UAVs)	16.38	20.82	61.05	970	5798500	61.0	0.0	5975
EPIC day, thermal view	10.72	14.80	93.31	71	2726600	93.3	0.0	38277
EPIC night, thermal view	12.62	17.62	79.24	67	2499130	79.2	0.0	37309
EPIC day, shadows off	16.42	21.02	60.89	206	5857520	60.9	0.0	28474
EPIC night, shadows off	17.18	22.03	58.22	905	5867990	58.2	0.0	6480
EPIC day, non-indexed	54.47	59.47	18.36	4046	5855790	18.4	0.0	1447
ULTRA day	16.76	22.08	59.67	257	4307730	59.7	0.0	16746
HIGH day	15.11	17.25	66.20	169	1411360	66.2	0.0	8365
LOW day	14.16	16.04	70.61	165	1315170	70.6	0.0	7950
v1 look (no post) day	13.01	14.69	76.87	148	1341050	76.9	0.0	9061

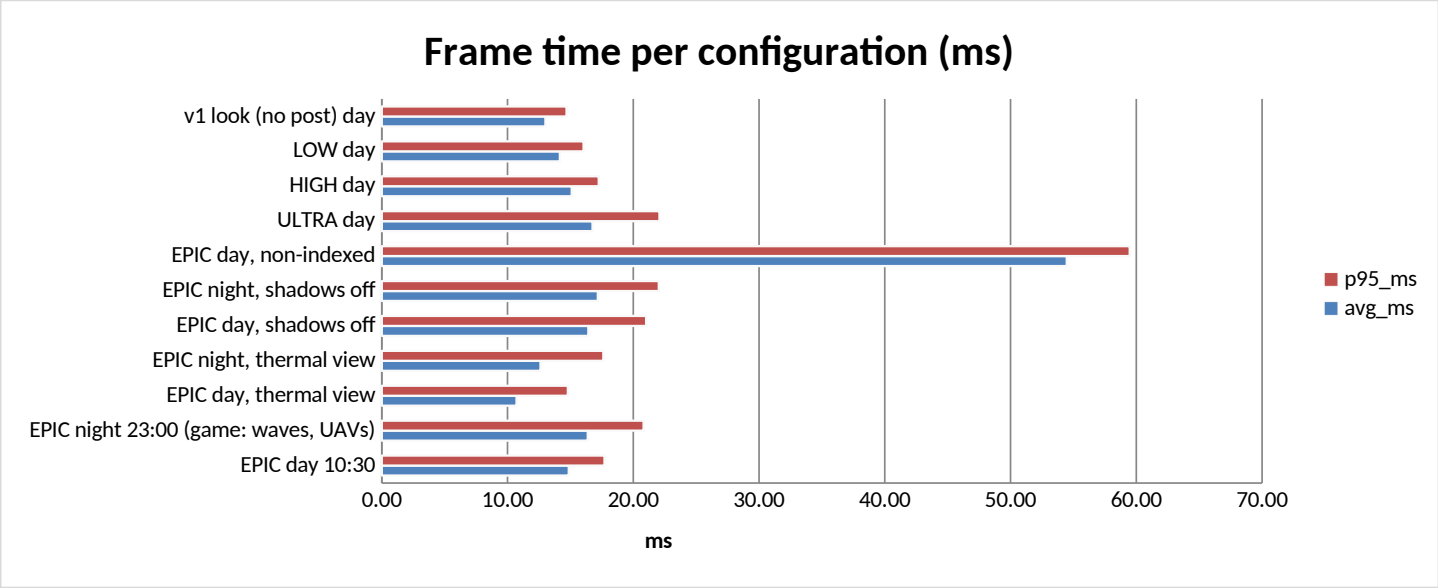


Figure A.21: Workbook sheet `Performance` (smallest text 8.7 pt at print size).